AUTOMATION TESTING

PLAYWRIGHT with TYPESCRIPT

Module 1: Introduction to Automation Testing & Playwright

+**Automation Testing Overview & Process**

-What is automation testing

-Advantages of Automation Testing

-Limitations of Automation Testing

-Difference between Manual & Automation Testing

-Why Automation Testing is important

-When to start Automation Testing

-How to start Automation [Automation Testing Process]

-Types of Automation Tools (Licensed Tools vs Open-Source Tools)

-Why Playwright over Selenium & Cypress

+**Introduction to Playwright**

-What is Playwright

-Playwright Features

-Playwright Architecture

-Supported Browsers [Chromium, Firefox, WebKit]

-Playwright vs Selenium

Module 2: TypeScript

+**TypeScript Introduction**

-Introduction (History) to TypeScript

-Why TypeScript for Automation Testing

-TypeScript vs JavaScript

-Installing Node.js

-Installing TypeScript

-Installing Visual Studio Code [IDE Support]

-Writing the First TypeScript Program

-Compile and Execute a TypeScript Program

-Compiling .ts to .js

+ TypeScript Basics

-Variables in TypeScript

-Data Types in TypeScript

+ Arrays, Strings & Functions

-Arrays in TypeScript

-Strings in TypeScript

-Array Methods

- Tuples

-Function Declaration

-Function Parameters

-Return Functions

-Optional Parameters

-Arrow Functions

- async / await

+ OOPS Concepts in TypeScript

-Classes and Objects

-Methods in TypeScript

-Keywords in TypeScript -Identifiers in TypeScript -Constants (const) in TypeScript -Comments in TypeScript -Type Assertion in TypeScript -Operators in TypeScript **+ Control Flow Statements** -Decision Making Statements [if, if–else, else–if, switch] -Looping Statements [for, while, do–while] -Jump Statements [break, continue, return]	-Access Modifiers -Constructors in TypeScript -Inheritance in TypeScript -Method Overloading in TypeScript -Method Overriding in TypeScript -Polymorphism in TypeScript -Abstraction in TypeScript -Interface in TypeScript -Encapsulation in TypeScript -Exception Handling in TypeScript
Module 3: Playwright Setup & Core Concepts	
+ Introduction to Playwright -Installing Playwright -Playwright Project Structure -Installing Browsers -Launching Chrome, Edge, Firefox and Webkit -Running first test -Headless & Headed Execution -Debugging Playwright Tests -Verify Page Title and URL (Title & URL Verification) **+ Browser Concepts** -Browser Context -Pages & tabs -Handling Multiple Pages -Multiple browser sessions -Playwright.config.ts Explanation	**+Web Elements & User Actions** **-Locators & Selectors in Playwright** -Text Locator -CSS Selector -XPath -Role Locator -Test ID Locator **+ Web Elements Handling** -Input Fields -Links -Buttons -Checkboxes -Radio Buttons -Drop-Downs -Auto Suggestions - Handling Web Tables

	- File upload and Download - Capture Screenshots -Video Recording
Module 4: Frames, Actions, Assertions and Waits	
+Handling Frames, Alerts and Windows -Handling Frames & iframes -Handling Multiple Tabs -Window switching -Handling Popups -File Uploads and File Downloads -Handling Alerts -Handling Confirmation Messages -Shadow DOM **+Mouse & Keyboard Actions** -Keyboard Actions -Mouse Actions -Mouse Hover -Click, Double click -Right Click -Drag and Drop	**+Assertions & Validations** -Hard Assertions -Soft Assertions -Page Assertions -Element Assertions -Text Validation -Attribute Validation -URL & Title Validation -Checkbox & Radio validation **+Wait Mechanisms in Playwright** -Auto waits -Explicit waits -Waiting for element -Waiting for navigation -Waiting for API response -Waiting for page load
Module 5: Framework Design	
Playwright Test Framework **+Playwright Test Runner** -Built-in Test Runner Architecture -Test Discovery (*.spec.ts) -CLI Execution (npx playwright test) -Parallel Execution (Workers) -HTML / JSON Reports -Assertions (Auto-wait enabled) -Test Lifecycle Hooks	**+Reporting & Logging** -Playwright HTML Reports -Screenshot on Failure -Video on Failure -Trace Report Analysis **+Cucumber BDD Framework** -Overview of BDD and Cucumber -How to install and setup Cucumber with - Playwright + VS Code

+Test Execution Control -Test Suite -Grouping Tests -Skipping Tests -Test Tags (@smoke, @regression) -Running Tests by Tag (--grep) -Parallel Execution (Workers) -Retry Failed Tests -Test & Global Timeouts **+Data-Driven Framework** -Data Driven Testing Concept -Reading Test Data from JSON -Reading Test Data from CSV -Reading Test Data from Excel -Parameterized Tests -Executing Data Driven Tests -Analysing Test Results **POM FRAMEWORK** -What is Page Object Model -Adding Test and Pages -Locators in Page Object Model -Implement Exception Handling -Implement Wrapper methods. -Page Object Manager -Parametrized Methods	-Overview of Gherkin keywords -How to create Feature file -How to generate Step Definition file -How to integrate Cucumber with - Playwright Automation Engine **+Test Runner Configuration (Playwright BDD)** -Cucumber CLI Test Runner configuration -Data Driven Testing in Cucumber -Configure Cucumber with npm and Jenkins -How to generate Reports in Cucumber **+Working with API** -API Fundamental -Restful API Payload and Verbs -CRUD Operation -API Authentication -API Request -API Response -Validating API Response -Network _interception. **+Final Project** -Playwright + POM + Data Driven + Jenkins + Reports
Module 6: GIT and JENKINS	
+ GIT and GIT HUB -Install & Configure Git -Create Local & Remote Repository -Git Commands	**+Jenkins** -Installing/Configuring Jenkins -Scheduling Test Execution in Jenkins -Auto mail configuration in Jenkins

-Create GitHub Account -Configure Git & GitHub with VS Code -Branching Concepts	-What is continues integration? -Continues integration with Jenkins

Module 7: Real Time Interview Questions and Answers

-Real Time Playwright Interview Questions & Answers

-Frameworks Interview Questions & Answers

- Scenario-Based Interview Questions & Answers

-GIT and GITHUB Interview Questions & Answers

-CI/CD and Jenkins Interview Questions & Answers

Module 8: Project Implementation

-End To End Testing on E-commerce Applications.

-End To End Testing on Banking Applications.

-End To End Testing on Social Media Applications.

-End To End Testing on Ticket Booking Applications.

-End To End Testing on Education Applications.

-End To End Testing on Online Meeting Applications.

****

AUTOMATION TESTING

PLAYWRIGHT with TYPESCRIPT

MODULE-1: Introduction to Automation Testing & Playwright

1. Automation Testing Overview & Process

1.1 What is Automation Testing:

Automation Testing is a software testing technique in which test cases are executed using automation tools instead of manual effort. In this approach, testers write scripts or programs that simulate user actions on the application and verify expected results automatically. In manual testing, a tester performs each step manually, such as clicking buttons, entering data, and checking outputs. In automation testing, these steps are written once in the form of scripts and can be executed multiple times without human intervention.

Automation testing is mainly used for:

- Repeated test execution
- Regression testing
- Large test suites
- Multi-browser and multi-environment testing
- Automation testing helps ensure that the application behaves correctly even after frequent code changes.

1.2 Advantages of Automation Testing

Automation testing offers several advantages that make it essential in modern software development.

1. Faster Execution: Automation scripts execute much faster than manual test cases. Hundreds of test cases can be executed in a short time, which is not possible manually.

2. Reusability of Test Scripts: Once automation scripts are created, they can be reused across multiple test cycles, releases, and environments.

3. Improved Accuracy: Automation eliminates human errors that may occur due to fatigue, distraction, or repetitive tasks.

4. Consistent Test Execution: Automation executes the same steps in the same manner every time, ensuring consistent test results.

5. Supports Parallel Testing: Automation tools can run multiple tests simultaneously on different browsers and systems.

6. Cost Effective in the Long Run: Although initial setup requires effort and investment, automation reduces overall testing cost over time.

1.3 Limitations of Automation Testing:

1. High Initial Investment: Automation requires time to design frameworks, write scripts, and set up tools.

2. Script Maintenance: Whenever the application UI or workflow changes, automation scripts must be updated.

3. Not Suitable for All Test Types: Automation is not effective for exploratory testing, usability testing, and ad-hoc testing.

4. Requires Programming Knowledge: Testers must have coding skills to write and maintain automation scripts.

Automation should be applied only where it provides real value.

1.4 Difference between Manual Testing & Automation Testing

- Manual testing is performed by human testers, whereas automation testing is executed using tools and scripts.
- Manual testing is slower and suitable for understanding application behavior, while automation testing is faster and suitable for repetitive and regression tests.
- Manual testing is flexible and exploratory in nature. Automation testing is structured and predefined.
- Both testing types are important and are used together in real projects.

1.5 Why Automation Testing is Important

- Has frequent releases.
- Undergoes continuous changes.
- Requires fast feedback.
- Without automation, testing becomes time-consuming and delays release cycles. Automation ensures that critical functionalities continue to work even after code changes.
- Automation also supports Continuous Integration and Continuous Delivery (CI/CD) pipelines by providing quick test feedback.

1.6 When to Start Automation Testing

- Application requirements are stable.
- Manual test cases are ready.
- Application functionality is not changing frequently.
- Automation should not start in early development phases where frequent changes occur.
- Poor timing leads to high maintenance and wasted effort.

1.7 How to Start Automation Testing

The automation testing process includes the following steps:

- **Identify Automation Test Cases:** Select test cases that are repetitive, stable, and business-critical.
- **Select Automation Tool:** Choose a tool based on application technology, budget, and team expertise.
- **Design Automation Framework:** Create a structured framework that defines folder structure, standards, and reporting.

- **Develop Test Scripts:** Write automation scripts using the selected programming language and tool.
- **Execute Tests:** Run scripts locally or through CI tools.
- **Analyze Test Results:** Review execution reports, failures, and logs.
- **Maintain Scripts:** Update scripts as application changes.

1.8 Types of Automation Tools: [Licensed Tools vs Open-Source Tools]

Licensed (Commercial) Tools: Licensed tools are paid automation tools that require a license or subscription for usage. They are commonly used by enterprises for large-scale automation and testing.

- Licensed tools require payment for usage.
- Examples include enterprise-level automation tools.
- Professional support
- Advanced features

Disadvantages:

- High cost
- License dependency
- Open-Source Tools
- Open-source tools are free and community-driven.

Examples:

- Micro Focus UFT One
- Tricentis Tosca
- Katalon Studio

Open-Source Tools: Open-source automation tools are generally free to use and are developed and maintained by the open-source community and organizations.

- No license cost
- High flexibility and customization
- Large community support
- Source code is publicly available

Disadvantages:

- No guaranteed official/vendor support in many cases
- May require skilled users for setup and maintenance
- Advanced enterprise features may require additional tools or configuration

Examples:

- Selenium
- Playwright
- Cypress

2.3 Difference between Playwright and Selenium

Playwright	Selenium

Uses a modern architecture with direct communication to browser engines	Uses browser-specific drivers like ChromeDriver and GeckoDriver
Built-in auto-waiting for elements and actions	Requires explicit waits for synchronization
Supports Chromium, Firefox, and WebKit using one single API	Supports multiple browsers but needs separate drivers for each
Faster execution with better stability	Slower execution and more prone to flaky tests
Comes with built-in test runner and reporting	Depends on external frameworks for execution and reporting
Parallel execution is supported out of the box	Parallel execution needs additional setup and tools
Designed specifically for modern and dynamic web applications	Older architecture with limited native support for modern web apps

****

MODULE-2

JavaScript:

Brendan Eich created the language in about 10 days at **Netscape** in May **1995**. It was originally named **Mocha**, briefly renamed to **LiveScript** in September, and finally rebranded to **JavaScript** in December 1995 through a marketing partnership between **Netscape** and **Sun Microsystems**, partly to benefit from the massive popularity of Java.

Because of this partnership, Sun Microsystems registered and held the JavaScript trademark. The USPTO [United States Patent and Trademark Office.] record shows Sun Microsystems, Inc. as the owner of the JAVASCRIPT registration.

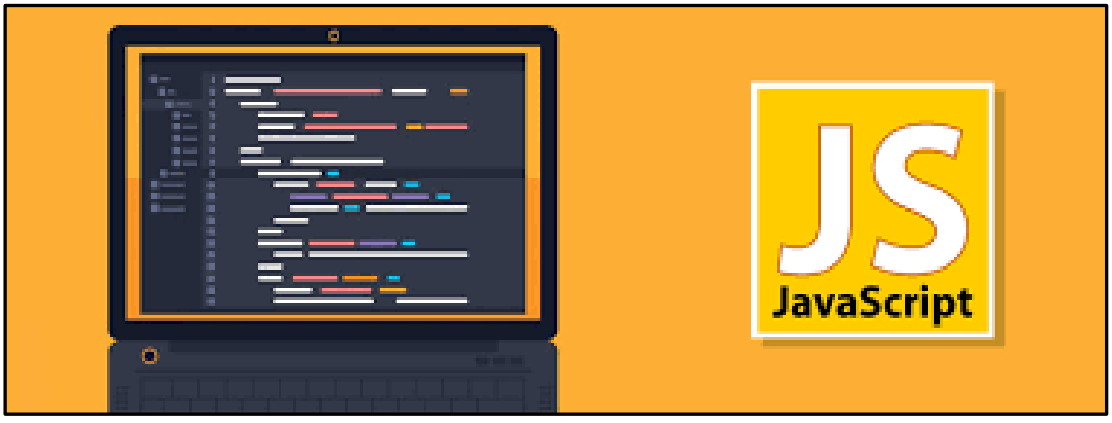
Oracle acquired Sun Microsystems in a deal finalized in January 2010. Through that acquisition, Oracle inherited Sun's intellectual property, including the JavaScript trademark.

Important: JavaScript and Java are two completely different programming languages. The name "JavaScript" was primarily a marketing decision associated with Java's popularity, **not** because JavaScript was a version of Java.

JavaScript started its life as a simple scripting language for browsers. At the time it was invented, it was expected to be used for short snippets of code embedded in a web page — writing more than a few dozen lines of code would have been somewhat unusual. Due to this, early web browsers executed such code pretty slowly. Over time, though, JS became more and more popular, and web developers started using it to create interactive experiences.

JavaScript is standardized at **ECMA** International — the European association for standardizing information and communication systems (ECMA was formerly an acronym for the **European Computer Manufacturers Association**) to deliver a standardized, international programming language based on JavaScript. This standardized version of JavaScript, called ECMAScript, behaves the same way in all applications that support the standard.

More than this, JS has become popular enough to be used outside the context of browsers, such as implementing JS servers using node.js. The “run anywhere” nature of JS makes it an attractive choice for cross-platform development. There are many developers these days that use only JavaScript to program their entire stack!

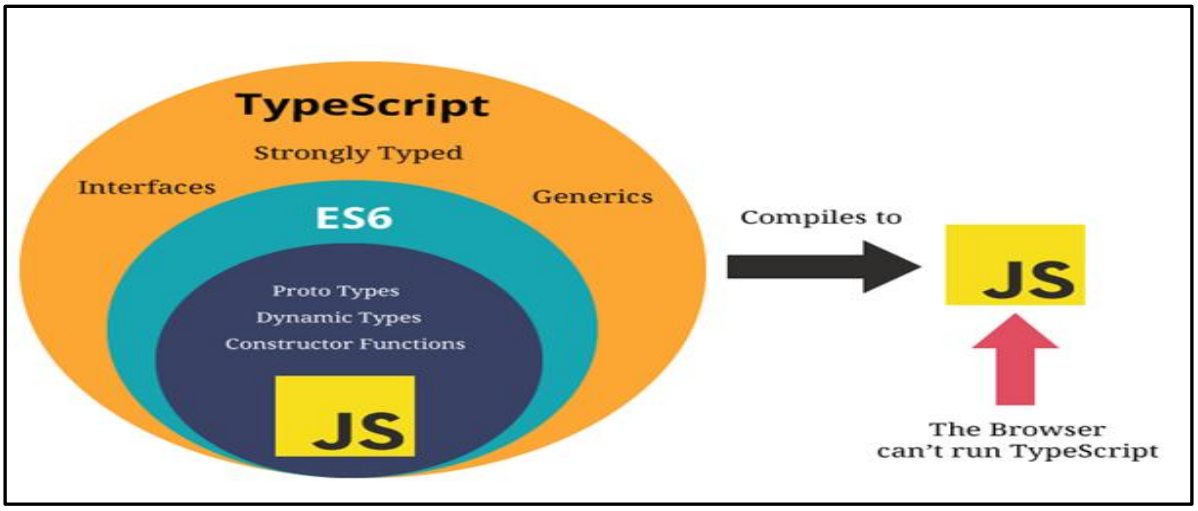


Features of JS:

1. **Interpreted Language:** JavaScript code doesn't require prior compilation and can be executed directly.
2. **Dynamic Typing:** As mentioned earlier, JavaScript variables can hold values of any type, and their types can also change during runtime.
3. **Event-Driven Programming:** JavaScript is purpose-built to respond to user actions and events, like clicks, key presses, and page loads. This event-driven responsiveness is crucial to create engaging user experiences.
4. **Asynchronous Programming:** JavaScript uses asynchronous constructs, like callback functions and promises, for non-blocking execution. This enables it to handle multiple tasks concurrently.
5. **Widely Supported:** JavaScript enjoys near-universal support across all modern web browsers and web development frameworks (frontend and backend).

TypeScript:

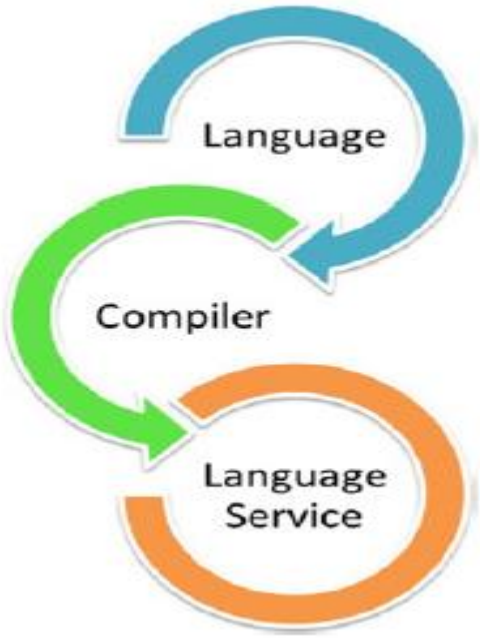
"TypeScript is JavaScript for application-scale development." TypeScript is a strongly typed, object oriented, compiled language. It was designed by **Anders Hejlsberg** at **Microsoft**. TypeScript is both a language and a set of tools. TypeScript is a typed superset of JavaScript compiled to JavaScript. In other words, TypeScript is JavaScript plus some additional features.



TypeScript and ECMAScript:

The ECMAScript specification is a standardized specification of a scripting language. There are six editions of ECMA-262 published. Version 6 of the standard is codenamed "Harmony". TypeScript is aligned with the ECMAScript6 specification.

Components of TypeScript: TypeScript has the following three Components:

Language: It comprises of the syntax, keywords, and type annotations. Because it is a superset of JavaScript, any valid JavaScript code is also valid TypeScript code, but the added type annotations allow developers to catch errors early during development.	
The TypeScript Compiler: The TypeScript compiler (tsc) converts the instructions written in TypeScript to its JavaScript equivalent.	
The TypeScript Language Service: The "Language Service" exposes an additional layer around the core compiler pipeline that are editor-like applications. The language service supports the common set of a typical editor operations like statement completions, signature help, code formatting and outlining, colorization, etc.	

Features of TypeScript:

1) TypeScript is just a JS: TypeScript code doesn't directly run on the browser. A program written in TypeScript starts with JavaScript and ends with JavaScript. Therefore, the code written in TypeScript can be compiled and converted to JavaScript equivalent for execution. This process is known as trans-piled, in which the browser can read TypeScript code and display the desired output.

2) Portable: TypeScript is portable as the code written in TypeScript can be run on any browser or operating system. It can run in any environment in which JavaScript can run.

3) Object Oriented Programming Language: TypeScript provides support for Object Oriented Programming concepts like Classes, Interfaces, Inheritance, etc. For Development, TypeScript code is valid for both client-side and server-side rendering.

4) Compilation: JavaScript is an interpreted language. Hence, it needs to be run to test that it is valid. It means you write all the codes just to find no output, in case there is an error. Hence, you have to spend hours trying to find bugs in the code. The TypeScript compiler (transpiler) analyses code and provides compile-time error checking. TypeScript will compile the code and generate compilation errors, if it finds some sort of syntax errors. This helps to highlight errors before the script is run.

Note: A transpiler converts code from one programming language to another language of the same level. TypeScript ⟶ transpiler(compiler) ⟶ JavaScript

5) Strong Static Typing: TypeScript has the feature of strong static typing which comes through TLS (TypeScript language service). If a variable is declared with no type specified then TLS can conclude it based on its value.

6) TypeScript Supports JavaScript Libraries: As it is a superset of JavaScript, all the libraries and existing JavaScript code are valid TypeScript code as well. As a result, Developers can access and use all the JavaScript libraries, tools, and frameworks easily.

7) Supports Type Definitions: It supports type definitions for existing JavaScript libraries. TypeScript definition file that has the .d.ts extension defines external JavaScript libraries.

8) JavaScript is TypeScript: It is a widely used TypeScript feature in which the code is written in JavaScript with a .js extension that can be converted to TypeScript by changing the extension from **.js to .ts**.

Differences Between JavaScript and TypeScript:

JavaScript	TypeScript
JavaScript was developed in 1995 by Brendan Eich while working at Netscape Trademark owner: **Oracle**	TypeScript was created by Microsoft in the year 2012. Trademark owner: **Microsoft**
Dynamically typed language: You don't have to specify the type of a variable. JavaScript determines the type from the value. Example: let x = 10;	**Statically typed language:** Statically typed — You can specify the type of a variable. Example: let x: number = 10;. TypeScript checks whether the value matches the declared type.
Types are mainly checked at runtime: Type checking at runtime — Many type-related problems are discovered when the JavaScript program actually runs.	**Many types are checked before execution:** Type checking before execution — TypeScript can identify many type-related problems while writing/compiling the code, before the program runs.
JavaScript code can run directly in browsers and Node.js: Runs directly — JavaScript can be executed by browsers and JavaScript runtimes such as Node.js.	**TypeScript code is converted to JavaScript first:** Converted to JavaScript first — Browsers and Node.js normally execute JavaScript, so TypeScript is first transpiled/compiled into JavaScript.
File extension is .js: .js file — JavaScript source files normally use the .js extension.	**File extension is .ts:** .ts file — TypeScript source files normally use the .ts extension.
Types usually don't need to be specified: Types don't normally need to be specified — let name = "Krishna"; is enough.	**Types can be explicitly specified, e.g. let age: number:**

JavaScript determines that name contains a string.	Types can be specified — let name: string = "Krishna";. Here string tells TypeScript what type of value is expected.
Supports OOP concepts such as classes and inheritance: Supports OOP — JavaScript supports classes, objects, inheritance, encapsulation, and polymorphism.	**Supports OOP plus interfaces, access modifiers, abstract classes, etc.** Additional OOP/type features — TypeScript supports JavaScript OOP and adds features such as interfaces, access modifiers (public, private, protected), and abstract classes.
More flexible with variable types: More flexible — A variable can hold different types of values during execution. **Example**: let value = 10; value = "Hello";	**Provides stronger type safety:** Stronger type safety — If a variable is declared as number, TypeScript can prevent assigning a string to it. **Example**: let value: number = 10; value = "Hello"; → type error.
Some errors are discovered only when the code runs: Errors can appear during execution — Some problems are not discovered until the particular code is executed.	**Many type-related errors are discovered before the code runs:** Many errors can be found earlier — TypeScript's type checker can detect many problems before the application is executed.
Used in many areas: JavaScript is used for web applications, backend applications, mobile, desktop, and games.	**Same application areas:** TypeScript can also be used for web, backend, mobile, desktop, etc. It is especially useful for large and complex projects because of static typing.

Key Differences between JavaScript and TypeScript

JavaScript	TypeScript
JavaScript is a scripting language which helps you create interactive web pages.	Typescript is Object oriented Programming language & superset of JavaScript.
JavaScript code doesn't need to compile.	Typescript code needs to be compiled.
There is no static typing. Ex: var num	There is static typing. We can declare variables with data types. Ex: var num : number
It does not have interfaces	It has interface
It has no optional parameter feature	It has optional parameter feature
It has no Rest Parameter feature	It has Rest Parameter feature
Generics not supported	Generics supported
Modules not supported	Modules supported
number, string are the objects	Number, string are the interfaces

When to use Typescript vs when to use JavaScript? [Dev or QA]

The decision between TypeScript and JavaScript isn't about picking a winner but finding the right tool for the project. Here's a quick guide to help you make an informed choice each time.

Use TypeScript if: [Dev]	**Use JavaScript if: [Dev]**
▪ You are building large-scale projects, especially those with multiple collaborators. ▪ You are working with complex data structures and APIs. ▪ You want to leverage modern IDE tooling and third-party libraries for maximum developer productivity.	▪ You are building quick prototypes or small projects. ▪ You are working with specific libraries or frameworks that don't have full TypeScript support. ▪ You have an experienced team of JavaScript developers who prefer JavaScript's flexibility and can write safe JS code.
Use TypeScript if: [QA]	**Use JavaScript if: [QA]**
▪ You are building large-scale automation frameworks (Playwright, Selenium, Cypress). ▪ You are working in a team with multiple testers contributing to the same framework. ▪ You are handling complex test data, APIs, and Page Object Models (POM). ▪ You want early error detection before test execution (compile-time checking). ▪ You want better IDE support like auto-completion, navigation, and refactoring.	▪ You are building quick automation scripts or small projects. ▪ You are creating Proof of Concept (PoC) or demo test cases. ▪ You are learning automation testing concepts as a beginner. ▪ You are working with simple frameworks without complex architecture. ▪ You prefer flexibility and your team is strong in JavaScript.

→ For example, let's look at a basic JavaScript function that adds two numbers:

```
function add(a, b) {
    return a + b;
}
```

→ To convert this to TypeScript, we can add type annotations to guarantee that both a and b are numbers:

```
function add(a: number, b: number): number {
  return a + b;
}
```

→ Problem in JavaScript: JavaScript's dynamic typing means that types are determined at

runtime, which can lead to bugs that are hard to track down. For example, mixing up types can lead to unexpected behavior or runtime errors.

```
function add(a, b) {
       return a + b;
    }

    console.log(add(5, "10")); // Outputs "510" instead of 15
```

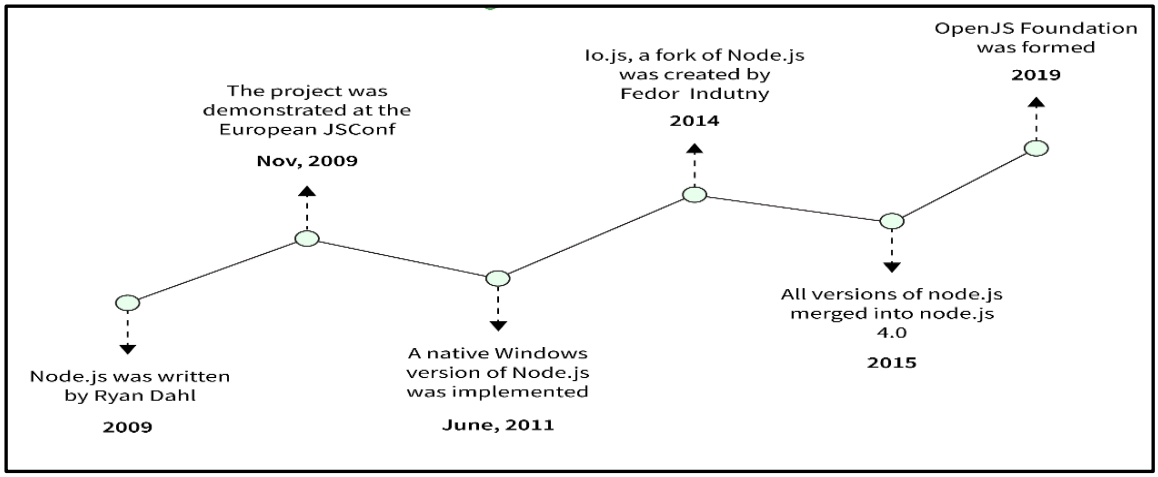
→ Solution in TypeScript: TypeScript introduces static type checking, allowing developers to catch errors at compile time long before the code is executed.

```
function add(a: number, b: number): number {
      return a + b;
    }

    console.log(add(5, "10")); // Error: Argument of type 'string' is not assignable to parameter of type
```

Node.js:

Node.js is an open-source framework for building fast and scalable server-side JavaScript applications. In 2009, a software engineer at **Google, Ryan Dahl**, created Node.js framework. Built on the V8 JavaScript runtime. In its first version, it only supported macOS and Linux, but eventually, other operating systems were also supported. Ryan Dahl led the development and maintenance of Node.js and was later supported by Joyent. Ryan Dahl created the Node.js framework in 2009. Built on the V8 JavaScript runtime. Some of the world's leading companies use Node.js in production, including Netflix, Walmart, and Uber. Node.js also provides access to a large library of JavaScript modules, greatly simplifying the development of Node.js web applications. It allows developers to create command-line tools and server-side scripts using JavaScript.

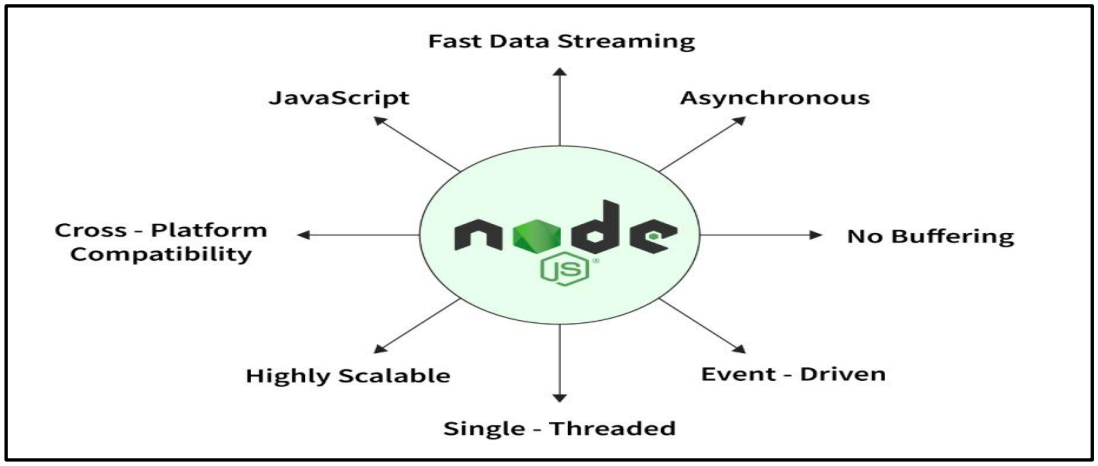


Node.js Features:

1. **Single-threaded:** In Node.js all requests are single-threaded and collected in an event loop. The event loop is what allows Node.js to perform all non-blocking operations. This means that everything from receiving the request to performing the tasks to sending the

response to the client is executed in a single thread. This feature prevents reloading and reduces context switching time.

2. **Cross-Platform Compatibility:** Node.js can be used on a wide variety of systems, from Windows to Mac OS, Linux, and even mobile platforms.
3. **JavaScript:** Most developers already have a good understanding of JavaScript, how it works, and other basic and advanced concepts related to it. Node.js allows developers to use JavaScript for backend development. This is convenient because developers don't have to switch between multiple programming languages and can make full-stack projects by only knowing JavaScript.
4. **No buffering:** Node.js works with data streams, which are aggregated data. Therefore, the user can get the data more easily and quickly because there is no need to wait for the entire operation to complete. It reduces the overall time required for processing. Because of this, there is little or no data buffering with Node.js.



Node Package Manager (NPM):

Node Package Manager provides two main functionalities:

→ Online repositories for Node.js packages/modules, which are searchable on Node.js documents.

→ A command-line utility to install Node.js packages, do version management, and dependency management of Node.js packages.

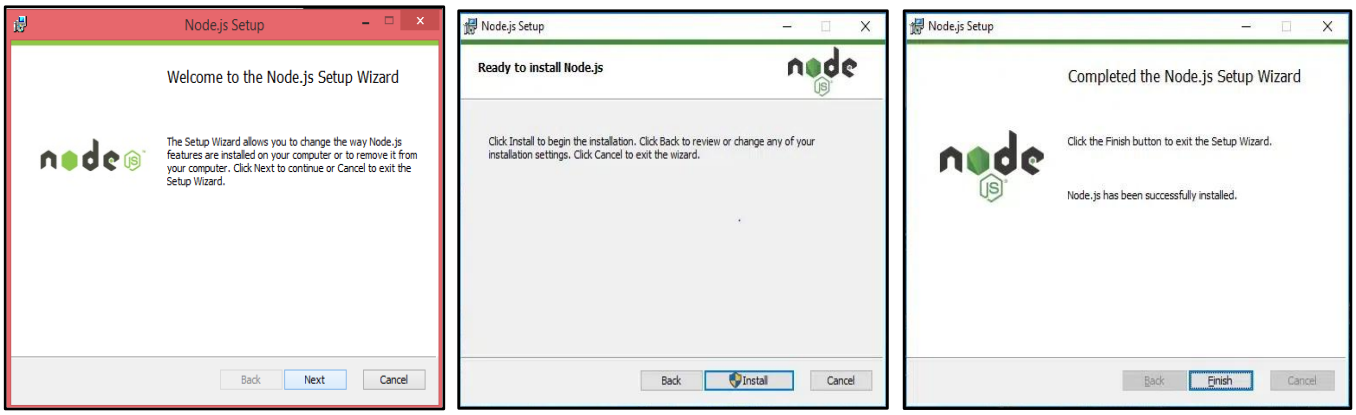
❖ When you install Node.js, NPM is also installed. The following command in CMD can verify if NPM is properly installed: **npm --version**.

Node.js Download and Installation:

To install Node.js on Windows 10, download the installer from the official Node.js website, run the installer, and follow the on-screen instructions. Verify the installation by checking the Node.js and npm versions in the command prompt.

- Go to the official Node.js website https://nodejs.org/en/download
- Download the Windows Installer (.msi) for the LTS (Long Term Support) version.
- The LTS version is recommended for most users due to its stability.
- Run the downloaded .msi file.
- Follow the prompts in the setup wizard.

- Accept the license agreement.
- Choose the installation location (or keep the default).
- Ensure you check the box to include npm in the installation.
- Click "Next" and then "Install" to begin the installation.



Verify the installation:

- Open Command Prompt.
- Type **node -v** or **node --version** and press Enter to check the Node.js version.
- Type **npm -v** or **npm --version** and press Enter to check the npm version.
- If the versions are displayed, Node.js and npm are installed correctly.

```
C:\Users\HP>node --version
v22.18.0

C:\Users\HP>node -v
v22.18.0

C:\Users\HP>npm -v
11.5.2

C:\Users\HP>npm --version
11.5.2
```

Note: No need to install npm separately; it is automatically installed when you install Node.js.

Note: You can run the following command to quickly update npm:

npm install -g npm

This updates the npm CLI (Command Line Interface) to the latest version.

Type Script Compiler Installation:

To install the TypeScript compiler on Windows, the recommended method is to use npm (Node Package Manager), which comes bundled with Node.js. Ensure Node.js and its package manager, npm, are installed on your system. You can download them from the official Node.js website.

- Open your command prompt
- Execute the following command to install TypeScript

npm install -g typescript

```
C:\Users\gfg0925>npm install -g typescript

added 2 packages in 3s
```

- After the installation completes, you can verify it by checking the TypeScript compiler version: **tsc --version (or) tsc -v**

This command should display the installed version of TypeScript.

```
C:\Users\HP>tsc --version
Version 6.0.3
```

Install a Specific Version of TypeScript:

- To install a specific version of TypeScript, use the following command:

npm install -g typescript@5.9.2

Replace 5.9.2 with the required version.

Uninstall TypeScript

- If you need to uninstall TypeScript, use the following command:

npm uninstall -g typescript

VS-Code Editor Download and Installation:

This is an open-source IDE from Visual Studio. It is available for Windows OS, Mac OS and Linux platforms. To download and install the VS Code editor on Windows, you will need to get the installer from the official website and follow the setup wizard. VS Code is lightweight and has modest system requirements.

Step 1: Download the installer

- Go to the official Node.js website: https://code.visualstudio.com/download
- Click the "Download for Windows" button to get the latest stable installer.
- The file, named something like VSCodeUserSetup-x64-1.xxx.x.exe, will be downloaded to your computer.

Step 2: Install VS Code

- Locate and double-click the downloaded .exe file to launch the setup wizard.
- The "License Agreement" window will appear. Read the agreement and select "I accept the agreement", then click Next.
- On the "Select Destination Location" screen, you can leave the default installation path or choose a different one. Click Next.
- The "Select Start Menu Folder" page will appear. Leave the default name and click Next.
- On the "Select Additional Tasks" screen, it is recommended to check the boxes for "Create a desktop icon" and "Add 'Open with Code' action to Windows Explorer file context menu" for easy access.
- Click Install. The process will only take a minute or so.

- Once the installation is complete, an option to "Launch Visual Studio Code" will be checked by default. Click Finish to open the editor.

Step 3: **Get started with VS Code**

- After installation, the editor will open. You can begin using it by:
- Installing extensions: Click the Extensions icon on the left-side panel to search for and install extensions for your favourite programming languages, themes, and more.
- Using the integrated terminal: Open a terminal directly within VS Code by going to the Terminal menu and selecting New Terminal.
- Opening a file or folder: Use the File > Open File... or File > Open Folder... menu options to start working on your projects.

First TS Script Program: Compilation and Execution:

TypeScript is a strongly typed superset of JavaScript. Before writing your first TypeScript program, ensure that Node.js and the TypeScript compiler are installed on your system.

Step 1: Create a Project Folder

- Create a new folder on your system, for example: **TypeScriptPrograms**
- Open **VS Code**
- Click **File → Open Folder…** and select the TypeScriptDemo folder [**OR**]
- Navigate to the project folder using this command: **cd typescript-hello-world**
- Generate a TypeScript configuration file: **tsc --init**
- This creates a **tsconfig.json** file containing the compiler configuration.

Step 2: Create Your First TypeScript File

- In VS Code, right-click inside the folder and select **New File**
- Name the file as **first.ts**
- Type the following code in the file:

```
let message: string = "Hello TypeScript";
console.log(message);
```

- Save the file using **Ctrl + S (or) Enable Auto Save**

  Enable Auto Save in VS Code
  - Go to File
  - Click Auto Save [Done ✅ (it will auto save after delay → 1000 (1 second)].
    - It saves automatically while coding.
    - Prevents data loss during crashes.

Compile & Run TypeScript into JavaScript:	**Directly Run TypeScript Program:**
Step 1: Compile TypeScript to JavaScript Command**: tsc Addition.ts** **Generates: Addition.js** **Step 2:** Run JavaScript using Node.js	**Step 1:** Install tsx **Command: npm install tsx** **Step 2: Run the TypeScript file directly** Command**: npx tsx s3.ts** Command**: npm --init**

Command**:** node Addition.js **When to use Compile & Run Way** ▪ Sharing compiled code ▪ When .js file is required	**When to use Direct Run** ▪ During development ▪ For learning and practicing TypeScript ▪ For quickly testing TypeScript programs
Example Program: let a: number = 10; let b: number = 20; let sum: number = a + b; console.log ("Addition: " + **sum**); → tsc **addition**.ts → node **addition**.js **Output:** Addition: 30	**Example Program:** let a: number = 10; let b: number = 20; let sum: number = a + b; console.log ("Addition: " + **sum**); → **npx tsx addition.ts** **Output:** Addition: 30

****

ReadLine Module (TypeScript / Node.js)

Hard Coding: If input values are initialized directly inside the program, it is known as Hard Coding. Hard coding means input values are fixed in the source code, and cannot be changed without modifying the program.

Disadvantages of Hard Coding

- Input values cannot be changed at runtime.
- Program must be edited for every change.
- Not suitable for real-time applications.

How can we provide different input values every time without modifying the TypeScript program?

Node.js provides a predefined built-in module called **readline**. Readline Module in Node.js allows the reading of input stream line by line. This module wraps up the process standard output and process standard input objects. The Readline module makes it easier to input and read the output given by the user. To use the Readline module, we typically start by creating a Readline interface that manages input and output streams.

Module Name: readline

Purpose:

- To read or scan input data from the keyboard.
- To accept user input during program execution.

Type: Predefined / In-built Node.js module.

Nature of readline Methods

- Methods are non-static
- Interface object creation is mandatory.
- Input handling is asynchronous.
- Input data is always received as String.

In TypeScript / Node.js:

- All keyboard input is treated as string data.
- Explicit type conversion is required for numeric operations.

Syntax:

import * as readline from 'readline'; const rl = readline.createInterface({ input: process.stdin, output: process.stdout });	import → keyword readline → predefined Node.js module createInterface() creates an input/output interface process.stdin → standard input stream (keyboard) process.stdout → standard output stream (console)

Once we have a Readline interface, we can use it to collect user input. The most common method for this purpose is rl.question():

Purpose: question () method

- Displays a prompt message
- Accepts input from the user

- Stores input in a callback parameter

Syntax: rl.question(prompt, callback) **Ex**: rl.question('Enter value: ', (answer) => { console.log(answer); });	▪ Displays prompt message ▪ Waits for user input ▪ Stores input inside answer ▪ Input is always a string

Type Conversion: Since input is received as string, conversion is required.

let num: number = Number(answer);

Common Type Conversions:

Required Data Type		**Conversion Method**
number	--	Number(value)
boolean	--	value === 'true'
string	--	string

Closing the Interface: rl.close();

- Releases system resources
- Terminates program execution properly
- Mandatory step after input operations

```
import * as readline from 'readline';
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});
rl.question('Enter value for a: ', (a) => {
  rl.question('Enter value for b: ', (b) => {
    let num1: number = Number(a);
    let num2: number = Number(b);
    let sum: number = num1 + num2;
    console.log("Addition: " + sum);
    rl.close();
  });
});
```

Execution Flow

- Program execution starts
- User enters first value
- User enters second value
- Input values are converted to numbers
- Arithmetic operation is performed
- Result is displayed
- Program terminates

Execution: npx tsx addition.ts

OutPut:

```
Enter value for a: 25
Enter value for b: 15
Addition: 40
```

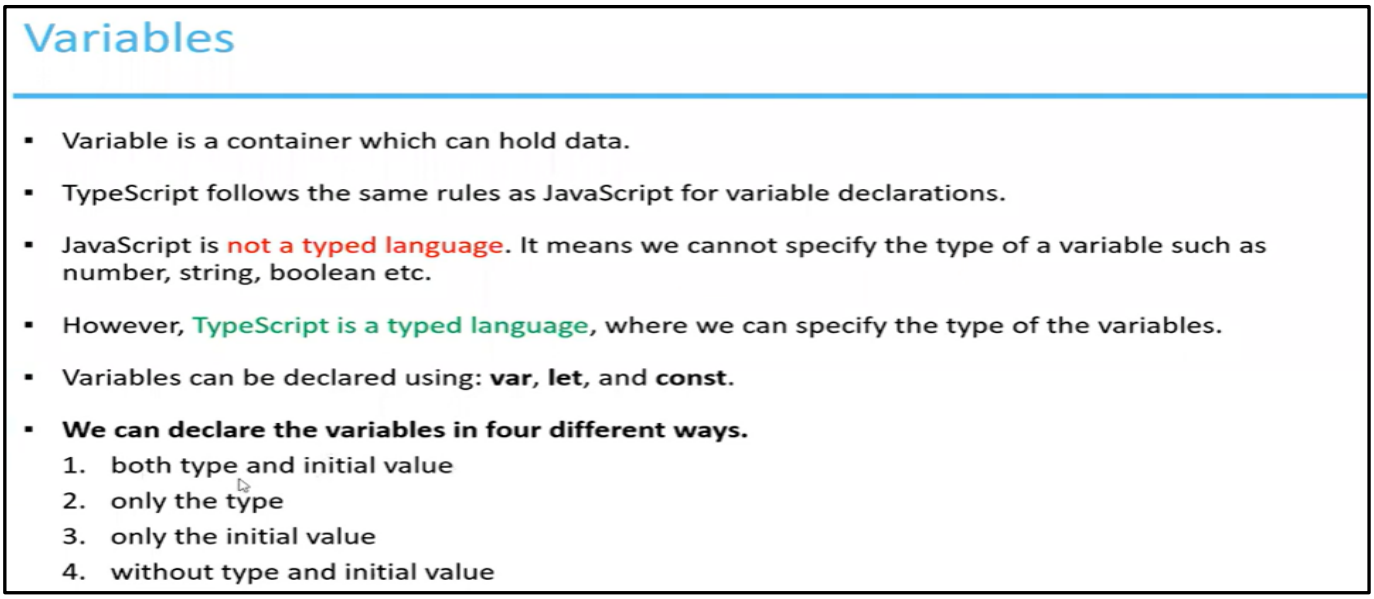
****

Variables and Data Types in TypeScript

Variables are containers that hold data values. A variable, by definition, is a named space in the memory that stores values.

TypeScript variables must follow the JavaScript naming **rules**:

- Variable name must be unique within the scope.
- The first letter of a variable should be an Upper-case letter, Lower case letter, underscore or a dollar sign.
- Subsequent letters of a variable can have upper case letter, Lower case letter, underscore, dollar sign, or a numeric digit.
- They cannot be keywords.
- Identifiers are case-sensitive.
- They cannot contain spaces.



Variable Declaration in TypeScript:

The type syntax for declaring a variable in TypeScript is to include a colon (:) after the variable name, followed by its type. Just as in JavaScript, we use the var keyword to declare a variable. When you declare a variable, you have four options –

Variable Declaration Syntax, Description and Example
A. Declare type and value in one statement **Syntax**: var identifier: type = value; **Ex**: var age: number = 25; Here, Type is number and Value is 25.
B. Declare type but no value The variable is initialized to undefined. **Syntax**: var identifier: type; **Ex**: var userName: string; Here, Type is string and the variable value is set to undefined by default.
C. Declare value but no type

The variable type is inferred from the assigned value.
Syntax: var identifier = value;
Ex: var isActive = true;
Here, Type is automatically inferred as boolean and Value is true.

D. Declare neither type nor value
The variable type becomes any and the value is initialized to undefined.
Syntax: var identifier;
Ex: var data;
Here, Type is any and Value is undefined

Example Program:

```
var name:string = "John";
var score1:number = 50;
var score2:number = 42.50
var sum = score1 + score2
console.log("name"+ name)
console.log("first score: "+score1)
console.log("second score: "+score2)
console.log("sum of the scores: "+sum)
```

On compiling, it will generate following JavaScript code.

```
//Generated by typescript 1.8.10
var name = "John";
var score1 = 50;
var score2 = 42.50;
var sum = score1 + score2;
console.log("name" + name);
console.log("first score: " + score1);
console.log("second score : " + score2);
console.log("sum of the scores: " + sum);
```

The **output** of the above program is given below:

```
name: John
first score: 50
second score: 42.50
sum of the scores: 92.50
```

Note: The TypeScript compiler will generate errors, if we attempt to assign a value to a variable that is not of the same type. Hence, TypeScript follows Strong Typing. The Strong typing syntax ensures that the types specified on either side of the assignment operator (=) are the same. This is why the following code will result in a compilation error −

Ex: var num: number = "hello" // will result in a compilation error

TypeScript Variable Scope:

The variables have a scope or visibility. The scope of a variable determines which part of the program can access it. The availability of a variable within a program is determined by its scope. We can define a variable in four ways, so as to limit their visibility.

1. Global Scope **2. Function Scope**
3. Block Scope **4. Class Scope**

1. **Global Scope:** Global variables are defined outside of any function, class, or code

block. They become part of the global scope and can be accessed from anywhere within the program.

2. Function Scope (var): Function-scoped variables are declared inside a function using the var keyword. These variables are accessible anywhere within the function in which they are declared, but they cannot be accessed outside that function.

3. Block Scope (let, const): Block-scoped variables are declared using let or const inside a block such as if, for, or while. These variables are accessible only within the block in which they are declared and cannot be accessed outside the block.

4. Class Scope (Fields / Static Fields): Class variables, also called fields, are declared within a class but outside its methods. These variables can be accessed using an object of the class. Fields can also be declared as static. Static fields are accessed using the class name.

TypeScript Variables: var, let, and const.

The process of creating a variable and giving it a name so that it can be used to store a value is known as variable declaration. In TypeScript, as in modern JavaScript (ES6+), var, let, and const are used for declaring variables, each with distinct rules regarding scope, reassignment, and hoisting.

1. var:

Scope of var:

Scope essentially means where these variables are available for use. var declarations are globally scoped or function/locally scoped. The scope is global when a var variable is declared outside a function. This means that "any variable that is declared with var outside a function block is available for use in the whole window", it becomes globally scoped. var declarations are function scoped "when it is declared within a function. This means that it is available and can be accessed only within that function".

var x = 10; function newFunction() { var y = 20; console.log(y); // Output: 20 } console.log(x); // Output: 10	var x = 10; function newFunction() { var y = 20; } console.log(x); // Output: 10 console.log(y); // Output: y is not being available outside the function.

Function	If Block
Purpose: Defines a reusable block of code that can be executed when called. **Scope:** Variables declared inside a function with var, let, or const are local to the function.	**Purpose**: Conditional execution of code based on a boolean expression. **Scope:** var inside if is function-scoped (or global if no function) → accessible outside the block.

Access outside: Function-scoped variables cannot be accessed outside the function unless returned or globally declared**.** **Example:** function newFunction() { var x = 10 console.log(x) } newFunction() console.log(x); // **Output**: x is not defined	**let and const inside if are block-scoped → accessible only inside the { }.** **Example:** if (true) { var y=1000 console.log(y) //ok // Output:1000 } console.log(y) //ok // Output:1000

Redeclare and Reassign variables declared with var:

Reassignment: var variables can be re-declared and updated. This means that we can do this within the same scope and won't get an error.

```
var x = 10;
console.log(x); // output :10
// redeclare the variable
var x;
console.log(x); // output :10 var x; just gets ignored (since x already exists)
// variable reassignment
//var a=20;
x = 20;
console.log(x); // output: 20
```

As you can see in this example, if we redeclare the variable, it does not affect its value. Its value remains the same. But if we reassign the value, the variable is storing the reassigned value.

Hoisting variables declared with var:

var declarations are hoisted, meaning the declaration is moved to the top of its containing scope during compilation. This allows var variables to be accessed before their declaration in the code, though their value will be undefined until the actual assignment is reached.

```
console.log(x); // output: undefined
var x=10;
console.log(x); // output: 10
 x = 20;
console.log(x); // output: 20
```

Due to hoisting, it does not matter where we have declared the variables, the variable declarations move to the top of their scopes. In this example, we have printed the variable before its declaration, and it is not giving any error. It will give an undefined value until we initialize it.

2. let:

The let keyword was introduced in ES6 to overcome the limitations of var. It provides block-level scope and better control over variable usage.

Scope of let: Variables declared using let are block-scoped. The variable is accessible only inside the block { } in which it is declared. Blocks include: if, for, while, { }

Function Scope with let	**If Block Scope with let**
When a variable is declared using let inside a function, it is accessible only within that function. **Example** function newFunction() { let x = 10; console.log(x); // Output: 10 } newFunction(); console.log(x); // Error: x is not defined	When let is used inside an if block, the variable is accessible only inside that block. **Example** if (true) { let y = 1000; console.log(y); // Output: 1000 } console.log(y); // Error: y is not defined

Redeclaration and Reassignment with let

Redeclaration	**Reassignment**
Variables declared using let cannot be redeclared in the same scope. let a = 10; // let a = 20; // Error: Cannot redeclare block-scoped variable	Variables declared using let can be reassigned. let a = 10; a = 20; console.log(a); // Output: 20

Hoisting in let:

Variables declared using let are hoisted, but they are placed in the Temporal Dead Zone (TDZ).

- The variable exists in memory
- It cannot be accessed before declaration
- Accessing it before declaration causes an error

Example

```
console.log(x); // Error
let x = 10;
```

3. Const:

The const keyword is used to declare constant values. Once a value is assigned to a const variable, it cannot be changed.

Scope of const: const is block-scoped and Same scoping rules as let

Declaration Rules for const:

- Must be initialized at the time of declaration
- Cannot be redeclared
- Cannot be reassigned

Example	**If Block Scope with const**
const x = 10; console.log(x); // Output: 10 // const y; // Error: Initialization required	if (true) { const z = 50; console.log(z); // Output: 50 } console.log(z); // Error: z is not defined

Reassignment and Redeclaration with const	**Hoisting in const**
const a = 100; // a = 200; // Error: Assignment to constant variable // const a = 200; // Error: Cannot redeclare block-scoped variable	Like let, const variables are also: Hoisted Placed in the Temporal Dead Zone Not accessible before declaration console.log(b); // Error const b = 30;

Comparison: var vs let vs const

Feature	var	let	const
Scope	Function	Block	Block
Value Assignment at Declaration	❌ Not Mandatory	❌ Not Mandatory	✅ Mandatory
Re-declare	✅ Allowed	❌ Not Allowed	❌ Not Allowed
Re-assign/ Reinitialization	✅ Allowed	✅ Allowed	❌ Not Allowed
Hoisting	✅ (undefined)	❌ (Not initialized)	❌ (Not initialized)
Best Use	❌ Avoid	✅ Changing values	✅ Constants

****

Data Types in TypeScript

The Type System represents the different types of values supported by the language. The Type System checks the validity of the supplied values, before they are stored or manipulated by the program. This ensures that the code behaves as expected. We assign data types by simply placing a colon after the variable name but before the equal sign:

Syntax: datatype {variable name}: {variable type} = {variable value};

In TypeScript, data types are broadly categorized into two groups: primitive and non-primitive (or reference) data types.

1. Primitive Data Types: Primitive data types represent single, simple values and are immutable, meaning their values cannot be changed after creation. When a primitive variable is assigned to another, a copy of the value is created.

Number, String, Boolean, Null, Undefined, Any, Union, Type, Void.

A. Number:

In TypeScript, the number data type represents both integer and floating-point values or BigIntegers. TypeScript uses the same number type for all numeric values. These number and floating-point values get the type number, while BigIntegers get the type bigint.

let age=25; let price=25.5 let big=324223226n console.log(age) //25 console.log(price) //25.5 console.log(big) //324223226	let age: number = 30; // Integer number let price: number = 30.5; // Float number let big: bigint = =324223226n; //Big number console.log(age) console.log(price) console.log(big)
let age=25; let price=25.5 let big=324223226n console.log("Age:",age) console.log("Price",price) console.log("Big Number",big)	let age=25; let price=25.5 let big=324223226n **//Print datatype** console.log(typeof age); // number console.log(typeof price); // number console.log(typeof big); // bigint

B. BigInt:

The bigint type represents integers with arbitrary precision, meaning it can store integers larger than the number type limit.

Example: let bigNumber: bigint = 1234567890123456789012345678901234567890n;

let num1: number = 9007199254740991; let num2: number = 9007199254740992; console.log(num1 + 1); // 9007199254740992 console.log(num2 + 1); // 9007199254740992 // wrong	let big1: bigint = 9007199254740991n; let big2: bigint = 9007199254740992n; console.log(big1 + 1n); // 9007199254740992n console.log(big2 + 1n); // 9007199254740993n // correct

C. String:

The string data type is used to store the text/textual data. It's a sequence of characters. You can define a string using three ways: single quotes ('), double quotes (") and backticks (`). As in other languages, we use the type string to refer to these textual datatypes.

let first_name: string =' Krishna'; let last_name: string = Rao"; console.log(firstname) //Krishna console.log(lastname) //Rao	let firstname:String="Krishna" let lastname:String="Rao" console.log("Hello",firstname,lastname) //Hello Krishna Rao
But when we store variable use backtick(``) let greeting:string=` Hello, my name is ${firstname} ${lastname}`; console.log(greeting);// Hello, my name is Krishna Rao Note: So, whenever you see `${variable}`, the backticks are what allow embedding variables and expressions in strings.	

D. Boolean:

The boolean data type represents a logical entity and can only two values, true or false. It is mostly used to check a logical condition. Thus, Booleans are logical data types which can be used for comparison of two variables or to check a condition. The true and false implies a 'yes' for 'true' and a 'no' for 'false in some places when we check a condition or the existence of a variable or a value.

let isstudent:boolean=true; let hasJob:boolean=false; console.log("Is Student",isstudent) //true console.log("Has Job",hasJob) //false console.log(typeof hasJob) //boolean	var a =5; var b=6; a==b // returns false if(a<b){ alert(a is a smaller number than b); } If 'a<b' is true, the alert will pop on the screen.

Special Types: TypeScript provides special types to handle specific scenarios where normal type checking is not sufficient.

E. Null and Undefined:

TypeScript has two special types, Null and Undefined, that have the values null and undefined respectively.

Null refers to the data type or value. The null is a keyword in TypeScript, which we can use to represent the absent or empty value. So, we can use? null' to define the variable's data-type or initialize the variable. A null value means no value and null is used to explicitly indicate that a variable is empty or contains no meaningful value.

```
var a = null;
console.log(a);   // This returns null
let empty: null = null;
let data: string | null = null;
```

Undefined is a primitive value that indicates that the value is not assigned. i.e. whenever we do not explicitly assign a value to a variable, TypeScript assigns the undefined value to it. undefined indicates that a variable has been declared but hasn't been assigned any value. It's commonly used to represent missing function arguments or uninitialized object properties.

```
let a;
console.log("The value of a is " + a); //The value of a is undefined
let myVariable: string; // myVariable is undefined until assigned
console.log(myVariable); // Output: undefined
let price:number;
console.log(price); //undefined
```

While both can be used to check for the absence of a value, undefined is commonly encountered when variables are uninitialized or properties are missing from objects, whereas null is typically used when a value is deliberately set to be empty or non-existent. TypeScript's strict null checks (when enabled) treat null and undefined as distinct types, requiring explicit handling if a variable might hold either.

F. Symbol:

The symbol type represents a unique and immutable value. It is often used as a unique object key to avoid name collisions.

Example:

```
let id1: symbol = Symbol("id");
let id2: symbol = Symbol("id");
console.log (id1 === id2); // false (always unique)
```

Type	Keyword	Description
Number	number	Represents both integer and floating-point numbers.
String	string	Represents textual data.
Boolean	boolean	Represents logical values: true or false.
Null	null	Represents the intentional absence of any object value.
Undefined	undefined	Represents an uninitialized variable.
BigInt	bigint	Used for very large integers.
Symbol	symbol	Creates a unique identifier.

G. Any Type:

The any type is a special type in TypeScript that allows a variable to store values of any data type. When a variable is declared as any, TypeScript skips compile-time type checking for that variable. This means you can:

- ✓ Assign any value
- ✓ Call any method
- ✓ Access any property without errors

Example:

```
let variable: any = "Hello";
variable = 5;       // Valid
variable = true;    // Valid
```

H. Unknown Type:

The unknown type is similar to the any type, but it is safer. A variable of type unknown can store any value, but TypeScript does not allow using that value directly. You cannot perform operations on an unknown type until you perform type checking. Before performing any operation on an unknown variable, you must check its type using: typeof or instanceof etc. The unknown type is safer than any.

Syntax let value: unknown; **Example**1: Assignment Allowed let data: unknown; data = "Hello"; data = 10; data = true; ✓ Any value can be assigned	**Example**2: Direct Usage Not Allowed let data: unknown = "TypeScript"; data.toUpperCase(); // Error //Cannot use methods directly **Example3**: Type Checking Required (Correct Way) let data: unknown = "TypeScript"; if (typeof data === "string") { console.log(data.toUpperCase()); // OK }

I. Void Type:

The void type is used to indicate that a function does not return any value. It is commonly used as the return type of functions that only perform an action.

Syntax: function functionName(): void { // statements }	Example: function displayMessage(): void { console.log("Welcome to TypeScript"); }

****

TypeScript Tokens

I. Keywords in TypeScript:

Keywords in TypeScript are reserved words that have special meaning in the language. They cannot be used as identifiers (like variable names, function names, etc.) because they serve specific syntactic and control purposes. TypeScript inherits most keywords from JavaScript and introduces additional ones to support its type system.

The following table lists some **keywords** in TypeScript.

var	number	break	as	any	switch
case	if	throw	else	string	get
module	type	instanceof	typeof	public	private
enum	export	finally	for	while	void
null	super	this	new	in	return
extends	static	let	implements	interface	function
try	yield	const	continue	do	catch

II. Identifiers in TypeScript

Identifiers are nothing but the names given to any class members like a variable, method name, class name, array name, etc. Certain **rules** to be followed while declaring Identifiers:

- Identifier names can include both upper-case as well as lower-case letters and digits but can't start with numbers.
- Identifiers cannot include special symbols except for underscore (_) or a dollar sign ($).
- Identifiers cannot be keywords and must be unique.
- The identifier is case-sensitive and doesn't contain spaces.

Valid identifiers	**Invalid identifiers**
firstName	var
first_name	first name
num1	first-name
$dollar	1number
_result	&name

III. Typescript Constants:

Typescript constants are variables, whose values cannot be modified. We declare them using the keyword const. They are block-scoped just like the let keyword. Their value cannot be changed neither they can be redeclared. We declare constants using the keyword const keyword.

Syntax: const variable=value;
Ex: const maxAttempts = 5; const greeting = "Hello, world!";

const isLoggedIn = true;
Each of these constants is immutable. Any attempt to reassign will cause a compile-time error. // Error! constant cannot be changed

Note: **Always Initialize a Constant During Declaration. If you don't initialize a constant at the time of declaration, it throws an error.**

For example: const age: number; // Error! Missing initializer in const declaration.

When to Use Const in TypeScript:

- The variable should never be reassigned after initialization.
- You're working with values that should stay constant across the function or module.
- You want better code predictability and fewer bugs.
- You're declaring arrays or objects that you won't reassign but may modify.
- In TypeScript, const helps make your intentions clear: this variable won't .

✓ **TypeScript is Case-sensitive:** TypeScript is case-sensitive. This means that TypeScript differentiates between uppercase and lowercase characters.

✓ **Semicolons are optional:** Each line of instruction is called a statement. Semicolons are optional in TypeScript.

Example:

```
console.log("hello world")
console.log("We are learning TypeScript")
```

IV. Errors in TypeScript:

An error is a problem in a program that prevents it from executing correctly or producing the expected output. TypeScript provides compile-time error checking, which helps detect mistakes early, improves code quality, and reduces runtime failures.

Classification of Errors in TypeScript

1. Compile-Time Errors	2.Runtime Errors	3.Logical Errors

1. Compile-Time Errors:

Compile-time errors are errors that are detected during the TypeScript compilation process, before the program is executed. These errors occur when the program violates TypeScript's syntax rules or type system rules.

Common Compile-Time Errors

a) Type Mismatch Error: Occurs when a value of one data type is assigned to a variable of another incompatible data type.

Ex: let a: number = "hello"; // Error: Type 'string' is not assignable to type 'number'.
console.log (a);

b) Variable Not Declared Error: Occurs when a variable is used without being declared.

Ex: console.log(x); // Error: Cannot find name 'x'.

c) Redeclaration Error: Occurs when a variable declared using let or const is redeclared in the same scope.

```
let x = 10;
let x = 20;   // Error: Cannot redeclare block-scoped variable.
console.log (x);
```

d) Missing Initialization Error (const): Occurs when a const variable is declared without initialization.

```
const x;   // Error: 'const' declarations must be initialized.
console.log (x);
```

2. Runtime Errors:

Runtime errors occur while the program is executing, even though the program has no compile-time errors. These errors are usually caused by invalid operations, undefined values, or unexpected input.

Examples of Runtime Errors

a) Accessing Property of null: Occurs when a property or method is accessed on a null value.

```
let data: any=null;
console.log(data.length);  // Runtime Error
```

TypeError: Cannot read properties of undefined (reading 'length')

b) Undefined Value Error: Occurs when an operation is performed on an undefined value.

```
let a: any = 100;
console.log(a.toUpperCase());
```

TypeError: a.toUpperCase is not a function

3. Logical Errors:

Logical errors occur when the program executes successfully without errors, but produces incorrect or unexpected output. These errors occur due to incorrect logic or wrong implementation of the program.

Written Logic:	**Correct Logic:**
let a: number = 10; let b: number = 20; let average = **a + b / 2**; console.log(average); // Output: 20 But the correct answer is: 15	let average = **(a + b) / 2**; console.log(average); // Output: 15
here is: ❌ **No syntax error** ❌ **No compile-time type error** ❌ **No runtime error** ✅ **Program executes** ❌ **Result is logically wrong**	

V. Comments in TypeScript:

Comments in TypeScript are non-executable annotations embedded within the code to explain its behavior or mark sections for future reference. They do not affect runtime execution but serve as guidance for developers. TypeScript comments can describe code functionality, provide context, or temporarily disable blocks during debugging. While execution of the code, compiler ignore the comments and compile the rest of the code.

Types of TypeScript Comments:

1) Single-Line Comments: Single-line comments start with two forward slashes (//). Any text between // and the end of the line is ignored by js/ts (will not be executed). Use them for brief explanations or to disable specific lines temporarily.

Syntax: //Comments here (Text in this line only is considered as comment).

Ex**:** // Prints the below text.

console.log("Hello World");

Note: The keyboard shortcut to comment out all lines at once in Visual Studio Code depends on your operating system:

Windows / **Linux**: Press Ctrl + A (to select all) then Ctrl + /

Mac: Press Cmd + A (to select all) then Cmd + /

```
// This variable holds the user's age
let age: number = 25;
```

2) Multi-Line Comments: Multi-line comments start with /* and ends with */. Any text between /* and */ will be ignored by js/ts. Single line comments can be tedious to write since we have to give '//' at every line. So, to overcome this multi-line comments can be used. Enclosed between /* and */, these are useful for longer explanations or block commenting during debugging.

Note: If you prefer block comments (wrapping the whole code block in /* ... */) instead of adding // to every individual line, use these shortcuts after selecting your text:

Windows: Shift + Alt + A

Mac: Shift + Option + A

```
/*
  This function calculates the area of a rectangle.
  It takes two numeric parameters: width and height.
*/
function getArea(width: number, height: number): number {
  return width * height;
}
```

VI. Type Conversion in JavaScript /TypeScript

As the name implies, type conversion is the process of converting a value from one type to another. Values in JavaScript can be of different types. You could have a number, string, object, boolean – you name it. Sometimes, you may want to convert data from one type to

another to fit a certain operation. Type conversion can either be implicit (automatically done during code execution) or explicit (done by you the developer). Implicit Type Conversion is also known (and more commonly referred to) as Coercion while Explicit Type Conversion is also known as Type Casting.

1. Implicit Type Conversion (Coercion):

Type coercion is the process where JavaScript automatically converts or coerces a value to another type when required by an operation. There is no special syntax for coercion because JavaScript performs it automatically. There are some operations that you might try to execute in JavaScript which are literally not possible.

Example:1 — Number → String

Here, you're trying to add a number and a string. This is, practically not possible. You can only add numbers (sum) together or add strings (concatenate) together.

```
const sum = 35 + "hello"
console.log(sum)  // 35hello
console.log(typeof sum)  // string
```

Coercion is usually caused by different operators used between different data types:

```
const string = ""
const number = 40
const boolean = true
console.log(!string)  // true - string is coerced to boolean `false`, then the NOT operator negates it
console.log(boolean + string)  // "true" - boolean is coerced to string "true", and concatenated with the empty string
console.log(40 + true)  // 41 - boolean is coerced to number 1, and summed with 40
```

2. Explicit Type Conversion (Type Conversion):

Explicit type conversion is the process of explicitly converting a value from one datatype to another. To explicitly convert types, you use the type Constructors.

Common conversion functions:

Number(value) **String(value)**

Boolean(value) **BigInt(value)**

Convert a number to string:	**Convert a String → Number**
const number = 30 const numberConvert = **String**(number) console.log(numberConvert) // "30" console.log(typeof numberConvert) // string	let a1: string = "25"; let a2: number = **Number**(a1); console.log(a2); //25 console.log(typeof a2); //number
Convert a boolean to string: const boolean = false const booleanConvert = **String**(boolean) console.log(booleanConvert) // "false"	**Convert a number to boolean:** const number = 30 const numberConvert = **Boolean**(number) console.log(numberConvert) // true

console.log(typeof booleanConvert) // string	console.log(typeof numberConvert) // boolean

Type Casting in TypeScript:

Type casting means converting a value from one type to another, especially a compatible type.

This terminology is common in languages such as Java, C, and C++. TypeScript does not have separate int, float, and double types.

Type casting allows us to convert a variable from one type to another type, but it is not supported in javascript as it is dynamic while in typescript, we can declare the type of every variable explicitly. There is no concept of type casting in javascript mainly because it has dynamic types.

Type Script has:

let a: number = 25;

let b: number = 25.75;

Both are: number

Therefore, Java-style:

int → float

float → int

does not directly apply to TypeScript.

Type Assertion in TypeScript:

Type assertion is a TypeScript feature used to tell the compiler: "Treat this value as this specific type." It does not perform runtime conversion.

There are mainly two ways to assert a variable in typescript:

1.Using the as Keyword. **2.Using the <> (Angle-Bracket) Operator.**

1. Type Assertion: Using the as Keyword:

One of the ways in typescript cast to typecast a variable is using the **as** keyword, which directly changes the type of the variable. Now let's see an example below of how to typecast a variable using the as keyword.

Ex1: let var1: unknown = 'yash'; console.log((var1 **as** string).length); **Output: 4**	**Ex2:** let var1: unknown = 4; console.log((var1 b string).length); **Output: Undefined**

The following **Ex2** gives an error because the number does not have a length. Type casting does not change the type of the data within the variable, for example, the following code will not work as expected because the variable var1 is holding a number.

2. Type Assertion: Using the <> Operator:

Another way to typecast a variable (typescript cast) is by using the <> operator, which also directly changes the type of the variable.

Syntax: let var1: typevar1;

let var2 = <typevar2>var1;

Ex1: let var1: unknown = 'yash'; console.log((<string>var1).length); **Output: 4**	**Ex2**: let colors: string = ‘green’; let len: number = (<string> green).length; console.log(len); **Output: 5**

****

Double Equality Operator (==) and Triple Equality Operator (===):

In JavaScript, there's both the double equality operator (== which is called the **loose equality** operator) and the triple equality operator (=== which is called the **strict equality** operator). You use both operators to compare values' equality.

Double Equality (==) Operator and Coercion: Loose Equality

The loose equality operator does a loose check. It checks if values are equal. The types are not a focus for this operator – only the values are the major factor.

Example1: Here is 20, a value of a number type, and "20", a value of the string type, are equal when you use double equality:

```
const variable1 = 20
const variable2 = "20"
console.log(variable1 == variable2) // true
```

Though the types are not equal, the operator returns **true** because the values are equal. What happens here is coercion. When you use the loose equality operator with values of different types, what happens first is coercion. Again, this is where one value is converted to the type that fits the other, before the comparison occurs.

In this case, the string "20" is converted to a number type (which is 20) and then compared with the other value, and they are both equal.

Example2:

```
const variable1 = false
const variable2 = ""
console.log(variable1 == variable2)  // true
```

Here, variable1 is the value false (boolean type) and variable2 is the value "" (an empty string, of the string type). Comparing both variables with the double equality returns true. That's because the empty string is coerced to a boolean type (which is false).

Triple Equality(===) Operator: Strict Equality

This operator does a strict check – that is, it strictly checks the values compared, as well as the types. Type coercion does not occur here, so there are no unexpected answers. Here are the examples from above:

Example1: const variable1 = 20 const variable2 = "20" console.log(variable1 === variable2) // false	**Example2:** const variable3 = false const variable4 = "" console.log(variable3 === variable4) // false

Here, variable1 and variable2 have the same values, but the types are not the same. So the triple equality returns false.	Here variable3 and variable4 have the same values (if one is converted to the type of the other) but the types are not the same, so the triple equality returns false this time, too.

****

Operators in TypeScript

An operator in TypeScript is a special symbol used to perform operations on values and variables. You can use operators throughout your code to assign values, compare data, manipulate collections, or perform logic checks.

Types of Operators in TypeScript:

In TypeScript, operators can be classified into three categories based on the number of operands they take: unary, binary, and ternary operators.

A) Unary Operator: Unary operators are operators that take a single operand. Some examples of unary operators in TypeScript are the negation operator (-), the logical not operator (!), and the typeof operator.

Ex: let x: number = -10;
let y: boolean =! true;
let z: string = typeof 'hello';

B) Binary Operator: Binary operators are operators that take two operands. Most operators that are available in TypeScript are binary operators. Some examples of binary operators in TypeScript are the addition operator (+), the subtraction operator (-), the multiplication operator (*), and the logical and operator (&&).

Ex: let a: number = 5 + 3;
let b: number = 10 - 5;
let c: number = 3 * 4;
let d: boolean = true && false;

C) Ternary Operator: The ternary operator is the only operator in TypeScript that takes three operands. It is represented by the ? and : symbols and is used as a shorthand for if-else statements.

Ex: let age: number = 25;
let message: string = age >= 18 ? 'You are an adult' : 'You are not an adult';

Types of Operators: TypeScript operators are categorized based on their functionalities.

Arithmetic Operators	**Relational Operators**
Logical Operators	**Assignment Operators**
Unary Operators	**Ternary Operator**
Bitwise Operators	**Shift Operators**

1. Arithmetic Operators:

Arithmetic operators are used to perform mathematical operations. TypeScript supports standard arithmetic operators such as addition (+), subtraction (-), multiplication (*), and division (/).

Name	Description	Syntax	Example let a = 5, let b = 3
Addition(+)	Adds two values or expressions.	a + b	console.log (a + b) // output: 8
Subtraction(-)	Subtracts the right operand from the left operand.	a - b	console.log (a - b) // output: 2
Multiplication(*)	Multiplies two values or expressions	a * b	console.log (a * b) // output: 15
Division(/)	Divides the left operand by the right operand.	a / b	console.log (a / b) // output: 1.6666666666666667
Modulus(%)	Returns the remainder of the division of the left operand by the right operand.	a % b	console.log (a - b) // output: 2

2. Comparison Operators (Relational Operators):

Comparison operators compare two values and return a Boolean value, true or false based on the comparison result. Relational Operators test or define the kind of relationship between two entities.

Name	Description	Syntax	Example let operand1 =10, let operand2 =20.
Greater than (>)	Returns true if the value of the left operand is greater than the value of the right operand.	operand1 > operand2;	10 > 20 (Output: false)
Less than (<)	Returns true if the value of the left operand is less than the value of the right operand.	operand1 < operand2;	10 < 20 (Output: true)
Greater than or equal to (>=)	Returns true if the value of the left operand is greater than or equal to the value of the right operand.	operand1 >= operand2;	10 >= 20 (Output: false)
Less than or equal to (<=)	Returns true if the value of the left operand is less than or equal to the value of the right operand.	operand1 <= operand2;	10 <= 20 (Output: rue)
Equal to (==)	Returns true if the values of the two operands are equal, after type coercion.	operand1 == operand2;	10 == 20 (Output: false)
Not equal to (!=)	Returns true if the values of the two operands are not equal, after type coercion.	operand1 != operand2;	10 != 20 (Output: true)

Strictly equal to (===)	Returns true if the values of the two operands are equal, without type coercion (strict equality).	operand1 === operand2;	10 === "20" (Output: false)
Strictly not equal to (!==)	Returns true if the values of the two operands are not equal, without type coercion (strict inequality).	operand1 !== operand2;	10 !== "20" (Output: true)

3. TypeScript Logical operators:

In TypeScript, logical operators are used to perform logical operations on Boolean values. Logical Operators are used to combine two or more conditions. Logical operators return a Boolean value.

In TypeScript, there are 3 logical operators:

a. Logical AND (&&): Returns true if both operands are true.

b. Logical OR (||): Returns true if either operand is true.

c. Logical NOT (!): Returns the opposite of the Boolean value.

Name	**Description**	**Syntax**	**Example** let operand1 = true, operand2 = false
Logical AND (&&)	Returns true if both operands are true.	operand1 && operand2;	console.log(condition1 && condition2) // output: false
Logical OR (||)	Returns true if at least one of the operands is true.	operand1 || operand2;	console.log(condition1 || condition2) // output: true
Logical NOT (!)	Returns true if the operand is false, and vice versa.	!operand;	console.log(!condition1) //output: false

4. TypeScript Assignment Operators:

In TypeScript, assignment operators are used to assign values to variables and modify their values based on arithmetic or bitwise operations. Assignment operators in TypeScript are used to assign values to variables. The most basic operator is the simple assignment operator (=), which assigns the right-hand operand to the left-hand operand. Other compound operators, like +=, -=, *=, and /=, perform an operation and assignment in one step.

Name	**Description**	**Syntax**	**Example** let a: number = 5, b: number = 10
Assignment (=)	Assigns the value of the right operand to the left operand.	variable = value;	let a: number = 5 // = or assign operator
Addition Assignment (+=)	Adds the value of the right operand to the current value of the left operand and assigns the result to the left operand.	variable += value;	b += a // b = b + a console.log(b) // 15

Subtraction Assignment (-=)	Subtracts the value of the right operand from the current value of the left operand and assigns the result to the left operand.	variable -= value;	b -= a // b = b - a console.log(b) // 5
Multiplication Assignment (*=)	Multiplies the current value of the left operand by the value of the right operand and assigns the result to the left operand.	variable *= value;	b *= a // b = b * a console.log(b) // 50
Division Assignment (/=)	Divides the current value of the left operand by the value of the right operand and assigns the result to the left operand.	variable /= value;	b /= a // b = b / a console.log(b) // 2
Modulus Assignment (%=)	Calculates the remainder when dividing the current value of the left operand by the value of the right operand and assigns the result to the left operand.	variable %= value;	b %= a // b = b % a console.log(b) // 0

5. Ternary Operator (Conditional Operator):

Ternary operator is used to represent a conditional expression. The ternary operator is a shorthand for the if-else statement, which is used to assign a value to a variable based on some specified condition.

Syntax: condition ? expression1 : expression2

Example:
```
var num:number = -2
var result = num > 0 ?"positive":"non-positive"
console.log(result)
```

Line 2 checks whether the value in the variable num is greater than zero. If num is set to a value greater than zero, it returns the string positive else the string non-positive is returned.

6. Unary Operators:

Unary operators are operators that work on only one operand. They are used to perform operations such as incrementing, decrementing, changing the sign of a number. An operator that acts on a single operand is called a unary operator.

Common unary operators in TypeScript include unary plus (+), unary minus (-), increment (++), decrement (--).

a) Unary Plus Operator (+): The unary plus operator is used to represent a positive value and is usually optional. It can also be used to convert a value into a number.

Ex: let x: number = 5; let y: number = +x;

b) Unary Minus Operator (-): The unary minus operator negates a numeric value by changing its sign from positive to negative or vice versa.

Ex: let x: number = 5; let y: number = -x;

c) Increment Operator (++): The increment operator increases the value of a variable by one. It has two forms: pre-increment and post-increment.

- Pre-Increment (++a): The value is increased before it is used in the expression.

Ex: let b: number = 5; let a: number = ++b;

- Post-Increment (a++): The value is used first, then increased by one.

Ex: let b: number = 5; let a: number = b++;

d) Decrement Operator (--): The decrement operator decreases the value of a variable by one. It also has two forms: pre-decrement and post-decrement.

- Pre-Decrement (--a): The value is decreased before it is used.

Ex: let b: number = 5; let a: number = --b;

- Post-Decrement (a--): The value is used first, then decreased by one.

Ex: let b: number = 5; let a: number = b--;

Note: In pre-increment and pre-decrement, the updated value is used immediately. In post-increment and post-decrement, the original value is used first and updated after the expression.

7. Bitwise Operators:

Bitwise operators are used to perform operations on the individual bits of integer values. In TypeScript, bitwise operators work on 32-bit signed integers. Before performing the operation, numbers are converted into binary form, the operation is performed bit by bit, and the result is converted back to a number. Bitwise operators work on each bit of a number, not on the entire number at once.

a) Bitwise AND (&) Operator: The bitwise AND operator is a binary operator denoted by &. It performs a bit-by-bit AND operation. If both corresponding bits are 1, the result bit is 1; otherwise, it is 0.

Ex: let a: number = 5; // 0101 let b: number = 3; // 0011
let result: number = a & b; Result is 1 (0001)

b) Bitwise OR (|) Operator: The bitwise OR operator is a binary operator denoted by |. It performs a bit-by-bit OR operation. If any one of the corresponding bits is 1, the result bit is 1; otherwise, it is 0.

Ex: let a: number = 5; let b: number = 3;
let result: number = a | b; Result is 7 (0111)

c) Bitwise XOR (^) Operator: The bitwise XOR operator is a binary operator denoted by ^. It performs a bit-by-bit XOR operation. If both bits are different, the result bit is 1; if both bits are the same, the result is 0.

Ex: let a: number = 5; let b: number = 3;
let result: number = a ^ b; Result is 6 (0110)

Shift Operators: Shift operators are special bitwise operators used to shift the bits of a number to the left or right. These operators are commonly used to multiply or divide a number by powers of two.

d) Signed Left Shift Operator (<<): The left shift operator shifts bits to the left by a specified number of positions. Zeros are filled in from the right side. Each left shift multiplies the number by 2.

Ex: let a: number = 5; // 0101 let result: number = a << 1;

Result is 10

e) Signed Right Shift Operator (>>): The signed right shift operator shifts bits to the right while preserving the sign bit. Each right shift divides the number by 2.

Ex: let a: number = 5; let result: number = a >> 1; Result is 2

Operator Categories	**Operator Symbols**
Arithmetic Operators	+ , - , * , / , %
Relational Operators	< , > , <= , >= , == , !=
Logical Operators	&& , || , !
Assignment Operators	= , += , -= , *= , /= , %=
Unary Operators	Unary + , - , ++ , --
Ternary Operator	? :
Bitwise & Shift Operators	& , | , ^ , <<, >>

****

Control Flow Statements in TypeScript

Control flow statements are used to control the order in which statements in a TypeScript program are executed. They help in managing the smooth and logical flow of a program.

- **Sequential Programming in TypeScript:** Sequential programming means executing statements one by one in a fixed order. The TypeScript compiler executes the code from top to bottom, and no statement is skipped. Each statement completes its execution before the next statement begins. Sequential programming always produces the same output for the same input. In, Sequential Programming:

• Statements execute in the order in which they are written.

• No overlapping or skipping of statements occurs during execution.

Control Flow Statements in TypeScript:

Control flow statements are used to control the order in which statements in a TypeScript program are executed. Normally, the program executes statements from top to bottom. Control flow statements allow the program to make decisions, repeat statements, or change the execution path based on conditions. These statements are essential for writing dynamic programs and for solving real-world problems. They help decide whether a block of code should be executed or skipped.

Types of Control Flow in TypeScript:

Sequential Flow: Statements execute one by one from top to bottom.

Conditional or Selection Flow: Only one block of code is executed based on a condition that evaluates to true or false.

Repetition or Loop Flow or Decision-Making: A block of statements is executed repeatedly as long as a specified condition is true.

Jumping or Branching Flow: Control jumps from one part of the program to another using statements such as break, continue, and return.

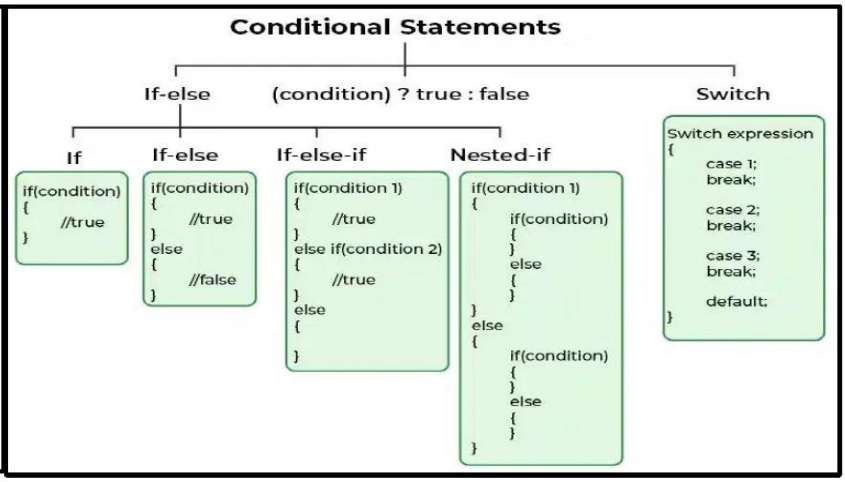
Decision-Making Statements (Conditional) in TypeScript

Decision-making statements are used to control the flow of a program based on conditions. These statements evaluate a boolean expression that results in either true or false. If the condition is true, a specific block of code is executed; if the condition is false, that block is skipped.

Decision-making statements are used when a program needs to choose between different blocks of code for execution. They allow a program to break the normal sequential flow and execute different code based on conditions. These statements are essential for implementing logic and decision-making in real-world applications.

Types of Conditional Statements:

If Condition	Switch Condition
Simple-if Statement	
if-else Statement	
if-else-if Ladder	
Nested if Statement	



A. if Statement:

The if statement is the simplest decision-making statement. It is used to decide whether a particular statement or block of statements should be executed or not. If a given condition is true, the block of code inside the if statement is executed; if the condition is false, the block is skipped.

a. Simple-if Statement:

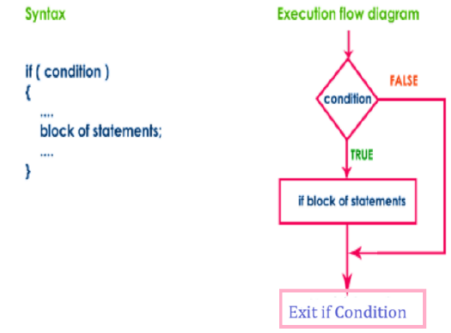
The simple if statement is the most basic form of conditional statement. It is used to execute a block of code only when a specified condition evaluates to true. If the condition is false, the control directly moves to the next statement after the if block.

In a simple if statement, the condition is evaluated first. If the condition is true, all statements inside the if block are executed. If the condition is false, none of the statements inside the block are executed.

Syntax:

```
if(condition)
{
Body of if condition;
//Executes when condition
is true
Statements;
//TBS
}
```

Flowchart:



Flowchart:

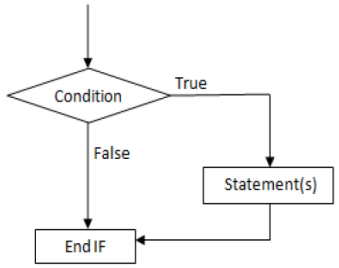
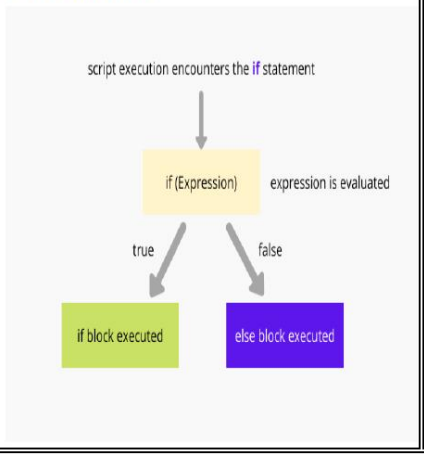
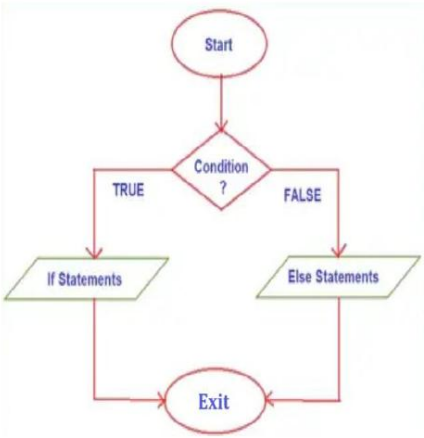


fig: Flowchart for if statement

b. if–else Statement:

The if statement executes a block of code when a condition is true. However, when we want to execute another block of code if the condition is false, we use the if–else statement. The if–else statement contains two blocks: one for the true condition and one for the false condition.

The if–else statement is an extension of the if statement. If the given condition evaluates to true, the statements inside the if block are executed and control exits the if block. If the condition evaluates to false, the statements inside the else block are executed and control exits the else block. The if–else statement is a decision-making statement used to choose between two alternative blocks of code based on the result of a test expression.

Syntax:	Flowchart:	Flowchart:
if(condition) { Body of if block; //Executes when condition is true Statements; //TBS } else { Body of else block; //Executes when condition is false Statements; //FBS }		

Flow Explanation:

Step1: When program control reaches the if–else statement, the condition is evaluated.

Step2A: If the condition is true, the statements inside the if block are executed, and control exits the if block.

Step2B: If the condition is false, the statements inside the else block are executed, and control exits the else block.

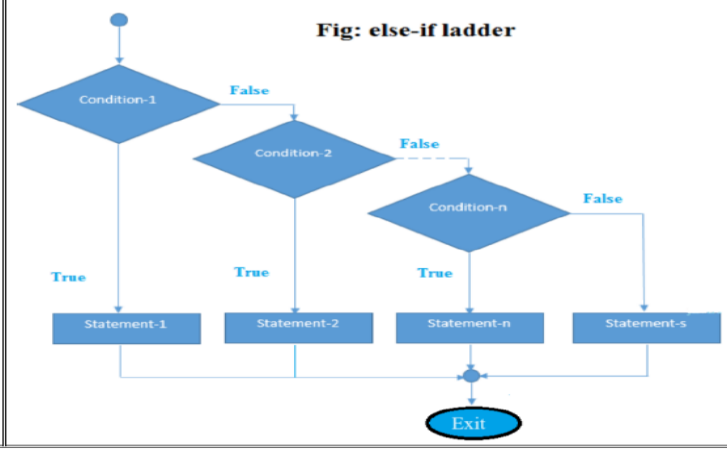
c. else–if Ladder (if–else–if ladder):

The if–else–if ladder is an extension of the if–else statement. It is used when there are multiple conditions to be checked and different blocks of code need to be executed based on which condition is true.

In an if–else–if ladder, conditions are evaluated sequentially from top to bottom.

- If the first condition is true, the corresponding if block is executed and the rest of the conditions are skipped.
- If the first condition is false, the next else–if condition is evaluated.
- This process continues until a condition evaluates to true.
- If none of the conditions are true, the statements inside the final else block are executed.

Multiple else–if blocks can be used in a ladder. The else block is optional but is executed only when all conditions are false. The if–else–if ladder is similar to the switch statement and is used when a program must choose among multiple options.

Syntax:	Flowchart:
if(condition-1) { statement – 1; //Code to be executed if condition1 is true } else if (condition -2) { statement – 2; //Code to be executed if condition2 is true } else if (condition -3) { statement – 3; //Code to be executed if condition3 is true } .... else if (condition – n) { statement – n; } else { Statement; //Code to be executed if all the conditions are false }	

Flow Explanation:

Step1: The condition in the if statement is evaluated first.

Step2A: If condition1 is true, its block is executed and the remaining conditions are not checked.

Step2B: If condition1 is false, condition2 is evaluated.

Step3: This continues for each else–if condition.

Step4: If all conditions are false, the else block is executed.

d. Nested If Statement in TypeScript:

A nested if statement is an if statement placed inside another if or else block. In other words, it is an if-within-if structure. TypeScript allows nesting of if statements, which is useful when a program needs to check multiple conditions in sequence.

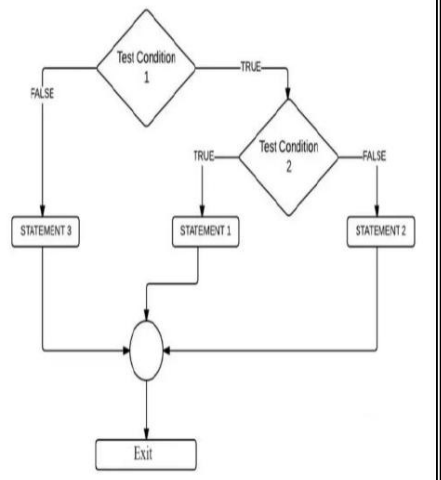
Nested if statements work in the same way as normal if–else statements. The only difference is that one if condition is evaluated only when the outer if condition is true. This structure is commonly used when decisions depend on multiple related conditions.

If the first condition is false, the program executes the else block of the outer if. If the first condition is true, the program checks the second condition and executes the corresponding block.

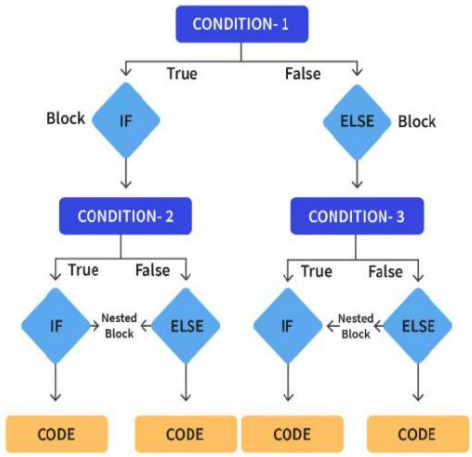
Syntax:

```
if (Condition-1)
{
//If the test Condition 1 is TRUE then these it will check for test Condition 2.
  if (Condition-2)
  {
//If the test Condition 2 is TRUE, these statements execute,
Test condition 2 True statements;
   Statement-1
  }
  else  {
 //If the c test Condition 2 is FALSE, then these statements execute,
Test condition 2 False statements;
Statements-2
  }
else  {
//If the test condition 1 is FALSE then these statements will be executed,
 Test condition 1 False statements;
Statement-3
}
```

Flowcharts:



Flowcharts:



Flow Explanation:

Step**1**: Condition1 is evaluated first.

Step**2**: If Condition1 is true, then Condition2 is evaluated.

Step**2.1**: If Condition2 is true, Statement-1 is executed.

Step**2.2**: If Condition2 is false, Statement-2 is executed.

Step**3**: If Condition1 is false, then Condition3 is evaluated.

Step**3.1**: If Condition3 is true, Statement-3 is executed.

Step**3.2:** If Condition3 is false, Statement-4 is executed.

B. Switch Statement:

When there are multiple conditions to be checked, writing many if–else statements become complex and difficult to manage. In such cases, the switch statement provides a clear and organized way to handle multiple conditions. The switch statement is a multi-way decision-making statement used to control the flow of a program.

In TypeScript, the switch statement evaluates an expression only once. The value of the expression is compared with the values of different case labels. When a match is found, the corresponding case block is executed. The break statement is used to terminate the switch block. If no case matches the expression, the default block is executed.

Syntax:

```
switch (expression)
{
case value1 : // Code statement to be executed
statement1;
break;
//break statement is used to terminate the statement sequence
case value2 :
statement2;
break;
.
.
case value n:
statement n;
break;
default: // code to be executed if all cases are not matched;
default statement;
}
```

Flowchart:

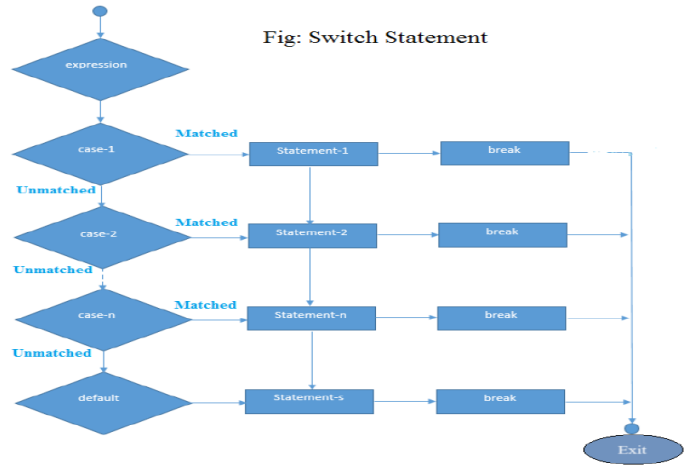


Fig: Switch Statement

Rules for Switch Statement in TypeScript:

- The switch expression can be of type number or string.
- Case values must be constant expressions.
- Duplicate case values are not allowed.
- The break statement is optional but recommended.
- If break is not used, execution continues to the next case (fall-through).
- The default case is optional and executes when no case matches.
- Only one default case is allowed, and it can appear anywhere inside the switch block.

Flow Explanation:

Step**1**: The expression inside the switch statement is evaluated once.

Step**2**: The value is compared with each case label.

Step**3**: If a match is found, the statements of that case are executed.

Step**4**: If a break statement is encountered, control exits the switch block.

Step**5**: If no case matches, the default block is executed (if present).

Break Keyword: The break keyword is used to terminate the switch statement. When break is encountered, control immediately exits the switch block. Without break, execution continues to the next case, which is known as fall-through.

Default Keyword: The default block is executed when none of the case values match the switch expression. If no default block is present and no case matches, no statements inside the switch block are executed.

Else-if vs Switch Statement:

Else-if or Ladder	Switch Statement
Used to check multiple conditions sequentially	Used to select one case from multiple options
Conditions can be complex using logical and relational operators	Conditions are based on simple equality checks
Each condition is evaluated one by one	Expression is evaluated only once
Supports range-based conditions	Does not support range conditions
Works with any boolean expression	Works with number, string, and enum values
Becomes lengthy and less readable for many conditions	More readable and structured for many cases
No fall-through behavior	Fall-through occurs if break is not used
Default handled using else block	Default handled using default case
Suitable for complex decision logic	Suitable for menu-driven or fixed-value logic

All Conditional Statements in One Table:

if Statement	**if-else Statement**	**else-if Statement**	**Nested if Statement**	**Switch Statement**
Only one condition is evaluated.	If the initial condition is false, the else block is executed.	If the initial condition is false, the else if condition is checked.	Conditions are checked in order until a true condition is found.	Expression is evaluated once, and control jumps to matching case.
Simple and straightforward, efficient for single conditions.	Efficient for handling two mutually exclusive conditions.	Efficient for handling multiple mutually exclusive conditions.	Efficiency decreases with increasing nesting levels.	Efficiency is comparable to if-else if chains.
Supports only one condition.	Supports two conditions: one for true, one for false.	Supports multiple conditions, each checked sequentially.	Supports multiple conditions, each within its own block.	Supports multiple cases, each associated with a value.
if (condition) { // code block }	if (condition) { // code block } else { // code block }	if (condition) { // code block } else if (condition) { // code block }	if (condition) { if (condition) { // code block } } else if (condition) { // code block }	switch (expression) { case value: // code break; case value: //code break; default: // code }

Executes the block of code if the condition evaluates to true.	Executes one block of code if the condition evaluates to true, otherwise executes another block.	Executes the block of code associated with the first true condition, ignoring the rest.	Executes the block of code associated with the first true condition encountered, possibly within nested blocks.	Executes the block of code associated with the matching case.
No support for multiple conditions in the same block.	Supports handling multiple conditions sequentially.	Supports handling multiple conditions sequentially.	Allows nesting multiple conditions within each other.	Supports handling multiple cases with unique code blocks.
Does not guarantee exclusive execution of blocks.	Guarantees exclusive execution of either if or else block.	Guarantees exclusive execution of one condition block.	Blocks can be nested to achieve exclusive execution.	Guarantees exclusive execution of one case block.
No default case is available.	The else block serves as the default case.	No explicit default case; last else if serves as default.	Can include a default case inside the nested structure.	Includes a default case to handle unmatched values.
May lead to code duplication if multiple conditions need to be checked separately.	Helps to avoid code duplication when handling two exclusive cases.	Code duplication may occur if handling similar conditions in multiple else-if blocks.	Code duplication may occur if similar conditions are checked in nested blocks.	Code duplication is minimal, as each case is handled separately.
Suitable for simple, one-condition checks.	Suitable for handling two exclusive cases.	Suitable for handling multiple exclusive cases.	Suitable for complex, multi-condition scenarios.	Suitable for handling multiple exclusive cases based on value.

****

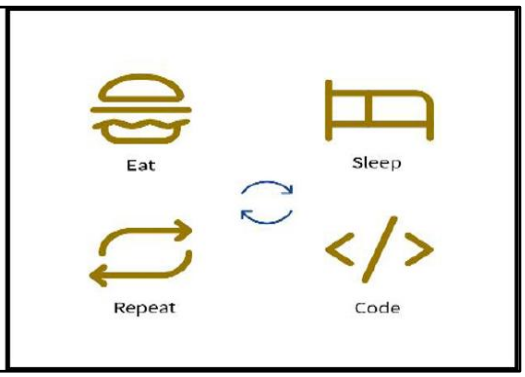
Looping (Iterative or Repetition) Statements in TypeScript

Looping statements are used to repeat a block of code multiple times. In TypeScript, loops allow a set of statements to be executed repeatedly as long as a specified condition remains true. When the condition becomes false, the loop stops executing.

Loops help avoid writing the same code again and again. Instead of repeating statements manually, a loop can be used to perform a task multiple-times efficiently. For example, to display a message 100 times, a loop can be used instead of writing the same statement 100 times.

The execution of looping statements depends on a condition. As long as the condition is satisfied, the loop continues; once the condition fails, the control exits the loop.

In order to understand what loops are, we have to look at some real-life cases of loops. Consider your daily routine, you wake up, you brush, you wear clothes and then head off to work, come back, eat and then sleep off. Again, the next day, you do the same things in the same order and go to sleep again. This cycle keeps on repeating. This concept of repetitive actions performed time and again is called a loop.



Need for Loops in TypeScript:

- Loops facilitate "Write less, Do more" – we don't have to write the same code again and again.
- They reduce the size of the code.
- Loops make the flow of control simple and structured.
- Loops are generally more efficient than recursion in terms of execution and memory usage, depending on the runtime environment.

Advantages of Loops:

- Loops allow execution of a block of code multiple times without rewriting it.
- It provides code reusability.
- Using loops, we do not need to write the same code again and again.
- By using loops, the chances of errors are reduced because repetitive code is avoided, making programs easier to maintain.

There are mainly Two types of loops:

A. Entry Controlled Loops:

In this type of loops, the test condition is evaluated before entering the loop body. If the condition is false, the loop body will not execute even once.

The **for** loop and **while** loop are entry-controlled loops.

B. Exit Controlled Loops:

In this type of loops, the test condition is evaluated after executing the loop body. Therefore, the loop body executes at least once, irrespective of whether the test condition is true or false.

The **do–while** loop is an exit-controlled loop.

In **TypeScript**, there are **4** types of loops that execute in a similar manner. However, they differ in syntax and condition checking time.

a) For Loop b) While Loop c) Do–While Loop d) Nested Loop

Elements of Loops in TypeScript:

→ **Initialization Expression(s):** Initialization is performed only once before entering the loop. Here, we declare and/or initialize the loop control variable(s) that will be used during execution. Multiple variables can also be initialized at the same time.

→ **Test Expression (Condition):** This is a boolean expression. It determines whether the loop should continue executing or terminate. If the condition evaluates to true, control enters the loop body; if it evaluates to false, the loop terminates.

→ **Body of the Loop:** The body of the loop contains the statements that are executed repeatedly. These statements continue to execute as long as the loop condition remains true.

→ **Update Expression(s):** In this step, the value of the loop variable(s) is updated. This is necessary to ensure that the loop eventually terminates. The update expression is executed after the loop body has completed execution and before the next iteration begins. In most cases, this involves incrementing or decrementing the loop variable.

In an **entry-controlled loop**, the test expression is evaluated before entering the loop body. In an **exit-controlled loop**, the test expression is evaluated after executing the loop body.

1) For Loop in TypeScript:

Loops in TypeScript are used to repeat a block of code until a specific condition is met. In a for loop, the condition is usually based on a numeric value. The for loop is used to execute a block of code a fixed number of times and is commonly used when the number of iterations is known in advance.

The TypeScript for loop consists of **3** main components that define the loop:

• Initialization statement
• Test condition
• Increment or decrement expression

A for loop executes a block of code as long as the given condition remains true. It is often called a counting loop because a counter variable is increased or decreased in each iteration. When you know exactly how many times a block of code needs to be executed, the for loop is preferred over other looping statements.

Syntax:

```
for (initialization; testcondition; increment/decrement)
 {
   // Body of the loop (statements)
}
```

Initialization: The initialization expression is used to declare and initialize the loop control variable. It is executed only once at the beginning of the loop.

Condition (Test Expression): The condition is evaluated before each iteration. If the condition evaluates to true, the loop body is executed. If it evaluates to false, the loop terminates.

Increment / Decrement: This expression updates the loop control variable after each iteration. It usually increments or decrements the variable to ensure that the loop eventually terminates.

Body of the Loop: This is the block of code that executes during each iteration of the loop as long as the condition is true.

Explanation:

• The loop starts with the initialization statement, which runs only once.

• The test condition is then evaluated. If it is true, the loop body executes.

• After the loop body executes, the increment or decrement statement is applied.

• Control returns to the condition check.

• This process continues until the condition becomes false, at which point the loop stops executing.

2) While Loop in TypeScript:

The while loop is used to execute a block of code repeatedly as long as a specified condition remains true. The condition is checked before entering the loop body, so the while loop is an entry-controlled loop. If the condition is false at the beginning, the loop body will not execute even once.

The while loop is generally used when the number of iterations is not known in advance. It continues execution until the given condition becomes false.

Syntax:

```
while (condition) {
   // Body of the loop ;(statements)
Increment / Decrement;
}
```

Initialization: The loop control variable is initialized before the while loop begins.

Condition (Test Expression): The condition is evaluated before each iteration. If the condition is true, the loop body executes. If the condition is false, the loop terminates.

Update (Increment / Decrement): The loop control variable must be updated inside the loop body. If it is not updated properly, the loop may become an infinite loop.

Body of the Loop: The body contains the statements that are executed repeatedly as long as the condition remains true.

Explanation:

• The loop control variable is initialized before the loop starts.

• The condition is evaluated.

• If the condition is true, the loop body executes.

• After execution, the control variable is updated.

• Control returns to the condition check.

• The loop continues until the condition becomes false.

3) Do–While Loop in TypeScript:

The do–while loop is used to execute a block of code at least once, even if the condition is false. In this loop, the condition is checked after executing the loop body. Therefore, the do–while loop is an exit-controlled loop.

The do–while loop is generally used when the loop body must be executed at least one time, regardless of the condition.

Syntax:

```
do {
   // Body of the loop (statements);
Increment / Decrement;
} while (condition);
```

Initialization: The loop control variable is initialized before entering the do–while loop.

Condition (Test Expression): The condition is evaluated after the loop body executes. If the condition is true, the loop continues; if the condition is false, the loop terminates.

Update (Increment / Decrement): The loop control variable is updated inside the loop body to ensure that the loop eventually terminates.

Body of the Loop: The body contains the statements that are executed repeatedly. The loop body executes at least once, even if the condition is false.

Explanation:

• The loop control variable is initialized.

• The loop body executes first.

• After execution, the condition is evaluated.

• If the condition is true, control returns to the loop body.

• If the condition is false, the loop terminates.

Differences Between for, while, and do–while Loop (TypeScript):

For Loop	While Loop	Do–While Loop
Used when the number of iterations is known in advance	Used when the number of iterations is not known	Used when the loop body must execute at least once
It is an entry-controlled loop	It is an entry-controlled loop	It is an exit-controlled loop
Condition is checked before entering the loop	Condition is checked before entering the loop	Condition is checked after executing the loop body
Loop body does not execute if condition is false initially	Loop body does not execute if condition is false initially	Loop body executes once even if condition is false initially
Initialization and update are part of the syntax	Initialization and update are written separately	Initialization and update are written separately

Best suited for counting or fixed-range loops	Best suited for condition-based loops	Best suited when at least one execution is required
No semicolon after the condition	No semicolon after the condition	Semicolon is required after the condition

4) Nested For Loop in TypeScript:

A nested loop is a loop placed inside another loop. It allows a set of instructions to be repeated multiple times within another loop. When one loop is nested inside another, the inner loop executes completely for each iteration of the outer loop.

In a nested for loop, the inner loop restarts its execution every time the outer loop executes. The inner loop must finish all of its iterations before the outer loop can move to its next iteration. Nested loops are commonly used when working with two-dimensional data, such as printing patterns in rows and columns.

Although it is possible to nest multiple loops inside one another, it is generally recommended not to exceed three levels of nesting, as excessive nesting can make the code difficult to read and maintain.

Syntax:

```
for (outerInitialization; outerCondition; outerIncrement/Decrement) {
   for (innerInitialization; innerCondition; innerIncrement/Decrement) {
      // statements of inner loop (body)
   }
   // statements of outer loop (body)
}
```

Explanation:

• The initialization value represents the starting point of the loop.

• The condition represents the endpoint of the loop.

• The increment or decrement expression updates the loop control variable to reach the condition.

• First, the outer loop starts execution with its initialization value.

• For each iteration of the outer loop, the inner loop starts executing from its initial value.

• The inner loop continues until its condition becomes false.

• Once the inner loop completes, control returns to the outer loop, which then proceeds with its next iteration.

****

Jumping Statements in TypeScript

Jumping statements are used to transfer control from one part of the program to another. These statements interrupt the normal sequential flow of execution and move the control immediately to a specific point in the program. Jumping statements are mainly used inside loops and switch statements to control program flow.

Types of Jump Statements in TypeScript:

A. Break Statement B. Continue Statement C. Return Statement

A) Break Statement in TypeScript:

The break statement is a jump statement used to terminate the execution of a loop or a switch statement. When the break statement is encountered, the control immediately exits from the loop or switch block and transfers to the statement following it.

The break statement is commonly used when a particular condition is satisfied and there is no need to continue further execution of the loop or switch.

Syntax: break;

Explanation:

- When the break statement is used inside a loop, the loop stops executing immediately.
- When the break statement is used inside a switch statement, it terminates the switch block.
- Control is transferred to the next statement after the loop or switch.
- The break statement is mainly used to avoid unnecessary iterations.

B) Continue Statement in TypeScript:

The continue statement is a jump statement used to skip the current iteration of a loop and move to the next iteration. Unlike the break statement, it does not terminate the loop; instead, it skips the remaining statements in the current iteration.

The continue statement is mainly used when a particular condition is met and the remaining code inside the loop should not be executed for that iteration.

Syntax: continue;

Explanation:

- When continue is encountered, the remaining statements in the loop body are skipped.
- Control moves directly to the next iteration of the loop.
- The loop condition is checked again before continuing.
- The continue statement can be used only inside loops.

C) Return Statement in TypeScript:

The return statement is a jump statement used to exit from a function and optionally send a value back to the caller. When the return statement is executed, the function execution stops immediately.

The return statement is commonly used to send results from a function or to terminate function execution early.

Syntax: return; or return value;

Explanation:

- The return statement ends the execution of the current function.
- It may return a value or no value depending on the function definition.
- Control is transferred back to the calling function.
- Any statements written after return inside the function are not executed.

****

Arrays in TypeScript

Arrays in TypeScript are non-primitive data types used to store multiple values of the same type in memory. An array is a homogeneous data structure, which means all elements stored in an array must be of the same data type.

An array is an ordered collection of elements, and each element is stored at a specific position called an index. The index of an array always starts from 0, which means the first element is stored at index 0, the second at index 1, and so on.

An array can be viewed as a numbered list of memory locations where each location holds a value. Instead of storing multiple values in separate variables, arrays allow storing all related values in a single variable, making programs easier to write, read, and manage.

Arrays are index-based, and for an array of size n, the index range is from 0 to n-1.

Need for Arrays:

• Arrays allow storing multiple values under a single variable name.
• They reduce the need to declare many separate variables.
• Arrays make data handling easier and more organized.
• Arrays work efficiently with loops for repetitive operations.

Array Creation in TypeScript:

Arrays can be created in **2** common ways.

A) Using Array Literal:

Syntax: let arrayName: datatype[] = [value1, value2, value3];

Example: let marks: number[] = [90, 97, 95, 99, 100];

Array elements are accessed using their index value.

```
console.log(marks[0]); // 90
console.log(marks[4]); // 100
```

For an array of 5 elements, valid index values are from 0 to 4.

B) Using the new Array():

Syntax: let arrayName: datatype[] = new Array(size);

Example: let marks: number[] = new Array(3);

Advantages of Arrays in TypeScript:

- Arrays are easy to declare, initialize, and use.
- Arrays allow storing multiple values in a single variable.
- Arrays support random access using index positions.
- Arrays can be easily processed using loops.
- Arrays can store primitive values as well as objects.
- Arrays improve code readability and organization.

Disadvantages of Arrays in TypeScript:

- Arrays store homogeneous data only.
- Managing dynamic sizes requires array methods or new arrays.
- Wasted memory may occur if more space is allocated than needed.
- Arrays are less flexible than objects for mixed data handling.

Types of Arrays in TypeScript:

1. One-Dimensional Array:

A one-dimensional array in TypeScript stores a sequence of elements of the same data type, accessed using an index. These elements are stored in contiguous memory locations conceptually, which makes accessing and manipulating them easy. A dimension refers to a single direction of data storage. A one-dimensional array can be visualized as a single row with multiple columns or a single column with multiple rows.

A one-dimensional array can store multiple values of the same type such as number, string, boolean, or objects of the same structure. All elements must be of the same type, and each element is accessed using its index position.

Declare and Initialize One-Dimensional Array:

A) Declaration with initialization (Array Literal):

Ex: let marks: number[] = [90, 97, 95, 99, 100];

In this declaration:

- ✓ number[] specifies that the array stores numeric values
- ✓ marks is the array name
- ✓ Values are stored directly at the time of declaration
- ✓ Index values will be marks[0] to marks[4]

B) Declaration first, then memory allocation using new:

Ex: let marks: number[] = new Array(5);

Memory is allocated for 5 elements, but values are not assigned yet.

Assigning values individually:

marks[0] = 90; marks[1] = 97; marks[2] = 95; marks[3] = 99; marks[4] = 100;	for (let i = 0; i < 5; i++) { marks[i] = i * 10; // example assignment }
More Examples of One-Dimensional Arrays:	
let salary: number[] = [99859.55, 15000, 14500.75, 9050]; let salary: number[] = new Array(10);	
let ch: string[] = ['K', 'R', 'I', 'S', 'H', 'N', 'A']; let ch: string[] = new Array(10);	
let colors: string[] = ["Orange", "Red", "Sky blue", "Green"]; let colors: string[] = new Array(5);	

2. Two-Dimensional Array:

A two-dimensional array represents data in the form of rows and columns (tabular or matrix form). It is essentially an array of arrays, where each element of the outer array is itself a

one-dimensional array. Two-dimensional arrays are useful for representing structured data such as marks of students, tables, grids, and matrices.

Each element in a two-dimensional array is accessed using two indexes:

→ First index → row position

→ Second index → column position

Example Scenario: Marks of three students in five subjects:

Students	Physics	Chemistry	Maths	Computer	English
Deep	80	97	80	99	82
Amit	99	98	100	81	97
Larry	87	99	93	95	97

This data can be stored using a two-dimensional array.

```
let marks: number [] [] = [
   [80, 97, 80, 99, 82],
   [99, 98, 100, 81, 97],
   [87, 99, 93, 95, 97]      ];
```

Here:

Number of rows = 3 (students) & Number of columns = 5 (subjects)

Declare and Initialize Two-Dimensional Array:

A) Declaration with initialization:

```
let marks: number[][] = [
   [50, 60, 78, 87, 95],
   [98, 87, 75, 76, 99],
   [98, 99, 97, 95, 100]    ];
```

B) Declaration first, then memory allocation:

```
let marks: number[][] = new Array(3);
```

****

Strings in TypeScript

A string is a non-primitive (reference) data type used to store a sequence of characters. Characters may include alphabets (A–Z, a–z), digits (0–9), special symbols (@ # $ % ^), and spaces. Strings are widely used to store text such as names, messages, passwords, addresses, and other textual data.

In TypeScript, strings are immutable, meaning once a string is created, it cannot be changed. Any operation on a string returns a new string, while the original string remains unchanged. Strings are index-based, and indexing starts from 0.

3 Ways to Create Strings in TypeScript:

Using double quotes " "	let s1: string = "Hello";
Using single quotes ' '	let s2: string = 'TypeScript';
Using template literals (backticks) ` `	let s3: string = `Welcome to TypeScript`;

Advantages of Strings in TypeScript

- Easy to store and manipulate text data
- Supports many built-in methods
- Template strings improve readability
- Widely used in input/output operations

Disadvantages of Strings in TypeScript

- Strings are immutable
- Frequent modifications create new string objects
- Not ideal for heavy character-by-character updates

String Methods in TypeScript:

a) toLowerCase(): Converts the string to lowercase.

```
let s: string = "STRING LOWER CASE METHOD";
console.log(s.toLowerCase());
```

Output: string lower case method

b) toUpperCase(): Converts the string to uppercase.

```
let s: string = "upper case string";
console.log(s.toUpperCase());
```

Output: UPPER CASE STRING

c) replace(): Replaces characters or words in a string.

```
let s1: string = "I am good in Java";
console.log(s1.replace("Java", "TypeScript"));
```

Output: I am good in TypeScript

d) trim(): Removes whitespace from both ends of a string.

```
let s: string = " Hello World! ";
console.log(s.trim());
```

Output: Hello World!

e) concat(): Concatenates two strings.

```
let firstName: string = "Full Stack ";
let lastName: string = "TypeScript";
console.log(firstName.concat(lastName));
```

Output: Full Stack TypeScript

f) isEmpty(): Checks whether a string is empty.

```
let s1: string = "Hello";
let s2: string = "";
console.log(s1.length === 0); // false
console.log(s2.length === 0); // true
```

g) String Comparison (===): TypeScript uses === to compare string values.

```
let a: string = "Hello";
let b: string = "Hello";
let c: string = "Hi";
console.log(a === b); // true
console.log(a === c); // false
```

Case-insensitive comparison:

```
let x: string = "JAVA";
let y: string = "java";
console.log(x.toLowerCase() === y.toLowerCase()); // true
```

h) contains() (Using includes()): Checks whether a string contains a specific sequence.

```
let msg: string = "TypeScript is powerful";
console.log(msg.includes("powerful")); // true
console.log(msg.includes("Java"));     // false
```

i) String Length: The length property returns the number of characters in a string.

```
let text: string = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
console.log(text.length); // 26
```

Accessing Characters in a String: Characters are accessed using index values.

```
let str: string = "HELLO";
console.log(str[0]); // H
console.log(str[4]); // O
```

Index range for a string of length n is 0 to n-1.

String Immutability: Strings cannot be modified directly.

```
let str: string = "Hello";
str[0] = "h"; // ❌ Not allowed
```

Correct way:

```
let str: string = "Hello";
str = "hello"; // ✓ New string created
```

Template Strings (Backticks): Template strings allow embedding variables using ${}.

```
let name: string = "Krishna";
let age: number = 25;
let info: string = `My name is ${name} and I am ${age} years old.`;
console.log(info);
```

****

OOPS (Object-Oriented Programming System)

OOPS stands for Object-Oriented Programming System.

- It is a programming paradigm based on the concept of objects, which can contain both data and methods that operate on that data.
- Object-oriented programming helps reduce code complexity and improves code reusability, maintainability, and scalability.
- It organizes programs by modelling real-world entities and their interactions, making software systems easier to understand and manage.
- The core idea of OOPS is to create objects, which are instances of classes.
- A class acts as a blueprint, and objects represent the actual entities created using that blueprint.
- OOPS promotes modularity by dividing large and complex systems into smaller, manageable components called objects.
- Each object encapsulates its own data and behavior.

Class: A class is a blueprint or template used to create objects.

It defines:

- Properties (data members)
- Methods (functions)
- A class does not occupy memory until an object is created from it.

Object: An object is an instance of a class.

- It represents a real-world entity and contains:
- Data (variables)
- Behavior (methods)
- Objects provide a modular and organized way to design programs.

Function (Method): A method is a block of code that performs a specific task when it is called. Methods help in:

- Code reusability
- Better readability
- Logical separation of functionality

A method may:

- Accept parameters or not
- Return a value or not

Types of Methods:

1. Predefined methods
2. User-defined methods
   - ✓ With ReturnType & With Parameters
   - ✓ With ReturnType & Without Parameters
   - ✓ Without ReturnType & With Parameters
   - ✓ Without ReturnType & Without Parameters

Constructor: A constructor is a special method used to initialize objects of a class.
Key characteristics:

- Constructor name is the same as the class name
- It does not have a return type
- It is automatically executed when an object is created

Types of Constructors:

- ✓ No-argument constructor
- ✓ Parameterized constructor
- ✓ Default constructor

Inheritance: Inheritance allows one class (child class) to acquire the properties and methods of another class (parent class).

Benefits:

- Code reusability
- Easy maintenance
- Establishes an "is-a" relationship
- TypeScript does not support multiple inheritance using classes.

Types of Inheritance:

- ✓ Single inheritance
- ✓ Multilevel inheritance
- ✓ Hierarchical inheritance

Note: Multiple and hybrid inheritance are achieved using interfaces, not classes.

Polymorphism: Polymorphism means "many forms".

- It allows a single action to behave differently based on the context.
- Polymorphism increases flexibility and extensibility of programs.

Types of Polymorphism:

- ✓ Compile-time polymorphism
- ✓ Runtime polymorphism

Method Overloading (MOL): Method overloading means creating multiple methods with the same name in the same class but with different parameter lists.

Purpose:

- Improves readability
- Provides flexibility
- Used to achieve compile-time polymorphism

Overloading can be done by:

- ✓ Changing the number of parameters
- ✓ Changing parameter data types
- ✓ Changing the order of parameters

Note: In TypeScript, method overloading is achieved using optional parameters or union types, not separate implementations.

Method Overriding (MOR): Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Key points:

- Method name and parameters must be the same
- Used to achieve runtime polymorphism
- Child class behavior overrides parent class behavior

Abstraction: Abstraction means exposing only the essential features of an object while hiding implementation details. It allows developers to focus on what an object does, not how it does it. Abstraction is achieved using:

- ✓ Abstract classes
- ✓ Interfaces

Encapsulation: Encapsulation is the process of bundling data and methods into a single unit (class). **Benefits**:

- Data hiding
- Controlled access to data
- Improved security and maintainability

Encapsulation is implemented using access modifiers:

- ✓ public
- ✓ protected
- ✓ private

****

FUNCTIONS (METHODS) IN TYPESCRIPT

In TypeScript, a function is a block of reusable code used to perform a specific task. When a function is defined inside a class, it is called a method. Functions may accept input values, process them, and optionally return output values. The primary purpose of using functions is to achieve code reusability, reduce program size, improve readability, and simplify maintenance.

A function executes only when it is called. By defining logic once and invoking it multiple times, redundancy is avoided. Functions also help in isolating errors, making modification easier without affecting other parts of the program.

Function Declaration in TypeScript:

To define a function, use the function keyword, followed by a name, parameter types, and a return type.

Syntax:

```
function functionName(parameter_list): returnType {
   // function body
}
```

Here:

function is Keyword

- Used to declare a function in TypeScript.
- Indicates the start of a function definition.

functionName:

- The name of the function
- Used to call or invoke the function
- Must follow valid identifier rules

Ex: calculateSum , displayResult

parameter_list:

- A list of input values passed to the function
- Parameters are enclosed in parentheses ( )
- Each parameter must have a type annotation
- Multiple parameters are separated by commas

Ex: (a: number, b: number)

: returntype:

- Specifies the type of value returned by the function
- Written after the parameter list using a colon:
- Ensures type safety
- If the function does not return a value, the return type is void

Ex: : number : string : void

Function Body:

- Enclosed within curly braces { }
- Contains executable statements

- Uses the return statement to return a value (if required)

```
Ex: function sum(a: number, b: number): number {
        return a + b;
        }
```

Function Calling in TypeScript:

A function executes only when it is called. When a function is called, program control is transferred to the called function. Control returns to the caller when the return statement is executed or when the end of the function body is reached.

Ex: sum(10, 20);

Predefined Functions:	**User-Defined Functions:**
Predefined functions are built-in functions provided by the JavaScript and TypeScript standard libraries. These functions can be used directly without user definition. **Ex**: console.log(), Math.sqrt(), and Array.isArray().	User-defined functions are created by programmers to meet specific application requirements. These functions help in organizing code and implementing business logic efficiently.

Rules for Writing Functions:

- Functions should not be written inside other functions
- Execution begins from the entry TypeScript file
- A function with return type void does not return any value
- A function with a non-void return type must return a value

Return Statement:

The return statement sends a value back to the calling function. The value returned must be compatible with the declared return type. Any statement written after the return statement will not be executed.

Types of User-Defined Functions:

User-defined functions can be classified into four types based on parameters and return values.

1. Without Return Type and Without Parameters:

This function does not take any parameters and does not return any value. It is mainly used to perform actions such as displaying messages.

```
Ex:  function greet(): void {
     console.log("Welcome");
     }
```

2. With Return Type and Without Parameters:

This function does not accept any parameters. It returns a value of a specified return type.

```
Ex:  function getNumber(): number {
      let x: number = 10;
     let y: number = 20;
```

```
    return x + y;
}
```

3. Without Return Type and With Parameters:

This function accepts one or more parameters. It performs operations using the parameters but does not return any value.

```
Ex:  function calculateSquare (x: number): void {
     let result: number = x * x;
     }
```

4. With Return Type and With Parameters:

This function accepts parameters and returns a value of a specified type. It is the most commonly used form of function in TypeScript.

```
Ex:  function multiply (a: number, b: number): number {
      return a * b;
     }
```

Method Signature:

The method signature uniquely identifies a method. In TypeScript, it includes the method name, parameter list with types, and the return type.

Ex: sum (a: number, b: number): number

Access Modifiers in TypeScript

Access modifiers define the visibility and scope of methods. TypeScript supports three access modifiers.

public: A public method can be accessed from anywhere. If no access modifier is specified, the method is public by default.

private: A private method can be accessed only within the class in which it is declared.

protected: A protected method can be accessed within the same class and by derived (child) classes.

```
Ex: class Sample {
   public show(): void {
      console.log("Public method");
   }
   private calculate(): void {
      console.log("Private method");
   }
   protected display(): void {
      console.log("Protected method");
   } }
```

Return Type:

The return type specifies the type of value returned by a function or method. If a function does not return any value, the return type is declared as void.

Common return types include: number, string, boolean, objects, arrays, and void.

Naming Conventions for Methods: Rules should be followed while naming methods:

- Method names should begin with a lowercase letter
- Camel case notation should be used
- Method names should be verbs representing actions

Ex: addNumbers(), calculateTotal(), and getStudentDetails().

Types of Methods in TypeScript:

Methods in TypeScript are classified into two categories:

1. Static Methods 2. Non-Static (instance) Methods

1. Static Methods:

A static method belongs to the class rather than to an object. It can be called without creating an instance of the class. Static methods are generally used for utility or helper operations.

```
Ex:    class Utility {
   static square (x: number): number {
      return x * x;
   }    }
Utility.square(5);
```

2. Non-Static (Instance) Methods

Non-static methods are associated with objects of a class. An object must be created before calling such methods. Instance methods can access instance variables and static members using the - this keyword.

```
Ex:    class Student {
   name: string;
   constructor(name: string) {
      this.name = name;
   }
   showName(): void {
      console.log(this.name);
   }    }
Student s1 = new Student("Krishna");
s1.showName();
```

Difference Between Static and Non-Static Methods

Static methods belong to the class and are invoked using the class name, whereas non-static methods belong to objects and require object creation. Static methods cannot directly access instance variables, while non-static methods can access both instance and static members.

→ Static methods are called using the class name, while non-static methods are called using an object of the class.

Scope of Methods: Scope of a method is determined by its access modifier:

→ public methods are accessible everywhere.

→ private methods are accessible only within the class.
→ protected methods are accessible within the class and its subclasses.

****

Constructors in TypeScript

A constructor in TypeScript is a special method of a class that is used to initialize an object. The constructor is automatically invoked when an object of a class is created using the new keyword. At the time of calling the constructor, memory is allocated for the object and its instance variables are initialized.

- A constructor in TypeScript is a special type of method that:
- Is used to initialize instance variables.
- Is called automatically at the time of object creation.
- Uses the keyword constructor.
- Does not have any return type.
- Executes only once per object.
- Unlike normal methods, constructors are not called explicitly.

Need of Constructors in TypeScript

- Constructors are used to initialize objects.
- They assign initial values to class variables.
- They are automatically executed during object creation.
- They ensure objects start in a valid state.
- The main purpose of a constructor is to initialize data members of a class.

Important Points About Constructors	**Rules for Writing Constructors in TypeScript**
✓ Constructor name must be same as Classname. ✓ A class can have only one constructor. ✓ Constructor does not return any value. ✓ Constructor is executed only once per object. ✓ Constructor can accept parameters. ✓ Constructor cannot be overridden. ✓ If no constructor is defined, TypeScript provides a default constructor.	✓ Constructor keyword must be constructor ✓ Constructor must not have any return type [even void also] ✓ Constructor can have access modifiers (public, private, protected) ✓ Constructor is automatically called when an object is created ✓ Constructor initializes instance variables

Syntax of Constructor	**Example**
class ClassName { constructor() { // constructor body } }	class Course { name: string; constructor() { console.log("Constructor Called"); this.name = "TypeScript"; } }

	let obj = new Course(); console.log("The name is " + obj.name); **Output** Constructor Called The name is TypeScript

Types of Constructors in TypeScript:

1. No-Argument Constructor:

A constructor that does not accept any parameters is called a No-Argument Constructor. If multiple objects are created using a no-argument constructor, all objects will have the same values.

Syntax:	**Example**
class ClassName { constructor() { // No-argument constructor } }	class Person { name: string; age: number; constructor() { this.name = "John Doe"; this.age = 30; console.log("Hello, my name is " + this.name + " and I am " + this.age); } } let p1 = new Person();

2. Parameterized Constructor:

A constructor that accepts one or more parameters is called a Parameterized Constructor. Parameterized constructors are used to:

- Initialize objects with different values.
- Assign user-defined values at runtime.

Syntax	**Example**
class ClassName { constructor(parameters) { // parameterized constructor } }	class Person { name: string; age: number; constructor(name: string, age: number) { this.name = name; this.age = age; console.log("Name: " + name); console.log("Age: " + age); } } let person1 = new Person("John Doe", 30); let person2 = new Person("Jane Doe", 25);

3. Default Constructor:

If a class does not define any constructor, TypeScript automatically provides a default constructor. The default constructor initializes instance variables with undefined values.

Example	**Output**
class Product { id: string; productName: string; price: number; } let product = new Product(); console.log(product.id); console.log(product.productName); console.log(product.price);	undefined undefined undefined

****

Inheritance in TypeScript

Inheritance is one of the fundamental concepts of Object-Oriented Programming (OOP).
Inheritance is a mechanism in which one class (child/subclass) inherits the properties and methods of another class (parent/superclass).
The class whose properties are inherited is called the Parent class, and the class that inherits is called the Child class.
Inheritance allows a class to acquire the properties and methods of another class. In TypeScript, inheritance is used to reuse code, reduce duplication, and establish a relationship between classes. **In TypeScript, inheritance is implemented using the extends keyword.**

Purpose of Inheritance in TypeScript

- To achieve code reusability.
- To avoid code duplication.
- To improve maintainability.
- To support method reuse.
- To establish an IS-A relationship between classes.

Syntax and Example of Inheritance:

class ParentClass { // parent class members }	class ChildClass **extends** ParentClass { // child class members }
// Define the parent class 'Vehicle' class Vehicle { // Initialize the Vehicle with color and brand attributes constructor(public color: string, public brand: string) {} }	// Define the child class 'Car', inheriting from 'Vehicle' class Car **extends** Vehicle { constructor (color: string, brand: string, public doors: number) { // Call the parent class's constructor method to set color and brand Super (color, brand); } }

Types of Inheritance in TypeScript:

TypeScript supports only Single Inheritance and Multilevel Inheritance.

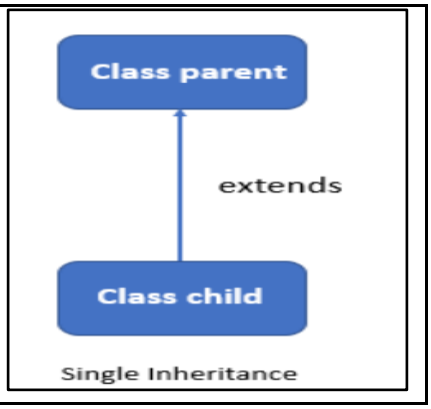
1. Single-Level Inheritance:

Single-level inheritance is a type of inheritance where one class (child class) inherits the properties and methods of only one other class (parent class). In this concept, there is only one base class and one derived class. The child class receives all the accessible features from a single parent class, which improves code reusability and allows adding new functionality without rewriting existing code.

Syntax:

In TypeScript, single-level inheritance is implemented using the "extends" keyword.

```
//Parent Class
class Parent {
// parent properties and methods
}
//Child Class
class Child extends Parent {
// child properties and methods
}
```



Explanation:

In single-level inheritance, the child class directly inherits from only one parent class. The child class can use all public and protected members of the parent class and can also define its own methods. This helps in reducing code duplication and makes the program more structured and maintainable. Unlike multiple inheritance, TypeScript allows inheriting from only one class at a time, which keeps the design simple and avoids ambiguity.

Example Program:

```
class Animal {
eat(): void {
console.log("eating...");
}
}
```

```
class Dog extends Animal {
bark(): void {
console.log("barking...");
}
}
```

```
class TestInheritance {
static main(): void {
let d = new Dog();
d.bark(); // child method
d.eat();  // parent method
} }
TestInheritance.main();
```

Output

```
barking...
eating...
```

In this example, the `Dog` class inherits the `eat()` method from the `Animal` class and also has its own method `bark()`. This clearly demonstrates single-level inheritance where one child class derives properties from a single parent class.

2. Multilevel Inheritance:

Multilevel inheritance is a type of inheritance where a class inherits from another class, and then another class inherits from that derived class. In this concept, there is a chain of inheritance involving more than two classes, such as grandparent, parent, and child. Each level inherits the properties and methods from its immediate parent, enabling code reusability across multiple levels.

Syntax:

In TypeScript, multilevel inheritance is implemented using the `extends` keyword in a chain.

```
//Parent Class
class Parent
{
// properties and methods
}
```

```
//Child1 Class
class Child1 extends Parent
{
// properties and methods
}
```

```
//Child2 Class
class Child2 extends Parent
{
// properties and methods
}
```

In multilevel inheritance, a class does not only inherit from its direct parent but also indirectly inherits from all ancestor classes. This means the final derived class can access the properties and methods of the base class as well as intermediate classes. This structure helps in organizing code logically and avoids repetition by sharing common functionality across multiple levels. TypeScript supports this type of inheritance naturally using the 'extends' keyword.

Example Program:

```
class Animal
{
eat(): void
{
console.log("eating...");
} }
```

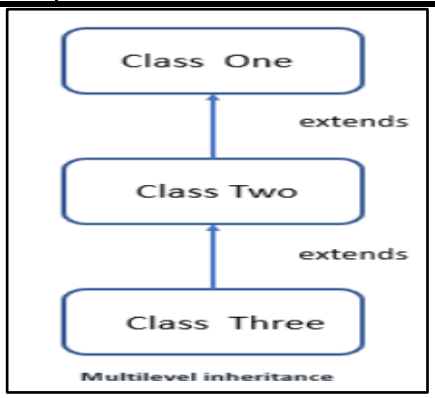
```
class Dog extends Animal
{
bark(): void
{
console.log("barking...");
} }
```

```
class Puppy extends Dog
{
weep(): void
{
console.log("weeping...");
} }
```

```
class TestInheritance {
static main(): void {
let p = new Puppy();
p.weep(); // child method
p.bark(); // parent method
p.eat();  // grandparent method
} }
TestInheritance.main();
```

Output

```
weeping...
barking...
eating...
```



In this example, the `Puppy` class inherits from `Dog`, and `Dog` inherits from `Animal`. Therefore, `Puppy` can access methods from both `Dog` and `Animal`, demonstrating multilevel inheritance where properties are passed through multiple levels of classes.

3. Hierarchical Inheritance:

Hierarchical inheritance is a type of inheritance where multiple child classes inherit the properties and methods from a single parent class. In this structure, one base class is shared by more than one derived class.

Syntax:

```
class Parent
{
// properties and methods
}
```

```
class Child1 extends Parent
{
// properties and methods
}
```

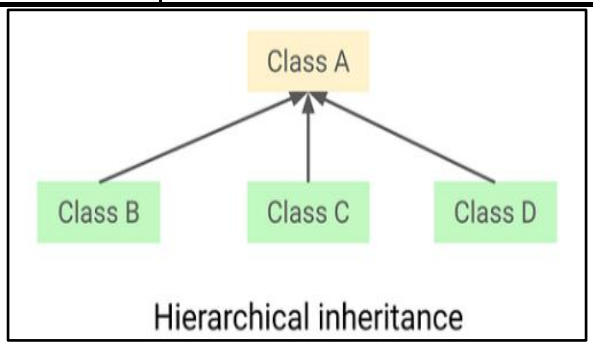
```
class Child2 extends Parent
{
// properties and methods
}
```

Explanation:

In hierarchical inheritance, a single parent class acts as a base for multiple child classes. Each child class can access the properties and methods of the parent class and can also define its own specific features. This helps in reusing common code across different classes while maintaining separate functionalities for each child class.

Example Program:

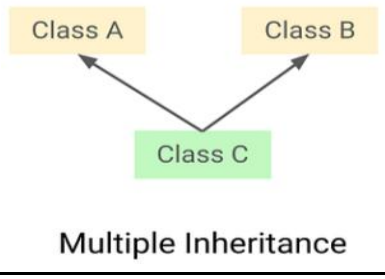
class Animal { eat(): void { console.log("eating..."); } }	**class Dog extends Animal** { bark(): void { console.log("barking..."); } }	**class Cat extends Animal** { meow(): void { console.log("meowing..."); } }

class TestInheritance { static main(): void { **let d = new Dog();** d.eat(); d.bark(); **let c = new Cat();** c.eat(); c.meow(); } } TestInheritance.main();	**Output** eating... barking... eating... meowing...	

4. Multiple Inheritance:

Multiple inheritance is a type of inheritance where one class inherits the properties and methods from **more than one parent class**. However, TypeScript does not support multiple inheritance using classes.

Syntax (Invalid in TypeScript):

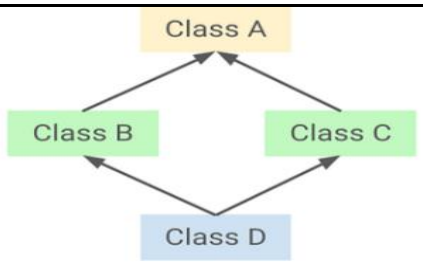
class A {} class B {}	class C extends A, B {} // Not allowed	

TypeScript does not allow a class to extend more than one class because it can create **ambiguity** and **complexity**. Therefore, multiple inheritance is not directly supported with classes. Instead, TypeScript provides interfaces to achieve similar functionality.

Hybrid Inheritance:

Hybrid inheritance is a combination of two or more types of inheritance (such as hierarchical and multiple inheritance). TypeScript does not support hybrid inheritance directly using classes.

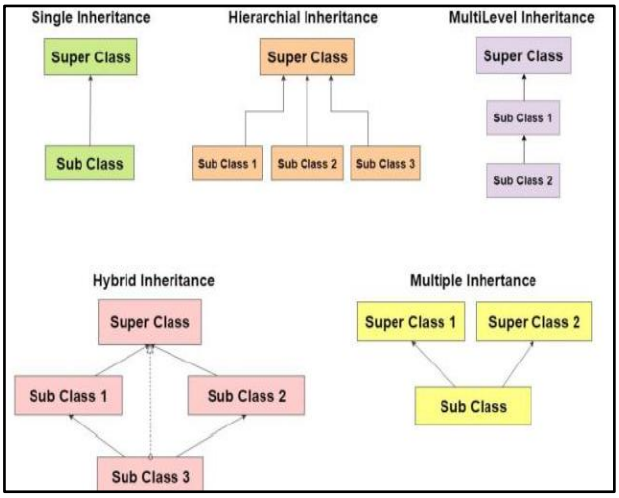
Syntax (Using class + interface combination):

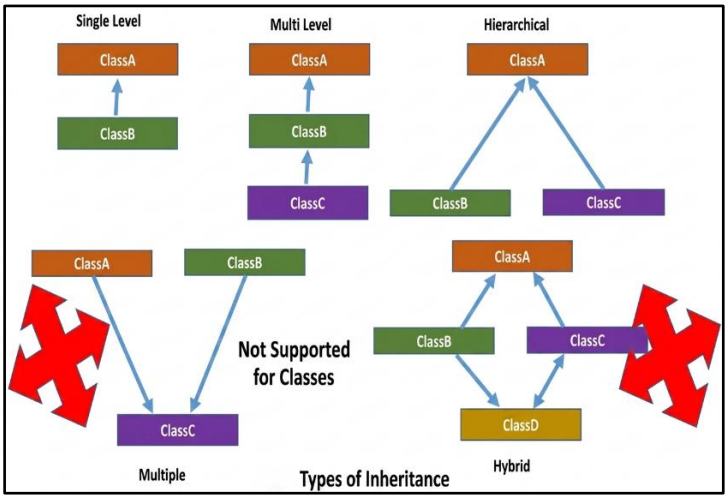
class A { // parent class }	**interface B** { // interface methods }	**class C extends A implements B** { // implementation }	

Hybrid inheritance cannot be implemented directly with classes in TypeScript because multiple inheritance is not supported. However, it can be achieved by combining class inheritance with interfaces. A class can extend one class and implement multiple interfaces, which simulates hybrid inheritance behavior.

The hybrid inheritance can be achieved by using the following combinations:

- Single and Multiple Inheritance (not supported directly in TypeScript but can be achieved through interfaces)
- Multilevel and Hierarchical Inheritance (supported in TypeScript)
- Hierarchical and Single Inheritance (supported in TypeScript)
- Multiple and Multilevel Inheritance (not supported in TypeScript)





****

Polymorphism in TypeScript

The word polymorphism is derived from two Greek words: **poly** and **morphs**. The word "poly" means many and "morphs" means forms. Therefore, polymorphism means “many forms”. That is, one thing that can take many forms.

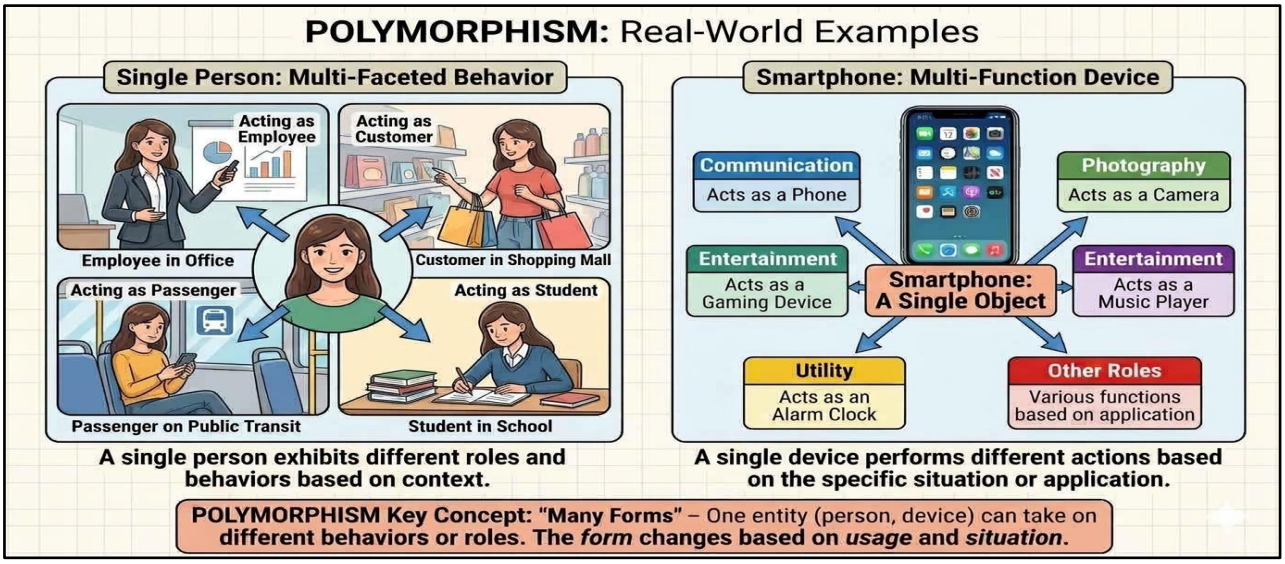
Polymorphism in TypeScript is a concept by which we can perform a single action in different ways. That is, when a single entity behaves differently in different situations, it is called polymorphism in TypeScript. Polymorphism means "many forms", and it occurs when we have multiple classes or objects that are related through inheritance or interfaces.

We can achieve flexibility in our code using polymorphism because we can perform various operations using methods with the same name but different implementations depending on the requirement. Polymorphism in TypeScript is one of the important concepts of Object-Oriented Programming (OOP).

Real-time Example of Polymorphism in TypeScript:

→ The best example of polymorphism is Human Behavior. One person can have different behaviours. For example, a person acts as an employee in the office, a customer in the shopping mall, a passenger in a bus/train, a student in school, and a son at home.

→ Another example is your Smart Phone. A smartphone can act as a phone, camera, music player, alarm, etc., performing different roles based on the situation. This demonstrates polymorphism.



Types of Polymorphism in TypeScript:

There are **2** types of polymorphism in TypeScript:

1) Static Polymorphism (Compile-Time Polymorphism).

2) Dynamic Polymorphism (Runtime Polymorphism).

1) Static Polymorphism in TypeScript (Compile-time Polymorphism):

Static polymorphism is also known as Compile-Time polymorphism. In this type, the behavior of a function is determined during compilation. In TypeScript, this is achieved mainly through **Function OverLoading**. The compiler decides which function signature to use based on the parameters passed.

It is achieved using function overloading, where multiple function signatures are defined with the same name but different parameter types. The TypeScript compiler determines which function to use based on the arguments.

Unlike Java, **TypeScript** does not support traditional Method or Constructor overloading in implementation, but it allows multiple function signatures with a single implementation.

Static polymorphism is achieved by:

- **Function Overloading**
- **Type-based behavior (Union Types)**

1) Function Overloading:

When multiple function signatures are defined with the same name but different parameters, it is called function overloading.

Function overloading is a feature in TypeScript that allows a function to have multiple definitions with the same name but different parameter lists. It enables writing flexible and reusable functions while maintaining type safety.

Function Overloading is the process of defining multiple function signatures for a single function name, where each signature differs in: **Number of parameters, or Type of parameters**

A function overload consists of:

- Overload Signatures (Declarations)
- One Implementation Function

Syntax:

```
// Overload Signatures
function functionName(param: type): returnType;
function functionName(param1: type, param2: type): returnType;
// Implementation
function functionName(param1: any, param2?: any): any {
   // function body
}
```

However, all these signatures are implemented using one single function body.

In TypeScript:

- Multiple signatures are declared.
- Only one implementation is provided.
- The correct behavior is decided at compile time.

Execution Process:

- When a function is called, TypeScript checks the arguments.
- It matches the call with the appropriate overload signature.
- This matching happens at compile time, not at runtime.
- At runtime, only the single implementation function is executed.

Rules of Function OverLoading:

- All overloads must have the same function name.

- Overloads must differ in parameter type or count.
- Only one implementation function is allowed.
- Implementation must handle all overload cases.
- Overload signatures must be placed before the implementation.

To Achieve Function Overloading:

1. **Changing the Number of Parameters (Overloading by Number of Parameters).**
2. **Changing the Type of Parameters (Overloading by Data Type).**

→ Overloading by Number of Parameters → Different argument count

→ Overloading by Data Type → Same arguments, different types

→ Both achieve compile-time polymorphism in TypeScript

1. Changing the Number of Parameters (Overloading by Number of Parameters):

Function overloading can be achieved by defining multiple function signatures with the same name but different number of parameters.

- One function may accept fewer arguments
- Another function may accept more arguments
- The implementation handles all cases using optional parameters

Example Program: Overloading by Number of Parameters

```
function add(a: number, b: number): number;
function add(a: number, b: number, c: number): number;
function add(a: number, b: number, c?: number): number {
   if (c !== undefined) {
      return a + b + c;
   }
   return a + b;
}
```

2. Changing the Type of Parameters (Overloading by Data Type).

Function overloading can also be achieved by defining functions with the same name but different parameter types.

- Same number of parameters
- Different data types (e.g., number, string)
- Implementation uses a general type (any or union)

Example Program: Overloading by Data Type

```
function display(value: number): number;
function display(value: string): string;
function display(value: any): any {
   return value;
}
display(100);       // number
display("Hello");   // string
```

2) Type-based behaviour (Union Types):

Type-based behaviour in TypeScript means a single function handles multiple data types and executes different logic based on the detected type using type narrowing. This is considered a form of static polymorphism because the compiler helps enforce correctness at compile time.

A) Union Types: A value that can be one of several designated types, written with a pipe symbol like string | number.

Ex: let value: string | number;

Ex: function process(value: string | number) {

Here, value can be string OR number

B) Type Narrowing (Type Guards): Using runtime checks (typeof, in, or custom tags) so the compiler safely treats the value as a specific sub-type.

Ex: if (typeof value === "string") {

Now TypeScript knows → value is string.

2) Dynamic Polymorphism (Runtime Polymorphism):

Dynamic polymorphism is also known as Runtime Polymorphism. In this type, the behavior of a function is determined during execution (runtime) rather than at compile time. Dynamic polymorphism is a process where the system chooses which method to run while the program is running.

In TypeScript, this is achieved mainly through Method Overriding using inheritance. The decision about which method to execute is made based on the actual object at runtime, not the reference type. This is done using method overriding, where a child class changes a method made by its parent class.

Unlike static polymorphism, the compiler does not decide the method. Instead, the runtime engine performs dynamic dispatch.

It is achieved using **Method Overriding**, where:

- A child class provides a specific implementation of a method already defined in the parent class.
- The method name and parameters remain the same.
- The execution depends on the object type at runtime.

Key Characteristics:

→Same Method name →Same Parameters

→Requires Inheritance →Decision happens at runtime

→Different implementations in parent and child classes

Dynamic polymorphism is achieved through method overriding, where the method execution is determined at runtime based on the actual object type."

****

Data Abstraction in TypeScript

In TypeScript, abstraction helps reduce complexity by allowing developers to focus on what an object does rather than how it does it. By hiding unnecessary details, abstraction makes code easier to understand, maintain, and reuse.

Abstraction in TypeScript is the Object-Oriented Programming (OOP) principle of hiding internal implementation details and exposing only the essential interface to the user. It reduces code complexity by ensuring developers interact with what an object does rather than how it does it. Abstraction is the process of hiding implementation details and showing only essential functionality to the user.

It helps:

- Reduce complexity.
- Improve maintainability.
- Provide security (hide internal logic).

Real-Life Examples:

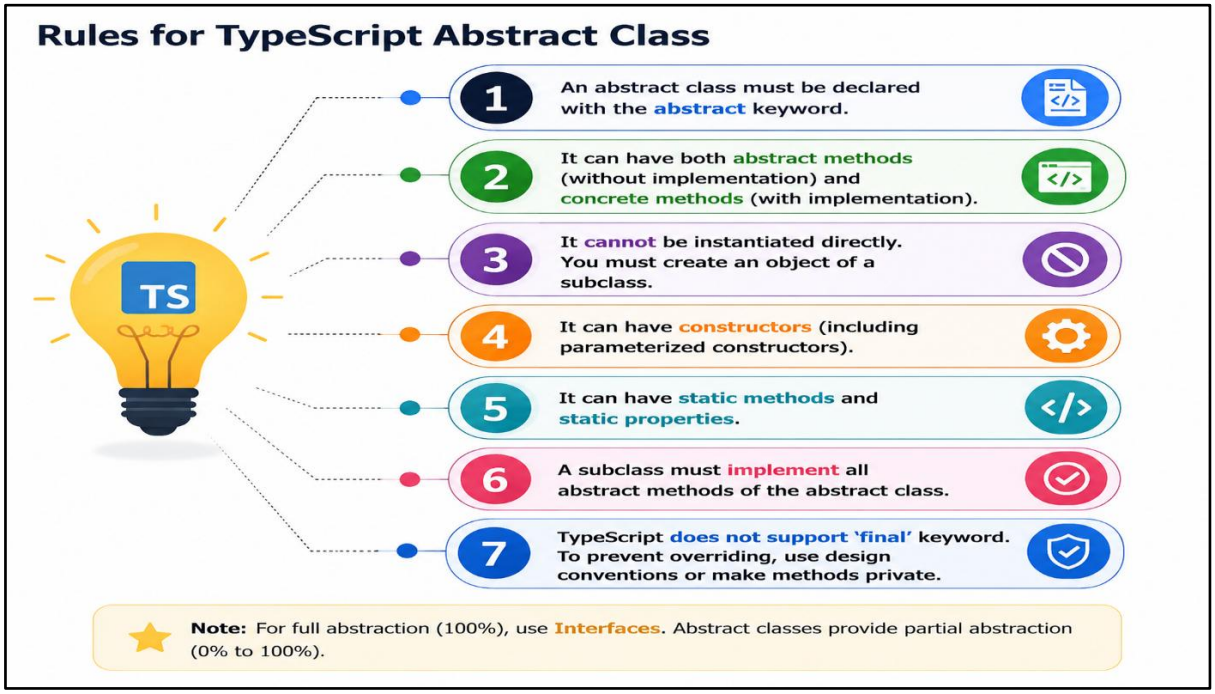
While driving a car, you use the steering wheel, accelerator, and brakes without knowing the internal working of the engine. Similarly, in programming, abstraction allows users to interact with an object through a simple interface without exposing its complex implementation.

📱 Mobile → You press call, but don't know internal network flow.

💬 SMS → You send message, but don't see backend processing.

🚗 Car → You accelerate/brake, but don't know engine mechanism.

Abstract classes help define a common structure for subclasses, ensuring that specific methods exist across all implementations. Abstract classes can include methods with implementations to provide common behavior while allowing subclasses to override them if needed.



Abstract classes can contain both implemented methods and abstract methods (methods without an implementation). Any concrete (non-abstract) class extending an abstract class must provide implementations for all of the abstract methods. This feature helps enforce structure and predictability in TypeScript code. To define an abstract class, use the abstract keyword before the class declaration. Any method inside an abstract class that lacks an implementation must also be marked as abstract.

<table>
<tr><td>abstract class Animal {
abstract makeSound(): void;
// Must be implemented by subclasses
move(): void {
console.log("Moving...");
} }</td><td>class Dog extends Animal {
makeSound(): void {
console.log("Bark!");
}
}</td></tr>
<tr><td colspan="2">const myDog = new Dog();
myDog.makeSound(); // Output: Bark!
myDog.move(); // Output: Moving...</td></tr>
<tr><td colspan="2">▪ The Animal class is abstract, meaning you cannot create an instance of it.
▪ makeSound() is an abstract method, so every subclass must implement it.
▪ move() is a regular method that all subclasses inherit automatically.</td></tr>
</table>

****

Interface in TypeScript

In TypeScript, an interface is a contract (blueprint) that defines the structure of an object by specifying its properties and methods without providing implementation. It ensures type safety, improves code reliability, and promotes abstraction and loose coupling. Interfaces describe the shape of data, making it easier to maintain consistency across simple and complex applications.

Interface defines the syntax for classes/objects to follow, which means a class/object which implements an interface is bound to implement all its members. The interface contains only the declaration of the methods and fields, but not the implementation. It is a shape of an object.

Declaring a TypeScript Interface:

We can declare an interface using the interface keyword followed by the interface name and interface body. For example, here we are declaring an interface called Person with two properties, firstName and lastName of the string type. And this means that any variable of type Person must define these two properties.

```
interface Person {
  firstName: string;
  lastName: string;
}
```

Key Points for Interface in TypeScript:

- An interface is a contract (structure) that defines properties and method signatures.
- It does not contain implementation, only declarations.
- A class implements an interface using the implements keyword.
- An interface can extend one or more interfaces (supports multiple inheritance).
- A class can implement multiple interfaces.
- Interfaces are not compiled into JavaScript (used only at compile-time).
- We cannot instantiate an interface directly.
- Properties in an interface can be:

Abstract Class vs Interface:

Abstract classes	Interfaces
Work best when we are trying to build up the definition/blueprint of an object.	Work best when we have very different objects that we want to work with together.
Work best when we are trying to build up the definition/blueprint of an object.	Serve as a “gatekeeper” to a function (function can enforce the contract)
We can't inherit more than one abstract class.	We can implement more than one interface in a class.
Strongly couple classes together (class is dependent on another class)	Promote loose coupling (class is dependent on an interface rather than a class for the implementation)

- While both abstract classes and interfaces define structures, they serve different purposes.
- Abstract classes can contain method implementations and state (properties).
- Interfaces only define method signatures and cannot include implementations.

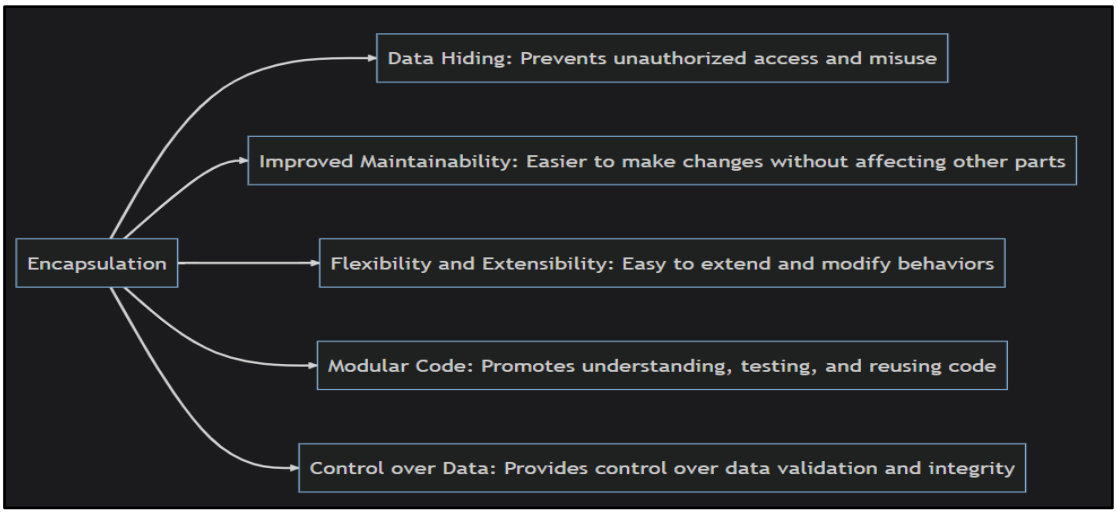
interface Animal { makeSound(): void; }	class Bird implements Animal { makeSound(): void { console.log("Chirp!"); } }

****

Encapsulation in Typescript

Encapsulation in object-oriented programming refers to restricting unauthorized access and mutation of specific properties of an object. Encapsulation bundles data and the operations that we perform on them into one unit, namely an object. It guards data against unwanted alterations, ensuring the creation of robust and maintainable software. In TypeScript, encapsulation is enhanced with type annotations, providing additional checks at compile-time to ensure error-free operation.

Consider a TypeScript class representing a bank account. **Without encapsulation**, the account balance could be directly altered. **With encapsulation**, the balance can only change through specified methods, like depositing or withdrawing. TypeScript enhances this with type annotations for class properties and method parameters. Encapsulation restricts direct access to an object's data and prevents unwanted data alteration. This principle is comparable to window blinds, allowing you to look out while preventing others from peeping in.



In TypeScript, access modifiers are used to achieve encapsulation. By default, all class properties in a class are public. This default behavior can be overridden by prefixing the properties with access modifiers.

Access Modifiers: An access modifier is a keyword that changes the accessibility of a property or method in a class.

There are **3** primary access modifiers in TypeScript:

public: This is the default visibility of every class property. A public property is accessible outside the class.

private: A property prefixed with the private keyword can’t be accessed anywhere outside the class and cannot be inherited by a subclass.

protected: The protected access modifier is very similar to the private access modifier with one key difference. Properties marked with the protected keyword are visible and can be inherited by a subclass.

In addition to the main three, TypeScript has two more access modifiers:

static: Properties or methods prefixed with static can only be accessed directly on the class and not on an object instantiated from the class. They also can't be inherited.
readonly: Properties prefixed with readonly can't be modified; their values can only be read. Since read-only properties cannot be modified, they must be set at the class declaration or inside a constructor function.
Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.

Realtime Example of Encapsulation in Java:

Suppose you have an account in the bank. If your balance variable is declared as a public variable in the bank software, your account balance will be known as public, in this case, anyone can know your account balance. So, would you like it? Obviously, No.

```
class BankAccount {
   public balance: number = 50000;
}
let account = new BankAccount();
console.log(account.balance);  // Anyone can access
account.balance = 100000;      // Anyone can modify
```

This is not secure because the balance can be directly accessed or changed from outside the class. To achieve data hiding and security, we declare the balance as private.
So, they declare balance variable as private for making your account safe, so that anyone cannot see your account balance. The person who has to see his account balance, will have to access only private members through methods defined inside that class and this method will ask your account holder name or user Id, and password for authentication. Thus, we can achieve security by utilizing the concept of data hiding. This is called Encapsulation in Java.

```
class BankAccount {
private balance: number = 50000;
public getBalance(): number {
return this.balance;
} }
let account = new BankAccount();
console.log(account.getBalance());  // ✅ Allowed
// console.log(account.balance);    // ❌ Error
// account.balance = 100000;        // ❌ Error
```

Here:

- balance is private, so it cannot be accessed directly from outside the class.
- getBalance() is public, so it provides controlled access to the balance.

- This concept of hiding data and providing controlled access through methods is called Encapsulation.

Getter and Setter Methods in Encapsulation:

Within encapsulation, TypeScript uses getter and setter methods to access or modify private fields. In a class, the getter method retrieves the field's value, and the setter method alters it.

- A **getter** in TypeScript is a method-like accessor that is used to retrieve or return the value of a private member variable. It allows controlled access to data that is hidden inside a class.
- In TypeScript, getters are defined using the get keyword.
- For every private variable that needs to be read from outside the class, we can provide a getter.
- A **setter** in TypeScript is an accessor that is used to update or modify the value of a private member variable.
- In TypeScript, setters are defined using the set keyword.
- A setter allows us to control how a private variable can be modified. We can also perform validation or business logic before assigning a new value.

```
class BankAccount {
private balance: number = 50000;
// Getter
get accountBalance(): number {
return this.balance;   }
// Setter
set accountBalance(amount: number) {
if (amount >= 0) {
this.balance = amount;
} else {
console.log("Balance cannot be negative");
}     }   }
let account = new BankAccount();
// Getter
console.log(account.accountBalance);
// Setter
account.accountBalance = 75000;
// Getter
console.log(account.accountBalance);
```

Output:

```
50000
75000
```

****

Exception Handling in TypeScript

An error is an event that occurs during the execution of a program that prevents it from continuing normally. Errors can be caused by a variety of factors, such as **Syntax** errors, **Runtime** errors, and **Logical** errors.

Error handling in TypeScript is a process to detect and handle errors that occurs during the execution of a program. Errors can be a syntax, runtime or logical errors. An error occurred during the execution of the program is called a runtime error or an exception.

An error can occur due to programming mistakes, incorrect user input, etc. Errors can disrupt code execution and lead to bad user experience. Effective error & exception handling is required for building robust, reliable and user-friendly applications in Typescript.

Syntax Errors:

Syntax errors, also called parsing errors, occur when TypeScript code does not follow the correct syntax rules. Syntax errors are errors in the structure or grammar of the code, and they are generally detected by the TypeScript compiler (tsc) before the program runs.

For example, the following TypeScript code has a syntax error because the closing parenthesis) is missing:

Error Code:	**Correct code:**	When you compile the incorrect code:
function displayMessage (message: string { console.log(message); } Here, the) is missing after string.	function displayMessage(message: string) { console.log(message); }	**npx tsc** TypeScript reports an error similar to: error TS1005: ')' expected.

Runtime Errors:

Runtime errors, also called exceptions, occur during the execution of a program, after the TypeScript code has been compiled into JavaScript.

For example, the following TypeScript code has correct syntax, but it causes a runtime error because it attempts to call a method that does not exist on the window object.

window.printme();

The code can pass the syntax/compilation stage, but when the program executes, the browser tries to find and call printme(). Since this method does not exist, a runtime exception occurs.

TypeError: window.printme is not a function.

Logical Errors:

Logical errors occur when the program has correct syntax and does not produce a runtime exception, but the program gives an incorrect or unexpected result because the logic or algorithm is wrong.

These errors are often difficult to identify because the TypeScript compiler does not consider the logic itself to be incorrect.

Logical Error Code:	**Correct code:**	**But the expected result is:**
let num1: number = 20; let num2: number = 30; let average: number = **num1 + num2 / 2;** console.log("Average:", average); **Output** is: Average: 35	let num1: number = 20; let num2: number = 30; let average: number = **(num1 + num2) / 2**; console.log("Average:", average); **Output** is: Average: 25	There is no syntax error, and the program runs successfully. However, the logic is incorrect. The problem is operator precedence. The expression: num1 + num2 / 2 is evaluated as: 20 + (30 / 2) = 20 + 15 = 35

Exception handling in TypeScript is a mechanism used to handle runtime errors so that the normal flow of program execution can be controlled instead of allowing the program to terminate unexpectedly. An exception is an unwanted or unexpected event that occurs during the execution of a program and disrupts the normal flow of instructions.

Functional programming languages such as TypeScript and JavaScript provide a way to handle errors using the **try-catch** blocks.

The **try{}** block catches the errors, and the catch block handles the errors. The try block can contain the code having errors or possibilities of the errors. If any error occurs, it throws the error and stops executing the code from the line where it occurred.

The **catch**{} block statement is used to catch and handle errors. We can write the code to handle the errors in the catch block according to the different errors. There can be multiple catch blocks for a single try block. We can create a different catch block to handle different kinds of errors, such as syntax and reference errors. If no error occurs in the code of the try{} statement, it will skip the code of the catch{} statement and execute the next lines of the code.

The **finally{}** block allows you to execute the particular code after completing the execution of the try{} and catch{} statements and the code that is always necessary to execute before program termination.

Syntax: The syntax below to use the try, catch, and finally block.

```
try {
// Code with error
} catch {
// handle the error
} finally {
// execute the code before the program termination
}
```

try { let user: any = null; console.log(user.name); } **catch {** console.log("An error occurred"); } **finally {** console.log("Program execution completed"); } **Output:** An error occurred Program execution completed	**try {** console.log("Hello TypeScript"); } catch { console.log("An error occurred"); } **finally {** console.log("Finally block executed"); } **Output:** Hello TypeScript Finally block executed

****

MODULE-3

Introduction to Playwright

Getting Started:

Playwright Test is an end-to-end test framework for modern web apps created by **Microsoft**. It was first released in **2020** and has quickly gained adoption among QA teams and developers. It bundles test runner, assertions, isolation, parallelization and rich tooling. Playwright supports Chromium, WebKit and Firefox on Windows, Linux and macOS, locally or in CI, headless or headed, with native mobile emulation for Chrome (Android) and Mobile Safari.

- Playwright is an open-source Node.js library allowing users to control web applications and conduct End to End test automation, making Playwright automation a key player in modern software testing.
- Playwright works with Chromium, Firefox, and WebKit, which means you can test across Chrome, Edge, Safari, and Firefox with a single API. Unlike older tools, you don't need separate drivers or a complex setup.
- Playwright stands out is its multi-language support. You can write tests in JavaScript, TypeScript, Python, Java, and .NET, making it flexible for teams with different tech stack.
- Playwright is also designed to support modern web apps like SPAs (single-page applications) (Ex:Gmail, Facebook, Twitter etc..) and PWAs (progressive web apps)(native mobile or desktop app..)

Playwright Features:

1) Cross-Browser Support: Playwright lets you use one API to test Chromium family (Chrome and Edge), Firefox, and WebKit(Safari). That matters because many bugs only show up in one engine. Instead of maintaining separate configs and drivers, you run the same tests everywhere with a small config change. This reduces code drift and cuts maintenance. It also makes your suite a real "cross-browser" suite, not just "Chrome-only."

2) Cross-Platform Compatibility: You can run Playwright on Windows, macOS, and Linux, both locally and in CI. That means your laptop, your workstation, and your build server all behave the same way. The installer fetches the right browser binaries, so setup is fast. In CI, Playwright integrates with GitHub Actions, Jenkins, GitLab, and Azure.

3) Multi-Language Support: Playwright supports JavaScript/TypeScript "natively," and also offers official bindings for Python, Java, and .NET. That lets mixed teams adopt one tool without forcing a language switch. You can standardize on a single framework while keeping devs in their comfort zone.

4) Headless and Headed Modes Execution: Headless mode runs the browser without a visible window. It's fast and perfect for CI. Headed mode shows the real browser, great for debugging and demos. You can flip between them with a flag or config.

5) Automatic Waiting: Playwright waits for elements to be actionable before it clicks,

types, or reads. It also waits for navigation and network to reach stable states. That means fewer manual sleep() calls and fewer timing bugs. When the DOM updates, the locator re-resolves; when the element becomes visible, the action proceeds. This is the biggest reason Playwright tests are less flaky than many Selenium-style suites. You still can (and should) add explicit waits when the app truly needs them, but the framework handles the common timing pain automatically.

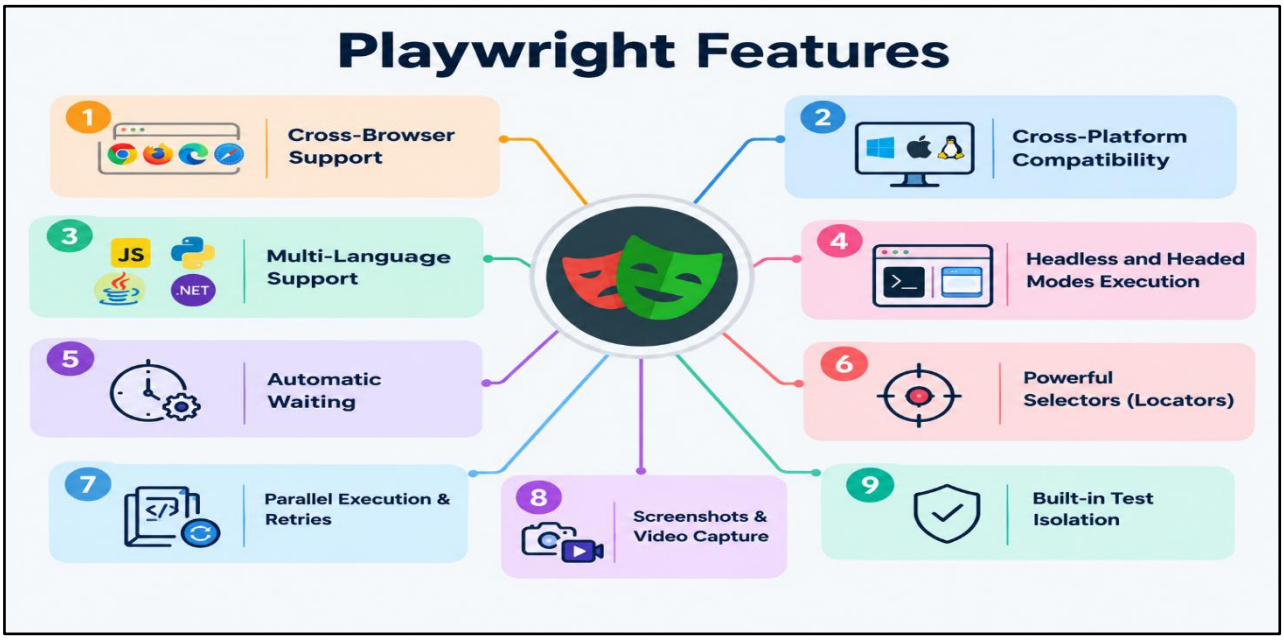
6) Powerful Selectors(Locators): Playwright provides reliable locator methods such as getByRole(), getByText(), getByLabel(), getByPlaceholder(), getByTestId(), and locator().

7) Parallel Execution & Retries: Playwright can execute multiple tests in parallel using workers, which helps reduce overall test execution time. It also supports retries to handle truly flaky cases while you investigate.

8) Screenshots & Video Capture: Playwright can capture screenshots on demand and record video of each test automatically or manually during test execution for debugging and test evidence.

This is great for debugging UI glitches, reporting bugs with proof, and auditing visual flows. Enable video at the context level and store it only for failed tests to save space.

- **HTML Reporting**: Playwright provides a built-in HTML reporter that displays test results, failures, execution time, screenshots, videos, and traces.
- **Device Emulation:** Playwright can emulate mobile devices, screen sizes, browsers, user agents, geolocation, permissions, and other device characteristics.
- **CI/CD Integration:** Playwright can be integrated with CI/CD platforms such as Jenkins, GitHub Actions, GitLab CI, and Azure Pipelines for automated test execution.
- **Debugging Support:** Playwright provides debugging features such as Playwright Inspector, --debug, and page.pause() for investigating test execution.
- **Web-first Assertions:** Playwright provides built-in assertions that automatically retry until the expected condition is met or the timeout is reached.
- **Trace Viewer:** Playwright Trace Viewer provides detailed information about test execution, including actions, screenshots, DOM snapshots, network activity, and timing.
- **Fixtures:** Playwright Test provides built-in fixtures such as page, browser, context, and request, and also allows users to create custom fixtures. Instead of manually writing repetitive setup and teardown code for every single test case, you define a fixture. The testing framework then automatically injects that fixture directly into your test functions when requested.
- **Test Retries:** Playwright can automatically retry failed tests based on the configured retry count, which is useful for handling temporary failures.
- **Code Generation:** Playwright Codegen can record user interactions in the browser and generate corresponding Playwright test code.
- **Authentication State Management:** Playwright allows authentication state such as cookies and local storage to be saved and reused across tests, reducing repeated login operations.



Common Features: Selenium vs Playwright:

Feature	Selenium	Playwright
Browser Automation	✅	✅
Cross-Browser Testing	✅	✅
Locators	✅	✅
Element Interaction	✅	✅
Click	click()	click()
Assertions	TestNG/JUnit assertions	Playwright expect()
Waits / Synchronization	Implicit, Explicit, Fluent	Auto-waiting+explicit waits
Headless Execution	Supported	Supported
Parallel Execution	Supported	Built-in support
Page Object Model	✅	✅
Data-Driven Testing	TestNG/JUnit + external data	Playwright Test + external data
Test Reports	TestNG/JUnit/Allure etc.	HTML, JSON, JUnit etc.
CI/CD Integration	✅	✅

Different Features: Selenium vs Playwright:

Feature	Selenium	Playwright
Auto-Waiting	❌Not built-in like Playwright	✅ Built-in
Browser Installation	Usually uses installed browsers + drivers	✅Downloads/manages browser binaries
Browser Contexts	❌ No direct equivalent	✅ Built-in
Video Recording	❌ Not a core feature	✅ Built-in
Fixtures	❌ No equivalent built into Selenium	✅ Built-in
Parallel Execution	Usually handled through TestNG/JUnit/Grid	✅ Built into Playwright Test
Locator Auto-Retrying	❌	✅
Web-first Assertions	❌	✅
Codegen	❌ No equivalent built into Selenium	✅ codegen
Inspector / Debugging Tools	External/browser tools	✅ Playwright Inspector
Network Interception	Limited / additional setup	✅ Built-in
API Testing	❌ Not a core Selenium feature	✅ Built-in APIRequest
Tracing	❌ No equivalent built into Selenium	✅ Built-in Trace Viewer

Difference between Playwright and Selenium

Playwright	Selenium
Uses a modern architecture with direct communication to browser engines	Uses browser-specific drivers like ChromeDriver and GeckoDriver
Built-in auto-waiting for elements and actions	Requires explicit waits for synchronization
Supports Chromium, Firefox, and WebKit using one single API	Supports multiple browsers but needs separate drivers for each
Faster execution with better stability	Slower execution and more prone to flaky tests

Comes with built-in test runner and reporting	Depends on external frameworks for execution and reporting
Parallel execution is supported out of the box	Parallel execution needs additional setup and tools
Designed specifically for modern and dynamic web applications	Older architecture with limited native support for modern web apps

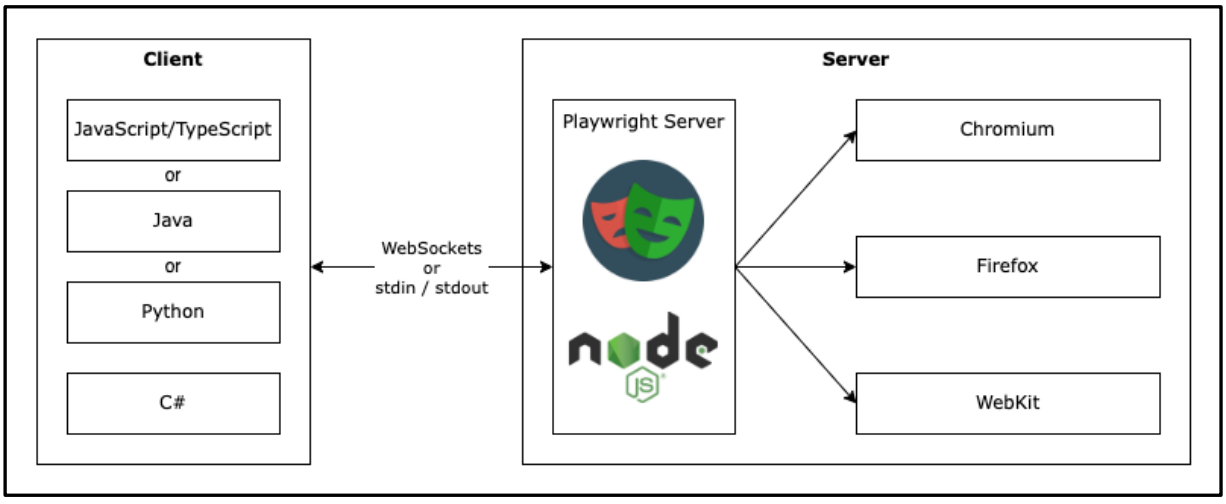
Playwright Architecture:

Playwright follows a modern, layered automation architecture that allows test scripts to communicate directly with browser engines in a fast, stable, and isolated manner. Unlike traditional automation tools that rely on external drivers or protocols, Playwright communicates with browsers using a native, low-level browser protocol, which improves performance and reliability.

In Playwright, the automation process begins with the test script written by the tester using a supported programming language such as TypeScript or JavaScript. These test scripts use Playwright APIs, which provide methods to perform actions like opening a browser, navigating to a page, clicking elements, entering text, and validating results.

The Playwright APIs internally communicate with the browser engines (Chromium, Firefox, and WebKit). These browser engines are controlled directly by Playwright without the need for separate browser drivers such as ChromeDriver or GeckoDriver. This direct communication removes the dependency on intermediary components and reduces failures caused by version mismatches.

Each test execution in Playwright runs inside a Browser Context. A browser context is an isolated environment within a browser that has its own cookies, cache, storage, and session data. This allows multiple tests to run independently in the same browser without interfering with each other. Browser contexts provide better test isolation and support parallel execution.



The complete communication flow in Playwright architecture works as follows:

- The tester writes automation scripts using Playwright APIs in a programming language such as TypeScript.

- The Playwright APIs translate these commands into low-level browser instructions.
- These instructions are sent directly to the browser engine (Chromium, Firefox, or WebKit).
- The browser engine performs the requested actions on the web application.
- The results and responses are sent back to the Playwright APIs and then to the test script.

Playwright Supported Browsers:

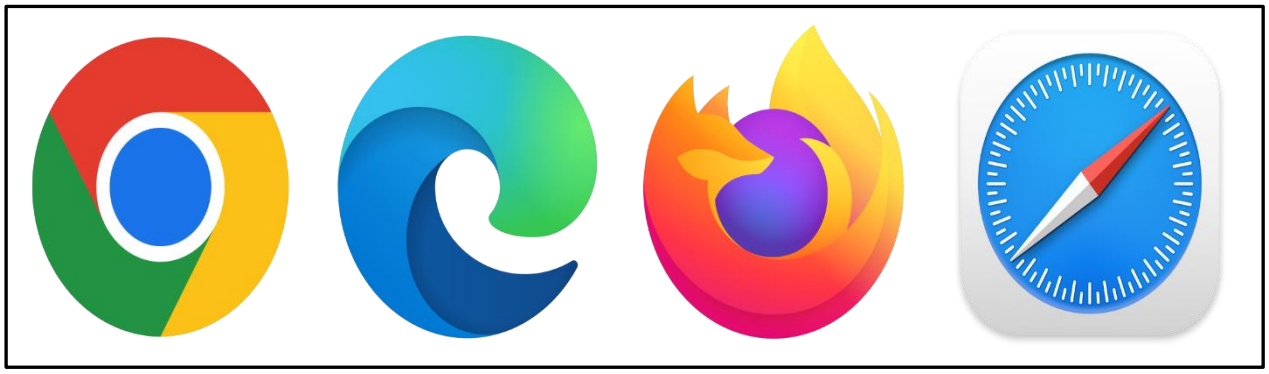
Playwright supports multiple browser engines, which allows testers to perform cross-browser testing using a single automation framework. A browser engine is the core component responsible for rendering web pages and executing browser actions.

Playwright supports the following Browser Engines:

Chromium: Chromium is the open-source browser engine used by browsers such as Google Chrome and Microsoft Edge. When Playwright runs tests on Chromium, it can validate application behavior on Chrome- and Edge-based browsers.

Firefox: Firefox is supported as an independent browser engine in Playwright. This allows automation of applications on Mozilla Firefox without any external browser drivers.

WebKit: WebKit is the browser engine used by Apple Safari. Playwright supports WebKit automation, which helps testers validate application behavior on Safari-based browsers.



All supported browsers in Playwright are automated using the same Playwright API. This means the same test script can be executed on Chromium, Firefox, and WebKit without any code changes. This ensures consistent test behavior across different browsers and simplifies cross-browser testing.

Playwright setup is the first and most important step before starting automation testing. Proper setup ensures that Playwright works correctly, browsers launch without errors, and test execution is stable. Without correct setup, automation scripts may fail due to configuration or environment issues.

- **Installing Playwright (Installation):**

Get started by installing Playwright using one of the following methods. Installing Playwright means setting up the Playwright automation framework in a project so that automated tests can be written and executed. Playwright is a Node.js–based framework, so it is installed using npm (Node Package Manager).

Before installing Playwright, Node.js must be installed on the system because npm is required to download and manage Playwright dependencies.

Step 1: Install Node.js

Playwright is built on Node.js. Node.js must be installed before installing Playwright because it provides the runtime environment required to execute Playwright commands and automation scripts.

URL: https://nodejs.org/en/download

Verify the Node.js version: **node -v** # Should print current version [Ex:"v24.16.0"]

Verify npm version: **npm -v** # Should print current version [Ex:"11.13.0"]

Step 2: Create a Project Folder

Create a new folder where all Playwright automation files will be stored. This folder acts as the root directory of the automation project.

Step 3: Open Command Prompt or Terminal

Open the command prompt or terminal and navigate to the project folder. All Playwright installation commands are executed from this location.

Step 4: Initialize Playwright

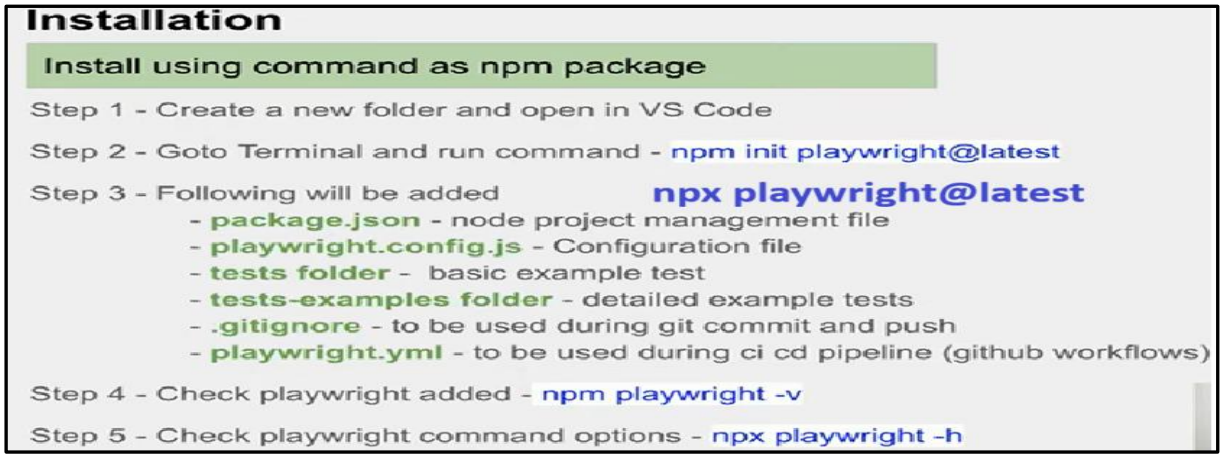
The Playwright project is initialized using the Playwright setup command. This step creates the basic project structure, installs required dependencies, and prepares the framework for automation. Playwright automatically downloads supported browser engines such as Chromium, Firefox, and WebKit. These browsers are managed internally by Playwright and do not require separate driver installation.

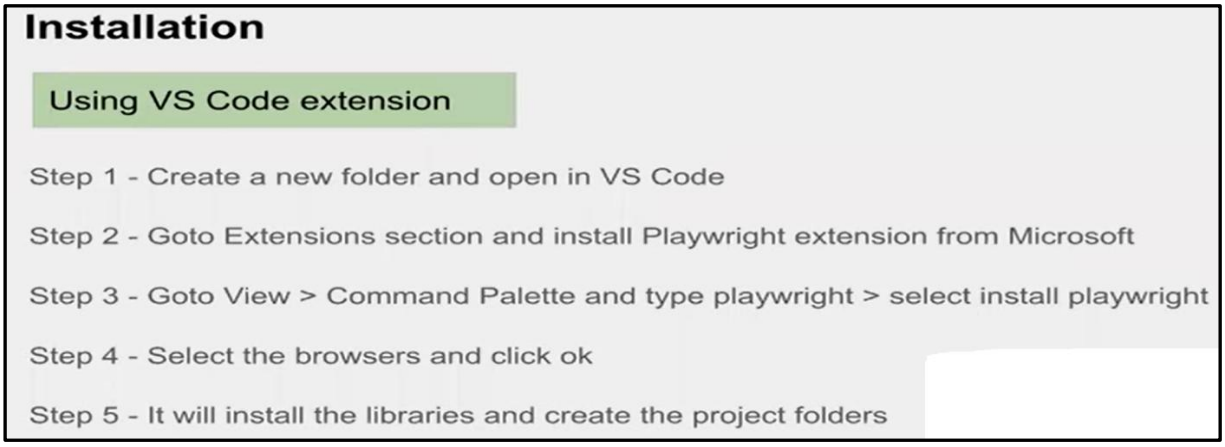
Command: npm init playwright@latest

This command installs:

Playwright downloads required browser binaries and creates the scaffold below.

- Playwright automation library
- Playwright built-in test runner (@playwright/test)
- Built-in assertion library (expect)
- Browser binaries (Chromium, Firefox, WebKit)
- TypeScript Configuration (optional, based on selection)
- Playwright configuration file (playwright.config.ts)
- Sample test files and project folder structure
- Required internal and runtime dependencies





Step 5: Verify Installation

To check the installed Playwright version:

Command: npx playwright --version [Ex:" Version 1.62.1"]

After setup, a sample test is executed to verify that Playwright is installed correctly and the environment is ready for automation.

Command: npx playwright test (or) npx playwright test example.spec.js

Note: When you run **npx playwright test**

- **If the test runs successfully:** The required browser is installed and available for use.
- **If the browser is missing:** Playwright will display an error indicating that the required browser executable is missing and will usually provide the command npx playwright install to install the required browsers.

Command: npx playwright install

➢ **Writing Tests [Simple Example Playwright Script]**

▪ **Launch a browser** ▪ **Open a web application** ▪ **Verifies the page title** ▪ **Close the browser after execution** This confirms that Playwright setup is successful and ready for further automation development.	**import { test, expect } from '@playwright/test';** **test('Verify page title – Playwright setup check', async ({ page }) => {** **await page.goto('https://example.com');** **await expect(page).toHaveTitle('Example Domain');** **const title = await page.title();** **console.log('Page Title:', title);** **});**
Running Playwright Test: npx playwright test **To run in headed mode (see browser):** npx playwright test --headed	

Playwright Execution Commands:

✓ Run tests in Headless mode (default browser) **npx playwright test** ✓ Run tests in headed mode (default browser)

npx playwright test --headed

✓ Run tests only on Firefox (headless)

npx playwright test --project=firefox

✓ Run tests on Firefox in headed mode

npx playwright test --project=firefox --headed

✓ Run only one specific test on Firefox (headed)

npx playwright test tests/SwagTests/SwagValidLogin.spec.ts --project=firefox --headed

✓ Run tests by test name on Firefox (headed)

npx playwright test -g "has title" --project=firefox --headed

✓ Run tests on Chrome (Chromium) in headed mode

npx playwright test --project=chromium --headed

✓ Run tests on WebKit (Safari engine) headed

npx playwright test --project=webkit --headed

✓ Run tests on Microsoft Edge headed

npx playwright test --project=msedge --headed

✓ Run tests in Debug mode

npx playwright test tests/SwagTests/SwagValidLogin.spec.ts --project=firefox --debug

✓ List all configured projects and tests

npx playwright test --list

✓ Show last HTML test report

npx playwright show-report

****

Headed and Headless Mode Execution in Playwright

Playwright allows browser tests to run in two execution modes: headed mode and headless mode. In headed mode, the browser window is visible during execution, and the tester can see all actions such as navigation, clicks, and validations. In headless mode, the browser runs in the background without displaying the user interface.

The method used to control headed or headless execution depends on how Playwright is implemented in the project. Playwright can be used either with the built-in Test Runner (framework approach) or with manual browser launch (script-based approach). Each approach controls the execution mode in a different way.

Test Runner (Framework Approach):

In the Test Runner approach, Playwright automatically manages the browser lifecycle. This means the tester does not manually launch or close the browser inside the test script. The Playwright Test Runner handles browser startup, test execution, reporting, and cleanup.

This approach is used in real-time projects because it provides better structure, scalability, reporting, parallel execution, and configuration control. When using the Test Runner, headed or headless execution is controlled through the configuration file or through the command line, not inside the test script.

There are two ways to control headed or headless execution in the framework approach:

1. Using Configuration File:

Execution mode can be defined inside the **playwright.config.ts** file.

Example:

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  use: {
    headless: false
  }              });
```

headless: false → Browser UI will be visible (Headed mode)
headless: true → Browser runs without UI (Headless mode)
This setting applies to all tests in the project.

2. Using Command Line:

Execution mode can also be controlled directly from the command line.

Headed execution:	Headless execution (default):
npx playwright test --headed	**npx playwright test**

This method is useful when you want to temporarily switch execution mode without changing the configuration file.

****

Debugging Playwright Tests

Debugging is the process of identifying and fixing errors in automation scripts. In Playwright, debugging helps testers understand why a test failed, where it failed, and what caused the failure. Since automation tests interact with dynamic web applications, failures can occur due to synchronization issues, incorrect locators, navigation problems, or unexpected application behavior.

Debugging is not only about fixing failures. It is about understanding how the automation script interacts with the application. A good automation engineer should be comfortable using headed mode, debug mode, and trace viewer for efficient failure analysis.

Playwright provides built-in debugging tools that make failure analysis easier and more efficient.

Why Debugging is Important:

- Tests may fail due to incorrect element locators.
- Timing issues may occur if elements are not ready.
- Application behavior may change unexpectedly.
- Network delays may affect execution.

Methods to Debug Playwright Tests:

1. Using Playwright Debug Mode:

Playwright provides a debug mode that pauses execution and opens the Playwright Inspector. The inspector allows step-by-step execution of test commands.

Command: **npx playwright test --debug**

In debug mode, testers can:

- Step through each action
- Inspect locators
- Resume or pause execution
- View action logs

This is one of the most powerful debugging features in Playwright.

2. Running Tests in Headed Mode:

Headed mode allows the tester to visually observe browser actions during execution. By watching the browser, testers can identify where the script is failing.

Command: npx playwright test --headed

In this mode, the browser UI is visible, making it easier to track test flow.

3. Using Slow Motion (SlowMo):

Slow motion execution slows down browser actions so that each step can be observed clearly. This is helpful for understanding execution sequence.

This is configured in the Playwright configuration file.

Example:

```
use: {
  launchOptions: {
    slowMo: 1000
```

} }

Note: The value 1000 means each action is delayed by 1 second.

4. Using Console Logs:

Testers can use console.log() statements inside test scripts to print values during execution.

Example: console.log("Page Title:", await page.title());

This helps verify variable values and understand execution flow.

5. Using Trace Viewer:

Trace Viewer records detailed execution information, including:

- Screenshots
- Network requests
- DOM snapshots
- Console logs

To enable trace collection:

```
use: {
  trace: 'on'
}
```

After execution, the trace report can be opened using:

npx playwright show-trace trace.zip

Trace Viewer helps analyze failures in detail, especially in CI environments where the browser UI is not visible.

****

Verify Page TITLE and URL (Title & URL Verification)

Verifying the page title and URL is one of the most basic and important validations in automation testing. These validations confirm that the application has successfully navigated to the correct page.

When a user performs actions such as login, clicking a link, or submitting a form, the application usually redirects to another page. Title and URL verification ensures that this redirection has happened correctly.

<table>
<tr>
<td>Why Title Verification is Important
▪ The page title represents the name of the webpage displayed in the browser tab. Verifying the title helps confirm:
▪ The correct page is loaded
▪ Navigation is successful
▪ The application did not redirect to an error page
▪ If the title does not match the expected value, it indicates a navigation or application issue.</td>
<td>Why URL Verification is Important
▪ The URL (Uniform Resource Locator) represents the address of the webpage. Verifying the URL ensures:
▪ The application redirected to the correct endpoint
▪ No unexpected page was opened
▪ The correct environment (QA, Stage, Production) is being tested
▪ URL verification is especially important after login, form submission, or clicking important navigation links.</td>
</tr>
<tr>
<td>Steps to Verify Page Title
• Launch the browser.
• Navigate to the application URL.
• Wait for the page to load completely.
• Validate the page title using Playwright assertion.
Example: Title Verification
import { test, expect } from '@playwright/test';
test('Verify the page title', async ({ page }) => {
await page.goto('https://playwright.dev/');
const title = await page.title();
console.log('Page Title:', title);
await expect(page).toHaveTitle(/Playwright/);
});</td>
<td>Steps to Verify Page URL
• Launch the browser.
• Navigate to the application URL.
• Wait for the page to load completely.
• Validate the page title using Playwright assertion.
Example: URL Verification
import { test, expect } from '@playwright/test';
test('Navigate and Validate URL', async ({ page }) => {
await page.goto('https://playwright.dev');
const currentURL = page.url();
console.log('Current URL:', currentURL);
await expect(page).toHaveURL('https://playwright.dev') ; });</td>
</tr>
<tr>
<td colspan="2">• Title and URL verification are basic navigation validations.
• Always verify title or URL after login or redirection.
• Use Playwright’s built-in assertions for automatic validation.</td>
</tr>
</table>

- Avoid using manual comparison logic when assertion methods are available

****

Web Elements & User Actions

A web application is made up of many components. These components are called web elements. In automation testing, interacting with web elements is one of the most important tasks. Web elements are the components present on a webpage, such as buttons, input fields, links, checkboxes, and dropdown menus. Before performing any action like click, type, or select, Playwright must first identify the element on the webpage. This identification is done using locators and selectors. Accurate element identification is critical because if the wrong element is identified, the automation script will fail or behave unexpectedly.

Examples of web elements:

- Text Box / Edit Box
- Text Area
- Buttons
- Links
- Checkboxes
- Radio buttons
- Dropdown lists

1. Text / Edit Box: Text / Edit Box is a basic text control that enables a user to type a small amount of text. In HTML, an edit box is represented using the <input> tag with type attribute having the value as "text".

Ex: <input type="text" id="username">

Ex: await page.locator('#username').fill('Krishna');

Ex: await page.getByRole('textbox', { name: 'Username' }).fill('Krishna');

2. Text Area: A Text Area is designed to input more than one line of text. It provides a larger input space compared to a text field. A text area is identified using the <textarea> tag in HTML.

Ex: <textarea id="comments"></textarea>

Ex: await page.locator('#comments').fill('Automation Testing Feedback');

Ex: await page.locator('#comments').fill('Sample Text');

TextField → Used to enter a single line of text.

TextArea → Used to enter multiple lines of text.

Interaction with input fields is common when automating Playwright web applications. Playwright offers two main methods for entering text into input fields: **fill**() and **type**(). Both are used for input operations, and they have different behaviors and use cases.

fill() method:

The fill() method in Playwright is used to populate form fields with a specified value. It works by directly setting the value of an input field, effectively replacing any existing content with the new value. It's designed for scenarios where you need to quickly fill out fields without simulating individual keystrokes.

fill() clears the existing value of an input field and then sets the new value in a single operation.

EX: await page.locator('#username').fill(standard_user);

type() method:

The type() method, on the other hand, simulates typing by sending individual keystrokes to the input field. It's ideal for scenarios where you want to replicate the user's actual typing behavior, including simulating typing delays or triggering input events like keydown, keypress, or keyup. This method is more realistic because it mimics how a user would enter text one character at a time.

type() does NOT clear existing text automatically

EX: await page.locator('#username').type('kanchan123');

Differences Between fill() and type():

Feature	`fill()`	`type()`
Clears existing text	✅ Yes	❌ No
Speed	⚡ Faster	🐢 Slower
Simulates real typing	❌ No	✅ Yes
Sends individual keyboard events	❌ No	✅ Yes
Suitable for simple form filling	✅ Yes	⚠️ Depends
Supports typing delay	❌ No	✅ Yes

3. **Button:** A Button represents a clickable element used to perform actions such as submitting a form, opening a dialog, cancelling an action, or executing a command. Buttons are created using <button> tag or <input type="submit">.

Ex: <button id="loginBtn">Login</button>

To click a button in Playwright, locate the element and use the **click**() method.

Ex: await page.locator('#loginBtn').click();

Ex: await page.getByRole('button', { name: 'Login' }).click();

4. **Link (Hyperlink):** A link, also known as a hyperlink, connects one web page to another. It allows users to navigate from page to page. A link is created using the anchor tag <a> in HTML.

Ex: Home

Types of links:

- Internal links – Navigate within the same website.
- External links – Navigate to another website.
- Backlinks – Links from another website to your website.

In Playwright, links can be identified using role, visible text, CSS selector, or XPath.

Ex: await page.getByRole('link', { name: 'Home' }).click();

Links can also be accessed using partial text:

Ex: await page.getByText('Hom').click();

5. CheckBox: A CheckBox is a web element that allows users to select one or multiple options. It can be checked or unchecked. A checked checkbox appears tick-marked, while an unchecked checkbox is unticked.

Checkboxes are defined in HTML using: <input type="checkbox" id="agree">

In Playwright, checkboxes are handled using the check() and uncheck() methods.

Ex: await page.locator('#agree').check(); [To Check]

Ex: await page.locator('#agree').uncheck(); [To uncheck]

To verify whether it is selected:

Ex: await page.locator('#agree').isChecked();

6. Radio Button: A Radio Button is a graphical element that allows the user to select only one option from a group of options. Selecting one radio button automatically deselects others within the same group.

Radio buttons are defined in **HTML** using: <input type="radio" name="gender" value="male">

In Playwright, radio buttons are handled using the **check**() method.

Ex: await page.locator('#male').check();

Only one radio button in a group can be selected at a time.

7. Dropdown List: A Dropdown List is a form element that presents a list of options to a user in a dropdown menu format. The user can select a single option from the predefined list. Dropdowns are commonly created using the <select> tag in HTML.

Ex:
```
<select id="country">
 <option value="india">India</option>
 <option value="usa">USA</option>
</select>
```

In Playwright, dropdown options are selected using the **selectOption**() method.

Ex: await page.locator('#country').selectOption('india');

Select by **visible** Text: await page.locator('#country').selectOption({ label: 'India' });

Select by **index**: await page.locator('#country').selectOption({ index: 0 });

****

Playwright Locaters and Selectors

In Playwright, web elements are handled using locators. A locator is a way to find and interact with elements on a web page. Locators in Playwright – Auto-Waiting & Retry-ability.

In Playwright, locators are not just simple selectors. They are the central piece that make tests stable and less flaky.

- A locator is a way to identify and point to a specific element within a user interface or a data structure.
- **Auto-waiting:** A locator automatically waits for the element to be ready before acting on it. You don't need to scatter manual sleeps throughout your code.
- **Retry behavior:** Playwright keeps retrying the action until the element meets the required conditions or a timeout is reached. This makes tests far less prone to random failures.
- Think of a locator as the address of a web element inside the webpage's DOM (Document Object Model).
- A locator represents a "live pointer" to an element (or multiple elements) on the page.
- Unlike Selenium, Playwright does not grab the element immediately.
- Instead, it re-evaluates the element on each action (click, fill, expect).
- This ensures your tests are always interacting with the current state of the page.
- Locators, auto-wait + retry until the element is ready.

Playwright auto-waits until the button is:

- Attached to the DOM
- Visible
- Enabled
- Stable (not moving/animating)
- If the element doesn't appear immediately, Playwright keeps retrying.
- Prevents flaky failures due to slow network or animations.

Why This is Important

- ✓ No more manual waits → reduces flakiness.
- ✓ Dynamic handling → works well with modern SPAs (React, Angular).
- ✓ Cleaner code → fewer explicit wait statements.

Simple understanding:

- Locator = Tool used to find element
- Selector = The method used to identify element

Playwright provides:

- ✓ Built-in auto-waiting
- ✓ Smart element handling
- ✓ Role-based locators
- ✓ Modern selector engine

I. Built-in Locators in Playwright:

Playwright provides powerful built-in locator strategies that are designed to make tests more stable, readable, and aligned with user behavior. These locators are based on accessibility attributes and visible content, making automation more reliable compared to traditional CSS or XPath selectors.

Playwright provides **7** built-in locators that allow us to interact with web elements in a stable, readable, and accessibility-friendly way. These locators are recommended because they are based on user-facing attributes such as roles, labels, text, and accessibility properties rather than CSS or XPath expressions.

getByRole() — Finds elements by their role and accessible name. Best for interactive items like buttons, links, and checkboxes.
getByText() — Locates elements by their visible text content. Great for non-interactive elements such as paragraphs or spans.
getByLabel() — Finds form controls through their associated label text.
getByPlaceholder() — Targets inputs using their placeholder text.
getByAltText() — Locates images or other elements by their alt text.
getByTitle() — Finds elements by their title attribute.
getByTestId() — Uses a custom attribute (by default data-testid) for stable identification.

1. getByRole → getByRole is a built-in Playwright locator that identifies elements using their role (textbox, button, checkbox, radio, or link) and their accessible name (the label shown to users). It's one of the most stable and recommended locator strategies, especially for accessibility-friendly applications.

Ex: await page.getByRole('button', { name: 'Sign in' }).click();

Example Program – Using getByRole

```
import { test, expect } from '@playwright/test'
```

```
test('Get By Role Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site/')
// Text Box
await page.getByRole('textbox', { name: 'First Name *' }).fill('Vinoth')
// Radio button
await page.getByRole('radio', { name: 'Female' }).check();
// Checkbox
await page.getByRole('checkbox', { name: 'Selenium WebDriver' }).check()
await page.getByRole('checkbox', { name: 'DevOps' }).uncheck()
// Button
await page.getByRole('button', { name: 'Submit' }).click()
// Link
await page.getByRole('link', { name: 'Tutorials' }).click()
})
```

2. getByText → Locate elements using their visible text content. It is useful when you want to validate or interact with labels, messages, headings, or links. It supports exact matches and partial matches.

Ex: await expect(page.getByText('Welcome, John!')).toBeVisible();

Example Program – Using getByText

```
import { test, expect } from '@playwright/test'
test('Get By Text Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-page-healthcare/')
// Heading validation
const formTitle = page.getByText('Patient Details Form')
await expect(formTitle).toHaveText('Patient Details Form')
console.log(await formTitle.textContent())
// Link
await page.getByText('Payment Link').click()
// Partial Link
```

```
await page.getByText('send for MRI').click()
})
```

3. getByLabel →getByLabel locates a form control (like textbox, dropdown, radio, or checkbox) using the text of its associated <label> tag. If a field has a visible label linked with it (via the for attribute or implicit nesting), Playwright can interact with it directly. This makes tests more readable and less dependent on technical attributes like IDs or classes.
Ex: await page.getByLabel('User Name').fill('John');

Example Program – Using getByLabel

```
import { test, expect } from '@playwright/test'
test('Get By Label Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site/');
// Enter your query
await page.getByLabel('Enter your query ').fill('Is playwright easy to learn?');
});
```

4. getByPlaceholder → getByPlaceholder locates an input field based on the value of its placeholder attribute. Useful when input fields don't have labels or stable IDs. Keeps your tests clean and user-focused, because placeholder text is what the end user sees.
Ex: await page.getByPlaceholder('name@example.com')

Example Program – Using getByPlaceholder

```
import { test, expect } from '@playwright/test'
test('Get By Place Holder Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site-keyboard-events/');
// Search Box
await page.getByPlaceholder('Type to search').fill('JAVA');
});
```

5. getByAltText → getByAltText locates elements (usually images or icons) by their alternative text (alt attribute). It's extremely useful for validating images, logos, and accessibility compliance. Helps ensure your tests interact with the intended visual elements.
Ex: await page.getByAltText('playwright logo').click();

Example Program – Using getByAltText

```
import { test, expect } from '@playwright/test'
test('Get By Alt Text Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/');
// Logo Image
const logoImage = page.getByAltText('Vinoth Tech Solutions');
await expect(logoImage).toHaveAttribute('alt', 'Vinoth Tech Solutions');
});
```

6. getByTitle → getByTitle locates elements by their title attribute (tooltip-style hints). This attribute is often displayed as a tooltip when you hover over an element. It is useful for inputs, buttons, and icons where no other accessible labels exist.

Ex: await expect(page.getByTitle('Issues count')).toHaveText('25 issues');

Example Program – Using getByTitle

```
import { test, expect } from '@playwright/test'
test('Get By Title Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/mouse-event/');
// Enter First Name text field located by its title attribute
const firstName = page.getByTitle('Enter First Name');
await expect(firstName).toHaveAttribute('title', 'Enter First Name');
});
```

7. getByTestId → getByTestId locates elements by their data-testid attribute (or any custom test ID attribute configured). These attributes are added by developers for the sole purpose of making automation easier. Unlike text or labels, data-testid values are less likely to change when UI design changes.

Ex: await page.getByTestId('directions').click();

Example Program – Using getByTestId

```
import { test, expect } from '@playwright/test'
test('Get By Test ID Locator', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site-create-account/')
// Form - data-testid
```

```
await page.getByTestId('input-firstName').fill('Vinoth')
await page.getByTestId('input-lastName').fill('R')
await page.getByTestId('input-address').fill('XYZ')
await page.getByTestId('input-city').fill('Chennai')
await page.getByTestId('input-state').fill('Tamil Nadu')
await page.getByTestId('input-zip').fill('600000')
await page.getByTestId('input-phone').fill('6383544892')
await page.getByTestId('input-email').fill('vinothtechsolutions@gmail.com')
await page.getByTestId('btn-register').click()
})
```

II. Traditional Locators [CSS Selector & XPath]:

Playwright also supports traditional locator strategies, but they are less recommended compared to built-in locators. Apart from Playwright's built-in locators (getByRole, getByText, etc.), Playwright also supports two traditional locator strategies CSS and XPath, but they are less preferred.

1. CSS Selector:

A CSS Selector is a query language used in web development to style elements and in automation to locate elements. It selects elements using IDs, classes, attributes, or hierarchy in the DOM. Playwright defaults to CSS when no prefix is provided.

Example – CSS Selector in Playwright

```
import { test, expect } from '@playwright/test'
test('Locator with CSS Selector', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site/');
// First name (using CSS ID selector)
await page.locator('css=#vfb-5').fill('Vinoth');
// Last name (short form without "css=")
await page.locator('#vfb-7').fill('R');
});
```

2. XPath Selector:

XPath (XML Path Language) is a query language that navigates the DOM like a map.
It can locate elements using attributes, text content, and hierarchical relationships.
Unlike CSS, XPath can move both forward and backward in the DOM tree.

Example – XPath in Playwright

```
import { test, expect } from '@playwright/test'
test('Locator with XPath Selector', async ({ page }) => {
// Launch application
await page.goto('https://vinothqaacademy.com/demo-site/');
// Email Id (explicit XPath)
await page.locator("xpath=//input[@id='vfb-14']").fill('vinothrwins@gmail.com');
// Mobile Number (short form without "xpath=")
await page.locator("//input[@id='vfb-19']").fill('123456789');
});
```

****

Screenshots in Playwright

The main objective of automation testing is to identify failures quickly without re-running the entire test manually. When a test fails, it is helpful to capture a visual representation of the application state at that moment. Screenshots help testers analyse bugs and understand what went wrong during test execution. Screenshots are an important feature in automation testing. They help testers capture the visual state of the application during test execution. When a test fails or behaves unexpectedly, screenshots provide a clear view of what happened on the screen at that moment. These screenshots help in debugging failures, verifying UI changes, and maintaining test evidence.

Screenshots are very useful for:

- Debugging test failures
- Verifying application flow
- Capturing UI state
- Maintaining test evidence

Playwright provides a method called screenshot() which allows capturing screenshots of the entire page, visible page, or specific elements.

Capturing Screenshots in Playwright

Playwright allows capturing screenshots by using the page.screenshot() method. This method captures the screenshot and saves it in the specified location.

Syntax: await page.screenshot({ path: 'path/imageName.png' });

Example: await page.screenshot({ path: 'Screenshots/screen01.png' });

In Playwright, screenshots can be captured easily using built-in methods. Testers can capture:

- Full page screenshots
- Visible page screenshots
- Screenshots of specific elements
- Screenshots automatically on test failure
- Screenshots with dynamic names (timestamps)

1. Capturing Full Page Screenshot:

Playwright provides the full-page option to capture the entire page. A full-page screenshot captures the entire web page including the parts that are not visible on the screen without scrolling. This is useful when testing large **pages** such as:

Dashboards, product listing pages, reports, tables etc….

Syntax: await page.screenshot({ path: 'Screenshots/fullPage.png', fullPage: true });

Example:

```
import { test } from '@playwright/test';
test('Capture Full Page Screenshot', async ({ page }) => {
 await page.goto('https://demoqa.com/webtables');
 await page.screenshot({
 path: 'Screenshots/fullPage.png',
   fullPage: true
 });  });
```

2. Visible Page Screenshot:

In automation testing, it is often necessary to capture a screenshot of the currently visible portion of the web page. This type of screenshot captures only what is displayed in the browser viewport (the visible area of the page) and does not include the content that requires scrolling. A Visible Page Screenshot is useful when testers want to verify the UI layout or capture the state of the application exactly as it appears on the screen.

Playwright provides the page.screenshot() method to capture visible page screenshots. By default, Playwright captures the visible portion of the page if the fullPage option is not specified.

Syntax: await page.screenshot({ path: 'Screenshots/page.png' });

Example:

```
import { test } from '@playwright/test';
test('Capture Full Page Screenshot', async ({ page }) => {
 await page.goto('https://demoqa.com/webtables');
 await page.screenshot({
 path: 'Screenshots/fullPage.png',
   fullPage: true
 });  });
```

Difference Between Visible Screenshot and Full-Page Screenshot

Screenshot Type	Description
Visible Page Screenshot	Captures only the currently visible browser viewport
Full Page Screenshot	Captures the entire webpage including hidden sections that require scrolling

3. Capturing Screenshot of a Specific Element:

Sometimes testers only need to capture a screenshot of a specific element instead of the entire page.

Examples include capturing: tables, forms, buttons, charts, specific UI components.

Playwright allows capturing element screenshots using the **locator.screenshot()** method.

Example:

```
import { test } from '@playwright/test';
test('Capture Element Screenshot', async ({ page }) => {
 await page.goto('https://demoqa.com/webtables');
 const table = page.locator(".rt-table");
 await table.screenshot({
  path: "Screenshots/tableScreenshot.png"
 });
});
```

4. Screenshot with Timestamp (Dynamic Name):

When multiple screenshots are captured during test execution, it is recommended to use dynamic file names. One common approach is to include a timestamp in the file name. This prevents screenshots from being overwritten.

Example:

```
import { test } from '@playwright/test';
```

```
test('Screenshot with timestamp', async ({ page }) => {
await page.goto('https://demoqa.com/webtables');
//const timestamp = Date.now();
const timestamp = new Date()
   .toLocaleString()
   .replace(/[/:, ]/g, '-');
await page.screenshot({
path: `screenshots/page_${timestamp}.png`,fullPage: true});
});
});
```

5. Capture Screenshot on Test Failure:

In real automation frameworks, screenshots are usually captured only when a test fails. This helps in quickly identifying the reason for failure. Playwright allows automatic screenshots using the configuration file.

playwright.config.ts

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
use: {
screenshot: 'only-on-failure'
 } });
```

****

Video Recording in Playwright

In automation testing, video recording helps testers capture the complete execution of a test case. This allows testers to visually analyse the behavior of the application during test execution. Videos are particularly useful when tests fail because they show exactly what happened step-by-step.

Playwright provides a built-in feature to record videos of test execution automatically. The recorded videos are saved in a specified directory and can be used for debugging, reporting, and documentation purposes.

Advantages of Video Recording:

- Helps analyse test failures easily
- Shows complete test execution flow
- Useful for debugging UI issues
- Provides visual evidence for test reports
- Helpful in CI/CD pipelines

Enabling Video Recording in Playwright

Video recording is enabled through the Playwright configuration file.

playwright.config.ts

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  use: {
    video: 'on'
  } });
```

This configuration records video for every test execution.

Video Recording Options

Option	Description
off:	Video recording disabled
on:	Video recorded for every test
retain-on-failure:	Video saved only when test fails
on-first-retry:	Video recorded only during first retry

Specifying Video Storage Location

Videos can be stored in a specific directory using recordVideo option.

Example:

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  use: {
    video: 'on',
    recordVideo: {
      dir: 'videos/'
    }    }  });
```

All recorded videos will be saved inside the videos folder.

Playwright Video Recording Example – Swag Labs Login

During the execution of this test, Playwright records the entire browser activity.

```
import { test } from '@playwright/test';
test('Swag Labs Login Video Recording Example', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');  // Open Swag Labs website
  await page.waitForTimeout(2000);
  await page.fill('#user-name', 'standard_user'); // Enter Username
  await page.waitForTimeout(2000);
 await page.fill('#password', 'secret_sauce');     // Enter Password
```

```
 await page.waitForTimeout(2000);
await page.click('#login-button');   // Click Login Button
});
```

Video File Output: After execution, videos are saved in the specified folder.
Example folder structure:

```
test-results/
   └── test-name/
       └── video.webm
```

The recorded file format is .webm.

****

Frames in Playwright

Frames and iFrames are important concepts in web automation testing. Many modern websites embed external content such as advertisements, videos, maps, payment pages, or widgets using frames. When automation tools interact with such pages, they must handle frames properly to access the elements inside them.

Frame: A frame is an HTML structure that divides the browser window into multiple sections. Each section loads a separate HTML document. Frames were commonly used in older web applications to split the screen horizontally or vertically. Frames were defined using the <frameset> and <frame> tags.

Example:

```
<frameset cols="50%,50%">
  <frame src="page1.html">
  <frame src="page2.html">
</frameset>
```

Each frame loads its own web page and behaves independently from other frames.

iFrame: An iFrame (Inline Frame) is an HTML element used to embed another webpage inside the current webpage. Today, most websites use iFrames instead of frames.

Example:

<iframe id="frame1" src="framePage.html"></iframe>

The iframe content belongs to a separate DOM structure, which means automation tools cannot directly interact with elements inside it unless they access the frame context.

iFrames are widely used in modern web applications for embedding external content.

- Advertisement banners
- YouTube video players
- Google Maps
- Payment gateway pages
- Chat support widgets

Why Frames Must Be Handled in Automation

Elements inside frames belong to a different HTML document. Because of this, automation tools cannot locate those elements directly from the main page. Before interacting with elements inside a frame, the automation script must access the frame context. If a script tries to interact with an element inside a frame without switching context, it will result in an element not found error.

Handling Frames in Playwright:

1. **Using frameLocator():** The frameLocator() method allows testers to interact directly with elements inside an iframe.

Syntax: page.frameLocator("iframe_selector").locator("element_selector")

Example Script:

```
import { test } from '@playwright/test';
test('Single Frame Example', async ({ page }) => {
 await page.goto("https://demo.automationtesting.in/Frames.html");
 await page.frameLocator("#singleframe")
        .locator("input[type='text']")
        .fill("Playwright Frame Handling");
});
```

2. **Using page.frame():** This method retrieves a frame using name or URL.

Syntax: const frame = page.frame({ name: "frameName" });

Example Script:

```
import { test } from '@playwright/test';
test('Frame using name', async ({ page }) => {
```

```
await page.goto("https://demo.automationtesting.in/Frames.html");
const frame = page.frame({ name: "singleframe" });
await frame?.locator("input[type='text']").fill("Automation Testing");
});
```

3. Using contentFrame(): Playwright can access frames by first locating the iframe element and then retrieving its frame object.

Example:

```
import { test } from '@playwright/test';
test('Frame using element', async ({ page }) => {
await page.goto("https://demo.automationtesting.in/Frames.html");
const frameElement = page.locator("#singleframe");
const frame = await frameElement.contentFrame();
await frame?.locator("input[type='text']").fill("Handling Frames");
});
```

Handling Nested Frames: Nested frames occur when an iframe is present inside another iframe.

In the practice website: https://demo.automationtesting.in/Frames.html

Click Iframe with in an Iframe

Switch to outer iframe

Switch to inner iframe

Enter Text into TextBox

Example:

```
import { test } from '@playwright/test';
test('Handle Nested Frames', async ({ page }) => {
// Step 1: Open the Frames website
await page.goto("https://demo.automationtesting.in/Frames.html");
// Step 2: Click on "Iframe with in an Iframe"
await page.click("text=Iframe with in an Iframe");
// Step 3: Switch to Outer Frame
const outerFrame = page.frameLocator("iframe[src='MultipleFrames.html']");
// Step 4: Switch to Inner Frame
```

```
const innerFrame = outerFrame.frameLocator("iframe");
// Step 5: Enter text into the textbox
await innerFrame.locator("input[type='text']").fill("Playwright Nested Frames");
});
```

Getting All Frames on a Page: Playwright can return all frames available on the page.

Example:

```
import { test } from '@playwright/test';
test('List all frames', async ({ page }) => {
 await page.goto("https://demo.automationtesting.in/Frames.html");
 const frames = page.frames();
 console.log("Total Frames:", frames.length);
});
```

Switching Back to Main Page:

Unlike Selenium, Playwright automatically manages frame context.

After interacting with a frame, you can simply interact with the page object again.

Example:

```
await page.locator("body").click();
```

Note: No explicit method like defaultContent() is required.

****

Browser Navigations in Playwright

During automation testing, there are situations where testers need to move between web pages that were previously visited in the browser. For example, a user may navigate to a registration page and then go back to the home page or move forward again.

Playwright provides built-in navigation methods that allow testers to control browser history. These methods simulate the same actions as moving forward, going back, refreshing, or opening a new page within the browser.

Browser Navigation Commands in Playwright

Navigate To Command

Back Command

Forward Command

Refresh Command

These commands help testers simulate real user navigation behavior.

1. **Navigate To Command:**

The goto() method is used to open a website or navigate to a specific URL. The url() method in Playwright can be used to get the URL of the current page. It returns the current page's URL in String format.

Syntax: await page.goto("URL");

This method loads a new web page in the current browser tab.

Example: await page.goto("https://demoqa.com/");

This command opens the specified webpage in the browser.

2. Forward Command:

The goForward() method in Playwright performs the browser's forward navigation. If forward navigation isn't possible, it returns null. This method is used to move to the next page in the browser history. The goForward() method performs the same action as clicking the Forward button in the browser.

Syntax: await page.goForward();

This command moves the browser forward to the next visited page.

3. Back Command:

The goBack() method in Playwright performs the browser's back navigation action. The goBack() method performs the same function as clicking the Back button in the browser.
It moves the browser to the previous page in the browser history.

Syntax: await page.goBack();

This command moves the browser back to the previous page.

4. Refresh Command:

The reload() method in Playwright is used for refreshing the web page. This method performs the same action as the user would refresh the current web page. It performs the same function as pressing F5 in the browser.

Syntax: await page.reload();

This command reloads the current webpage and refreshes all elements.

Example Script for Playwright Navigation Commands **Test Scenario:** ✓ Launch browser ✓ Open DemoQA website ✓ Click on Registration link ✓ Navigate back to Home page ✓ Navigate forward to Registration page ✓ Navigate back again using goto()	import { test } from '@playwright/test'; test('Browser Navigation Example', async ({ page }) => { // Open DemoQA Website await page.goto("https://demoqa.com"); // Click Registration link await page.click("text=Forms"); // Navigate Back await page.goBack(); // Navigate Forward

✓ Refresh the browser ✓ Close the browser	await page.goForward(); // Navigate to Home Page await page.goto("https://demoqa.com"); // Refresh the Page await page.reload(); });

****

Alerts in Playwright

Handling alerts is an important part of automation testing. Alerts are small popup message boxes that appear on the screen to provide information, warnings, or ask for confirmation before performing an action. Alerts block the browser until the user performs an action such as clicking OK or Cancel, so the automation script must handle them properly.

In Playwright, alerts are handled using the dialog event, which allows the automation script to detect and interact with the alert.

Alert: An Alert is a small message/notification box that appears on the screen to provide information, warnings, or request permission to perform a certain action. Sometimes the user may also enter text into the alert box.

Example: Imagine filling out an online form and leaving some fields empty. The website may display an alert message such as:

Please fill out this field

This alert helps notify the user that some information is missing.

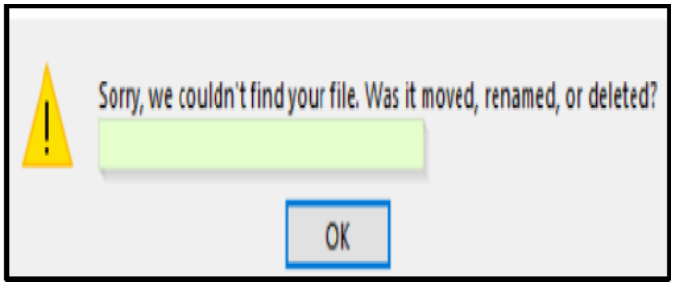
Example JavaScript alert: alert("Login Successful");

Types of Alerts: There are 3 types of alerts commonly used in web applications.

Simple Alert **Confirmation Alert** **Prompt Alert**

1. Simple Alert:

These alerts are just informational alerts. The simple alert in **Playwright** shows some information or warning on the window and have an OK button on them. Users can click on the OK button after reading the message displayed on the alert box. This alert is used to notify a simple warning message with an 'OK' button, as shown in the below snapshot.



Example: alert("This is a simple alert");

Playwright Script:

```
import { test } from '@playwright/test';
test('Simple Alert Example', async ({ page }) => {
await page.goto("https://demo.automationtesting.in/Alerts.html");
page.on('dialog', async dialog => {
console.log(dialog.message());
await dialog.accept();
});
await page.click("button[onclick='alertbox()']");
});
```

The dialog.accept() method is used to click the OK button.

2. Confirmation Alert:

The confirmation alert asks for the permission to do some type of operations. This alert is basically used for the confirmation of some tasks. These alerts get some confirmation from the user in the form of accepting or dismissing the message box. They are different from prompt alerts in a way that the user cannot enter anything as there is no text-box available. Users can only read the message and provide the inputs by pressing the OK/Cancel button.

For Example: Do you wish to continue a particular task? Yes or No? The snapshot below depicts the same.

Example: confirm("Do you want to continue?");

Users can confirm the action by clicking OK or cancel it by clicking Cancel.

Playwright Script:

```
import { test } from '@playwright/test';
test('Confirmation Alert Example', async ({ page }) => {
await page.goto("https://demo.automationtesting.in/Alerts.html");
await expect(page).toHaveTitle(/Alerts/);
page.on('dialog', async dialog => {
   console.log('Alert message:', dialog.message());
   await dialog.accept('Krishna');

 await page.locator('//a[@href="#Textbox"]').click();
 await page.waitForTimeout(2000);
  await page.locator('//button[@onclick="promptbox()"]').click();
   await page.waitForTimeout(2000);
   const result = await page.locator('#demo1').textContent();

  console.log('After alert:', result);
   await page.pause();
});
```

3. Prompt Alert:

A Prompt Alert allows the user to enter text inside the alert box. It contains a textbox where the user can type information before clicking OK or Cancel. In Prompt alerts, some input requirement is there from the user in the form of text needs to enter in the alert box. This alert will ask the user to input the required information to complete the task. A prompt alert box is displayed like below, where the user can enter his/her username and press the OK button or Cancel the alert box without entering any details.

Example: The dialog.accept("text") method enters text into the alert box and clicks OK.

```
prompt("Enter your name");
Playwright Example
import { test } from '@playwright/test';
test('Prompt Alert Example', async ({ page }) => {
await page.goto("https://demo.automationtesting.in/Alerts.html");
await page.click("text=Alert with Textbox");
page.on('dialog', async dialog => {
console.log(dialog.message());
await dialog.accept("Krishna");
});
await page.click("button[onclick='promptbox()']");
});
```

Methods Used for Handling Alerts in Playwright:

Method	Description
dialog.accept():	Accepts the alert by clicking OK
dialog.dismiss():	Dismisses the alert by clicking Cancel
dialog.accept("text"):	Sends text to a prompt alert
dialog.message():	Retrieves the message from the alert

****

File Upload & Download in Playwright

While Playwright testing you may have come across a requirement where you need to either download or upload file in Playwright. Almost every web-application over the internet may have a feature for allowing users to either download or upload a file.

Uploading Files in Playwright:

File upload is a common feature in many web applications where users upload files such as documents, images, resumes, or reports. Automation testers must verify whether the application correctly accepts and processes uploaded files.

In automation testing, handling file uploads ensures that the application supports different file formats and validates file size or type correctly.

Playwright provides a built-in method called “setInputFiles()” to upload files easily.

File upload refers to the process of sending a file from a user's local computer to a web application.

Example real-time scenarios:

- Uploading a profile picture
- Uploading a resume in job portals
- Uploading documents in banking applications
- Uploading images in social media platforms

Upload a Single File:

First, we need to locate the element. Next, we will use the “setInputFiles” method to provide the path of the file we want to upload.

```
import { test } from '@playwright/test';
test('File Upload Example', async ({ page }) => {
// Open File Upload page
await page.goto("https://demo.automationtesting.in/FileUpload.html");
// Upload file
await page.locator("input[type='file']")
.setInputFiles("C:/Users/Automation/Documents/sample.txt");
});
```

Upload Multiple Files:

Playwright allows uploading multiple files at the same time. The “setInputFiles” method also accepts a Path[] array, allowing us to pass the paths of multiple files,

```
import { test } from '@playwright/test';
test('File Upload Example', async ({ page }) => {
// Open File Upload page
await page.goto("https://demo.automationtesting.in/FileUpload.html");
// Upload file
await page.locator("input[type='file']")
.setInputFiles([
"C:/files/file1.txt",
"C:/files/file2.txt"
]);  });
```

Removing Uploaded Files [Clearing Uploaded Files]:

Sometimes you need to remove an already attached file before uploading a new one. So, to remove already attached files, pass an empty array.

```
import { test } from '@playwright/test';
test('File Upload Example', async ({ page }) => {
// Open File Upload page
await page.goto("https://demo.automationtesting.in/FileUpload.html");
// Upload file
await page.locator("input[type='file']")
.setInputFiles([
"C:/files/file1.txt",
"C:/files/file2.txt"  ]);
await page.locator("input[type='file']").setInputFiles([]);
});
```

File Download in Playwright:

File download is a common functionality in web applications where users download files such as reports, invoices, text files, or PDF documents from a website. When you click a download link or button, the browser typically sends an HTTP request to fetch the file, which then prompts the user to save it locally.

In automation, you cannot rely on the OS “Save As” dialog because it is outside the browser’s DOM. Playwright solves this by listening for the download event triggered by the browser and capturing it programmatically. Automation testers must verify that files are generated correctly and downloaded successfully to the system.

Playwright provides the **download event** to handle file downloads. When a file starts downloading, Playwright emits a **download event**, which can be captured using:
page.waitForEvent('download')
This returns a **Download object** that represents the downloaded file.

```
import { test } from '@playwright/test';
test('File Download Example', async ({ page }) => {
await page.goto('https://demo.automationtesting.in/FileDownload.html');
// Enter text
await page.fill('#textbox', 'Playwright File Download Example');
// Generate file
await page.click('#createTxt');
// Wait for download
const downloadPromise = page.waitForEvent('download');
// Click Download link
await page.click('#link-to-download');
// Capture download
const download = await downloadPromise;
// Save file locally
await download.saveAs('downloads/info.txt');
});
```

Get Download File Name

```
const fileName = download.suggestedFilename();
console.log(fileName);
```

Get Download File Path

```
const path = await download.path();
console.log(path);
```

****

Scroll Bar [Scroll a Page] in PLAYWRIGHT

Scrolling is an important feature of any web application because it is not possible to display all the content in a single screen. A Scrollbar is used to move the screen in horizontal or vertical direction when the content of the page does not fit into the visible area of the screen.

Playwright does not require scroll to perform actions as it manipulates DOM. But in certain web pages, elements only become visible once the user have scrolled to them. In such cases scrolling may be necessary.

It helps to:

→ Move the page up and down

→ Move the page left and right

Playwright Behavior:

In most cases, Playwright does not require manual scrolling because:

- Playwright directly interacts with the DOM (Document Object Model).
- It automatically waits for elements
- It automatically brings elements into view before performing actions

However, scrolling is required in some situations:

- Elements are loaded only after scrolling (lazy loading)
- Infinite scrolling pages
- Elements are hidden outside the viewport

Scroll Types: There are two types of scrolling:

1) Vertical Scrolling:	**2) Horizontal Scrolling:**
Moves the page up and down	Moves the page left and right
◆ Example	◆ Example
await page.mouse.wheel(0, 500); // down	await page.mouse.wheel(500, 0); // right
await page.mouse.wheel(0, -500); // up	await page.mouse.wheel(-500, 0); // left

Scrolling Methods in Playwright:

Playwright provides different methods to perform scrolling based on the requirement.

1. **Auto Scrolling (Default Behavior):** Playwright automatically scrolls elements into view before performing actions like:

click() fill() type()

So manual scrolling is not required in most cases.

Example: Click product without manual scroll

```
import { test } from '@playwright/test';
test('Auto scrolling behavior', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
// Login
await page.fill('#user-name', 'standard_user');
await page.fill('#password', 'secret_sauce');
await page.click('#login-button');
```

```
// Playwright auto scrolls
await page.locator('text=Sauce Labs Backpack').click();
});
```

2. Mouse Wheel Scrolling: Playwright allows scrolling using mouse wheel action. This method simulates real user behavior by scrolling the page using mouse movement. It supports both: Vertical scrolling and Horizontal scrolling

Example: Scroll down product list

```
import { test } from '@playwright/test';
test('Mouse wheel scrolling', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
// Login
await page.fill('#user-name', 'standard_user');
await page.fill('#password', 'secret_sauce');
await page.click('#login-button');
// Scroll down
await page.mouse.wheel(0, 800);
// Scroll up
await page.mouse.wheel(0, -400);
});
```

3. Keyboard Scrolling: Scrolling can be performed using keyboard keys.
Common keys used:
→PageDown → scroll down →PageUp → scroll up
→End → scroll to bottom →Home → scroll to top

Example:

```
import { test } from '@playwright/test';
test('Keyboard scrolling', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
// Login
await page.fill('#user-name', 'standard_user');
await page.fill('#password', 'secret_sauce');
await page.click('#login-button');
await page.keyboard.press('PageDown');
await page.keyboard.press('PageUp');
await page.keyboard.press('End'); // bottom
```

```
await page.keyboard.press('Home'); // top
});
```

4. scrollIntoViewIfNeeded(): This method is used to scroll to a specific WebElement that is not visible on the screen. It ensures that the element is brought into the visible area before performing any action. It is the most reliable and recommended method in Playwright.

Example (SwagLabs): Scroll to "Sauce Labs Bike Light" product

```
import { test } from '@playwright/test';
test('Scroll to specific product', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
// Login
await page.fill('#user-name', 'standard_user');
await page.fill('#password', 'secret_sauce');
await page.click('#login-button');
// Scroll to product
const product = page.locator('text=Sauce Labs Bike Light');
await product.scrollIntoViewIfNeeded();
});
```

****

Handle Multiple Windows In PLAYWRIGHT

A window in any browser is the main webpage on which the execution is performed after navigating to a URL. The main window is also known as the parent window. All other windows that open within the main window are called child windows. In automation,

handling multiple windows is important because modern web applications frequently open new tabs or windows based on user actions such as clicking links, buttons, or pop-ups.
When execution starts in Playwright, only one window is available by default. This window is represented by a Page object and is considered the main window. All operations are initially performed on this Page. When additional windows or tabs are opened during execution, Playwright creates new Page objects automatically. A Page object acts as a unique reference to a browser window. Instead of using string-based identifiers, each window is handled using its corresponding Page object.
To master multiple windows in Playwright, you must understand the hierarchical architecture that governs how it manages browser sessions and tabs.

A. context.pages(): This method is used to get all the pages that are currently opened within a BrowserContext. When multiple windows or tabs are opened, each of them is stored as a Page object, and this method returns all such objects in the form of an array. This array contains references to all open windows, including the main window and child windows. Using this method, it is possible to identify and access any window during execution.

```
const pages = context.pages();
// Example usage
console.log(pages.length);
pages.forEach((p, index) => {
  console.log(`Page ${index}: ` + p.url());
});
```

B. context.waitForEvent('page'): This method is used to capture a newly opened window or tab. When an action such as clicking a link opens a new window, Playwright generates a page event. This method waits for that event and returns the newly created Page object. It ensures proper synchronization and helps in handling new windows reliably.

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('a[target="_blank"]')
]);
await newPage.waitForLoadState();
console.log(newPage.url());
```

C. context.newPage(): This method is used to create a new window or tab manually within the same BrowserContext. It returns a new Page object which can be used to navigate to any URL and perform actions independently.

```
const newTab = await context.newPage();
```

```
await newTab.goto('https://example.com');
```

D. Page Object: In Playwright, each window is represented by a Page object. The Page object provides methods to perform actions such as clicking elements, entering data, retrieving information, and executing scripts. Each Page object operates independently, allowing multiple windows to be controlled within the same execution.

```
import { test } from '@playwright/test';
test('Open main page', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
await newPage.fill('#username', 'admin');
await newPage.click('#login');
console.log(await page.title());
});
```

E. Page Identification: When multiple windows are present, each Page object can be identified using its position in the array returned by context.pages(). This allows distinguishing between parent and child windows.

```
const pages = context.pages();
const parentPage = pages[0];
const childPage = pages[1];
```

F. page.bringToFront(): This method is used to bring a particular window into focus. When multiple windows are open, this method makes the selected Page active on the screen. It is mainly used for visual purposes such as debugging or validation.

```
await childPage.bringToFront();
await parentPage.bringToFront();\
```

G. page.waitForLoadState(): This method is used to wait until the page is fully loaded before performing further actions. It is commonly used after opening a new window to ensure that the page is ready for interaction.

```
await newPage.waitForLoadState();
```

H. URL and Title Retrieval: The Page object provides methods to retrieve information such as URL and title of the window. These methods are used for validation and verification during execution.

```
console.log(newPage.url());
console.log(await newPage.title());
```

I. Handling Multiple Windows: When more than two windows are open, all Page objects can be accessed and iterated using loops. This helps in performing operations across multiple windows.

```
const pages = context.pages();
for (const p of pages) {
  console.log(p.url());
}
```

J. Page Events: Playwright provides event handling mechanisms to detect new windows in real time. When a new window is opened, an event is triggered, and the Page object can be captured and used for further operations.

```
context.on('page', async (p) => {
  console.log('New page opened:', p.url());
});
```

K. page.close(): This method is used to close the current window. When multiple windows are open, this method closes only the Page on which it is applied, while the remaining windows continue to exist.

```
await childPage.close();
```

browser.close(): This method is used to close the entire browser instance. It terminates the browser process and closes all associated browser contexts and their pages (tabs) simultaneously. This is typically used at the very end of a script to ensure no background processes are left running.

```
await browser.close();
```

context.close(): This method closes a BrowserContext, which acts like an incognito session. It closes all pages (tabs) belonging to that specific session and clears all temporary data like cookies and cache, but it keeps the main browser process alive so you can start a new session quickly.

```
await context.close();
```

****

Web Tables In PLAYWRIGHT

Web tables, also known as HTML tables, are a widely used format for displaying data on web pages. They provide a structured representation of information in rows and columns, making it easy to read and manipulate data. Playwright allows interaction with these tables

by locating elements using locators such as CSS selectors or XPath, enabling operations like extracting data, clicking elements inside the table, and validating content.

A table consists of rows and columns. The table created for a web page is called a web table. A table is a type of HTML structure which is displayed using the <table> tag along with <tr> and <td> tags.

Below are some of the important tags associated with a web table:

<table> – Defines an HTML table

<th> – Contains header information in a table

<tr> – Defines a row in a table

<td> – Defines a column in a table

```
<table name="WebTableData">
  <tr>
    <th>Student Name</th>
    <th>Course Name</th>
    <th>Trainer Name</th>
  </tr>
  <tr>
    <td>Mounika</td>
    <td>Software Testing</td>
    <td>N.Krishna</td>
  </tr>
  <tr>
    <td>NagaRaju</td>
    <td>FullStack Python</td>
    <td>Srinivas Rao</td>
  </tr>
    <tr>
    <td>Arun</td>
    <td>FullStack Java</td>
    <td>N.Krishna</td>
  </tr>
</table>
```

Student Name	Course Name	Trainer Name
Mounika	Software Testing	N.Krishna
NagaRaju	FullStack Python	Srinivas Rao
Arun	FullStack Java	N.Krishna

In Playwright, web tables are classified into two main types based on their behavior.

Static Web-Table: A static web table has a fixed structure and content that does not change after the page loads. The data remains constant, and the number of rows and columns does not vary. Handling such tables is straightforward because elements can be located directly, and data can be extracted without considering dynamic changes.

Dynamic Web-Table: A dynamic web table changes based on user interactions or data updates such as searching, sorting, filtering, or pagination. The number of rows, columns, or the data itself may change during execution. Handling dynamic tables requires synchronization and proper locator strategies to ensure correct data extraction and interaction.

Handling Web-Tables in Playwright involves a sequence of steps to identify and interact with table elements.

Step#1: Locate the table element.

The table is first identified using locators such as CSS selectors or XPath. This acts as the parent element for all rows and columns.

const table = page.locator('table');

Step#2: Get the rows and columns of the table

Rows are identified using <tr> and columns using <td>. Header columns can also be identified using <th> for validation of table structure.

```
const rows = table.locator('tr');
const cells = table.locator('td');
const headers = table.locator('th');
for (let i = 0; i < await headers.count(); i++) {
  console.log(await headers.nth(i).innerText());
}
```

Step#3: Perform actions on the elements within the table

After locating rows and columns, operations such as reading data, validating values, or performing actions can be executed.

```
const cell = table.locator('tr').nth(1).locator('td').nth(1);
console.log(await cell.innerText());
```

Step#4: Printing Content of a Web Table in Playwright

The data present in a web table can be printed by iterating through rows and columns. Empty rows can be ignored to ensure valid data processing.

```
const rows = table.locator('tr');
for (let i = 0; i < await rows.count(); i++) {
const rowText = await rows.nth(i).innerText();
if (rowText.trim() !== '') {
const cells = rows.nth(i).locator('td');
for (let j = 0; j < await cells.count(); j++) {
process.stdout.write((await cells.nth(j).innerText()) + "  ");
}
console.log();
}  }
```

A specific row can be identified based on text and validated. This approach is commonly used in dynamic tables.

```
for (let i = 0; i < await rows.count(); i++) {
const text = await rows.nth(i).innerText();
if (text.includes('Cierra')) {
console.log('Row Found:', text);
}    }
```

Once a row is identified, actions such as edit or delete can be performed on elements within that row.

```
await page.locator('text=Cierra')
.locator('..')
.locator('span[title="Edit"]')
.click();
Column-wise data can also be extracted when validation of a specific
column is required.
const column = page.locator('table tr td:nth-child(2)');
for (let i = 0; i < await column.count(); i++) {
console.log(await column.nth(i).innerText());
}
```

Dynamic tables often include search or filtering functionality. Data can be filtered by entering values into the search field.

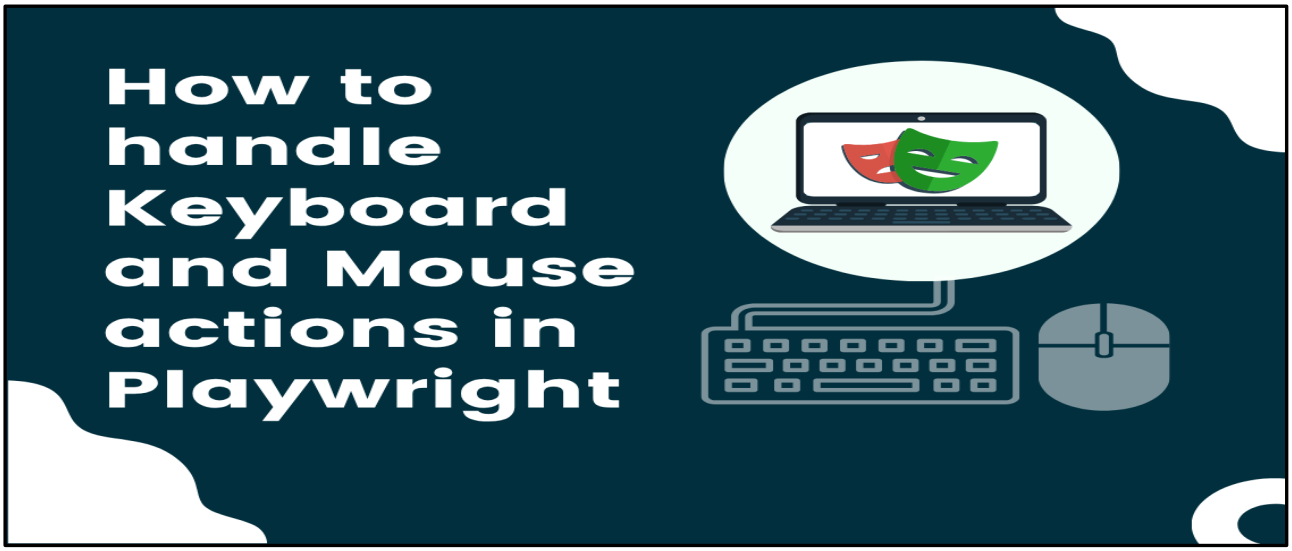
await page.fill('#searchBox', 'Cierra');

****

Actions In PLAYWRIGHT

Playwright can interact with HTML Input elements such as text inputs, checkboxes, radio buttons, select options, mouse clicks, type characters, keys and shortcuts as well as upload files and focus elements.

Actions in Playwright refer to user interactions performed on web elements such as clicking, typing, hovering, dragging, and keyboard operations. These actions simulate real user behavior on a web application and are essential for automating functional workflows.
Playwright provides built-in methods to perform actions directly on elements without requiring a separate action class. These methods are available through the Page object and allow interaction with elements using locators.



1) Keyboard Actions:

Handling keyboard actions is a common requirement in UI automation. From simple text entry to complex shortcuts like Ctrl + S, Tab navigation, or Enter submission. Playwright provides powerful and flexible APIs to simulate real user keyboard interactions. Keyboard actions in Playwright with TypeScript are handled using the keyboard API.

List of **Keyboard Actions:**

Action	Locator-Based Example	Keyboard API Example
Press Enter	await page.locator('#search').press('Enter');	await page.keyboard.press('Enter');
Backspace	await page.locator('#input').press('Backspace');	await page.keyboard.press('Backspace');
Delete	await page.locator('#input').press('Delete');	await page.keyboard.press('Delete');
Type Text	await page.locator('#name').type('Krishna');	await page.keyboard.type('Krishna');

Select All (Ctrl + A)	await page.locator('#input').press('Control+A');	await page.keyboard.press('Control+A');
Copy (Ctrl + C)	await page.locator('#input').press('Control+C');	await page.keyboard.press('Control+C');
Paste (Ctrl + V)	await page.locator('#input').press('Control+V');	await page.keyboard.press('Control+V');
Cut (Ctrl + X)	await page.locator('#input').press('Control+X');	await page.keyboard.press('Control+X');
Arrow Up	await page.locator('#list').press('ArrowUp');	await page.keyboard.press('ArrowUp');
Arrow Down	await page.locator('#list').press('ArrowDown');	await page.keyboard.press('ArrowDown');
Arrow Left	await page.locator('#input').press('ArrowLeft');	await page.keyboard.press('ArrowLeft');
Arrow Right	await page.locator('#input').press('ArrowRight');	await page.keyboard.press('ArrowRight');
Hold Key (Shift)	NA	await page.keyboard.down('Shift');
Release Key (Shift)	NA	await page.keyboard.up('Shift');

Mouse Actions: Mouse actions in Playwright are used to simulate user interactions performed using a mouse. These include clicking, hovering, dragging, and scrolling. Mouse actions are essential for interacting with elements that respond to pointer events. Playwright provides both high-level element actions and low-level mouse controls. These actions help in validating UI behavior based on user interactions.

a) Basic Mouse Click or Generic click:

A basic mouse click in Playwright simulates a user pressing and releasing the left mouse button on a web element. It is the most commonly used interaction for triggering actions such as form submissions, navigation, or button clicks. Playwright's locator-based click() method is highly reliable because it automatically waits for the element to be visible, enabled, and stable before performing the action. It also scrolls the element into view if necessary, making it more robust than coordinate-based clicks.

Example: await page.locator('#loginBtn').click();

b) Double Click:

A double click in Playwright simulates two rapid consecutive left mouse clicks on the same element. This action is typically used in scenarios such as opening files, selecting text, or triggering special UI behaviors that require two quick clicks. Playwright's dblclick() method ensures correct timing and automatically waits for the element to be visible, stable, and actionable before performing the operation, making it reliable for automation.

Example: await page.locator('#file').dblclick();

c) Right Click (Context Click):

A right click, also known as a context click, simulates pressing the right mouse button on an element to open a context menu. This is useful for testing custom menus or browser-based

options that appear on right-click. In Playwright, this is achieved by passing the { button: 'right' } option in the click() method, which triggers the appropriate contextmenu event.

Example: await page.locator('#menu').click({ button: 'right' });

d) Mouse Hover:

Mouse hover simulates moving the cursor over an element without clicking it. This action is commonly used to reveal hidden elements such as dropdown menus, tooltips, or dynamic UI components that appear only when hovered. Playwright's hover() method ensures the element is ready for interaction and then triggers the necessary hover-related events.

Example: await page.locator('#profile').hover();

e) Drag and Drop:

Drag and drop is a composite mouse interaction where an element is clicked and held, moved to a target location, and then released. This action is widely used in modern web applications such as kanban boards, file uploads, and UI rearrangement features. Playwright provides a high-level dragAndDrop() method that internally handles the sequence of mouse events (mousedown, mousemove, mouseup) and ensures stability during execution.

Example: await page.dragAndDrop('#source', '#target');

f) Click and Hold (Mouse Down)

Click and hold simulates pressing and holding the mouse button without releasing it immediately. This action is useful in scenarios like drag operations, sliders, or drawing interactions. In Playwright, this is handled using the mouse.down() method, which represents the press action.

Example: await page.mouse.down();

g) Moving the Mouse:

Moving the mouse refers to changing the cursor position across the screen without performing any click. This is useful for testing dynamic UI elements, canvas-based applications, or triggering hover effects across multiple areas. Playwright's mouse.move() method allows precise control over cursor positioning and can simulate smooth movement using steps.

Example: await page.mouse.move(300, 400);

h) Clicking at Specific Coordinates:

Clicking at specific coordinates involves performing a mouse click at exact (x, y) positions on the viewport instead of targeting a DOM element. This approach is typically used in scenarios such as canvas interactions, games, or graphical interfaces where locators are not applicable. However, it is less reliable because UI changes can shift coordinates, making tests fragile.

Example: await page.mouse.click(500, 300);

i) Release Mouse (Mouse Up):

Releasing the mouse simulates lifting the mouse button after it has been pressed. This action completes operations like drag-and-drop or click-and-hold interactions. In Playwright, the mouse.up() method is used to perform the release action.

Example: await page.mouse.up();

List of **Mouse Actions:**

Action	**Locator-Based Example**	**Mouse API Example**
Basic Click (Generic Clcik)	await page.locator('#btn').click();	await page.mouse.click(100, 200);
Double Click	await page.locator('#btn').dblclick();	Await page.mouse.dblclick(100,200);
Right Click (Context Click)	await page.locator('#btn').click({ button: 'right' });	await page.mouse.click(100, 200, { button: 'right' });
Mouse Hover	await page.locator('#menu').hover();	await page.mouse.move(200, 300);
Drag and Drop (and) Drag and Drop by Offset	await page.dragAndDrop('#source', '#target');	await page.mouse.move(100,100); await page.mouse.down(); await page.mouse.up();
Move to Element	await page.locator('#el').hover();	await page.mouse.move(200, 300);
Move by Offset	NA	await page.mouse.move(400, 500);
Click and Hold (Mouse Down)	NA	await page.mouse.down();
Release (Mouse Up)	NA	await page.mouse.up();
Click at Coordinates	NA	await page.mouse.click(500, 300);

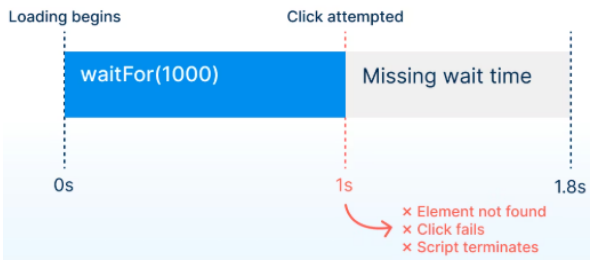
****

WAITS In PLAYWRIGHT

Waits are used to ensure that the application is ready before performing any action or validation. Since web applications are asynchronous, waits help avoid failures caused by timing issues. Playwright Wait refers to the mechanisms that pause the script execution until certain conditions in the browser are met; such as elements becoming visible or network responses being completed. These waits are integral to synchronizing tests with the dynamic nature of web applications.

In Playwright, waiting is mostly automatic (auto-waiting), which makes tests stable. In Playwright, you generally do not need to write manual "wait" statements like you do in Selenium. It uses a concept called Auto-waiting, where the tool automatically checks if an element is visible, stable, and enabled before performing an action.

Hard Wait: In automated testing theory, a "hard wait" is known as a static wait. It represents a fixed, blind pause that treats the application as a "black box," assuming it will always be ready after exactly amount of time. Hard waits and timeouts only do one thing: they instruct Playwright to wait for the specified time.

<table>
<tr><td colspan="2">Ex: await page.waitFor(5000) // hard wait for 5s
await page.click('#button-login')
await page.waitForTimeout(3000); // Waits for 3 seconds
In this type of situation, the following two scenarios can happen:</td></tr>
<tr><td>You end up waiting a shorter time than the element takes to load!</td><td>The element loads before your hard wait has expired.</td></tr>
<tr><td> 
</td><td> 
</td></tr>
<tr><td>Your hard wait terminates and your click action is attempted too early. The script terminates with an error, possibly of the “Element not found” sort.</td><td>While the element is correctly clicked once your wait expires and your script continues executing as planned, you are wasting time.</td></tr>
</table>

Hard waits and timeouts are always either too short or too long. And in the worst-case scenario, the fluctuating load times make the wait sometimes too long and sometimes too short, making our script fail randomly. You should avoid this situation at all cost because it will result in unpredictable, seemingly random failures, also known as test flakiness.

Modern frameworks like Playwright replace this with Smart Waiting, which is event-driven rather than time-driven.

1. Auto-waiting or Smart Waiting (Implicit):

Auto waiting is the default behavior in Playwright. Before any action (click, type, assertion), Playwright automatically waits for the element to be ready. Playwright has built-in auto-waiting, meaning actions like clicking or typing automatically wait for the target element to be ready. This reduces the need for manual waits and simplifies the code, making tests more efficient.

If your website has transitions or animations, Playwright wait ensures that these actions are done, before executing the tests, thereby reducing the opportunity of errors.

<table>
<tr><td>Ex: locator.click(), Playwright automatically waits for:
• The element should be displayed
• The element should be enabled
• The element should have proper stability
• The locator should identity only one element at a time.
• The element should be able to receive events.</td><td>Ex: await page.click('text=Login');
Here, Playwright automatically:
• Waits for the “Login” element to appear.
• Ensures it’s visible and interactable.
• Confirms it’s enabled.
• Clicks only when it’s ready.
All without a single manual wait statement.</td></tr>
</table>

This means you don't need manual waits if your selectors are written correctly and conditions are handled properly.

2. Explicit Waits in Playwright:

Explicit waits are manually defined wait commands used to pause test execution until a specific condition is met. These waits are useful when auto-waiting is not sufficient to handle dynamic behavior in applications. Automation testing improves efficiency and accuracy by automating repetitive tasks. In Playwright, explicit waits should be used carefully because the framework already provides built-in auto-waiting.

Below are the different types of explicit wait commands used in Playwright:

A) page.waitForTimeout(): This wait command is used to stop the execution of the test script for a fixed amount of time. It does not depend on application state. This method is not preferred because it slows down execution and may create unstable tests.

Syntax: await page.waitForTimeout(3000);
Example: await page.waitForTimeout(2000); await page.click('#login');

B) page.waitForSelector(): This command is used to wait for an element to appear or reach a specific state like visible or hidden. It ensures that execution continues only when the element is ready.

Syntax: await page.waitForSelector('#search-button'); await page.waitForSelector('#search-button', { state: 'hidden' });
Example: await page.waitForSelector('#search-button'); await page.click('#search-button');

C) page.waitForNavigation(): This command is used to wait until page navigation is completed after an action like click or redirect. It is mostly used while testing movement from one page to another webpage, redirection features or any other action that causes navigation.

Syntax: await page.waitForNavigation();
Example: await Promise.all([page.waitForNavigation(), page.click('#login')]);

D) page.waitForRequest(): This command is used to wait until a specific network request is sent. Suppose we are placing an order over an e commerce application searching for a particular thing like "Playwright" over google then some tasks will be done before the search request is sent to backend.

Syntax: await page.waitForRequest(request => request.url() === 'https://www.google.com/search?q=Playwright');
Example: await page.waitForRequest(req => req.url().includes('search')); await page.click('#search-button');

E) page.waitForResponse(): This command is used to wait until a specific response is received from the server. It is mostly used for waiting on API responses as well as for other server interactions. It can be used in two ways which are as follows:

Syntax:	await page.waitForResponse(response => response.url().includes('search?q=Playwright') && response.status() === 200 && response.request().method() === 'GET');
Example:	await page.waitForResponse(res => res.status() === 200); await page.click('#submit');

F) page.waitForLoadState(): This command waits for the page to reach a specific load state before continuing execution. This command ensures that the web page is fully loaded before any action is performed on the elements present in that page. Below is the syntax for the above wait command.

Load states:

load → full page loaded

domcontentloaded → DOM ready

networkidle → no network activity

Syntax: await page.waitForLoadState('load');
Example: await page.goto('https://example.com'); await page.waitForLoadState('load')

****

Assertions In Playwright

In the context of testing, assertions refer to statements or conditions that the QA engineers expect from Playwright test to be true during the test execution. Embedded within the test script, these statements help detect issues of the product under Playwright test and verify whether it behaves as expected.

An assertion is a pass or fail check that compares the actual state of the app or data against an expected state, such as visible text, URL, response status, or object values. Assertions are the validation points that turn an automated flow into a real test. A script can click buttons and navigate pages, but without assertions it cannot prove that the correct outcome happened.

There are mainly 2 types of Assertions:

1) Hard Assertion: When an assertion fails, the test stops execution immediately at that point. Hard assertions terminate the test execution and mark the test as failed instantly.

A hard assertion is a validation that interrupts test execution when it fails. It throws an immediate exception, preventing any further steps from executing. In Playwright, assertions are hard by default. Playwright internally evaluates the condition with auto-retry, and if it still fails within the timeout, it throws an error immediately, stopping the test execution.

- Execution stops immediately after failure
- Remaining steps are not executed
- Failure is reported instantly
- Ensures fail-fast validation approach
- Prevents execution on invalid application state

Syntax: import { expect } from '@playwright/test'; expect()
Example: await expect(page.locator('#login')).toBeVisible(); console.log("This will NOT execute if assertion fails");

2) Soft Assertion: When assertion got failed still test continue to run. Soft assertions do not terminate test execution, but mark the test as failed. A soft assertion is a validation that does not interrupt test execution when it fails. Instead of throwing an immediate exception, the failure is recorded and reported at the end of the test. Playwright internally captures assertion

errors instead of throwing them immediately. These errors are stored in the test context and evaluated after execution completes.

- Execution continues even after failure.
- Multiple assertion results are accumulated.
- Final test status becomes failed if any soft assertion fails.
- Enables comprehensive validation within a single test flow.

Syntax: expect.soft()
Example: await expect.soft(page.locator('#username')).toBeVisible(); await expect.soft(page.locator('#password')).toBeVisible(); console.log("This WILL execute even if above assertions fail");

Hard Assertions	Soft Assertions
Failure Impact	
If a hard assert fails, the test immediately stops, and the Playwright test case is marked as failed. Subsequent steps in the test script are not executed.	If a soft assert fails, the test continues to execute, and all assertions are evaluated. Finally, a summary of assertion failures is provided.
Number of Assertions	
Typically involves individual assert statements placed at different points in the test script.	Multiple assertions can be grouped together to provide evaluation of several conditions within a single test.
Execution Flow	
Execution stops at the first failed assertion and limits the information about multiple issues in a single run.	The test script continues to execute even after encountering a failed assertion and provide a more comprehensive overview of all assertion failures.
Use Cases	
Suitable for scenarios where subsequent steps are dependent on the success of previous steps, and further execution wouldn't make sense if a critical condition fails.	Useful when you want to collect information about multiple issues in a single test run without stopping the entire test case on the first failure.

Auto-Retrying Assertions (Web-First Assertions): Auto-retrying assertions are validations where Playwright automatically waits and retries until the expected condition becomes true or the timeout is reached. These are asynchronous and designed specifically for UI elements. They poll the page repeatedly until the condition is met or the timeout is reached. Auto-retrying assertions, known as Web-First Assertions, are built-in Playwright checks designed to handle the asynchronous nature of modern web apps. Instead of failing immediately if a condition isn't met, they automatically poll the page and retry the check until the expected state is reached or a timeout occurs. This mechanism eliminates the need for manual "sleep" or hard waits, significantly reducing test flakiness caused by slow network responses or UI animations. By using these, your tests wait for eventual consistency in the DOM before proceeding.

Ex1: // Retries for up to 5s (default) until the button is visible
await expect(page.getByRole('button')).toBeVisible();

Ex2: // Retries until the specific text appears in the element
await expect(page.locator('.status')).toHaveText('Success');

Playwright does not evaluate the condition just once. Internally, it: • Resolves the locator → finds the element • Checks the condition (e.g., visibility)

- If the condition fails → waits briefly
- Re-evaluates the condition

Repeats until:

- Condition becomes true
- Timeout is reached

This behavior is known as “auto-waiting with retry assertion
This is why Playwright tests are less flaky by design.

Example:
await expect(page.locator('.status')).toHaveText('Completed');
Even if:
Initially text = "Loading..."
Changes after 2 seconds
Playwright will wait and retry until “Completed” appears.

Non-Auto-Retrying Assertions (Generic Assertions)**:** Non-retrying assertions are validations that check the condition only once and do not wait or retry. These are synchronous. They check a value exactly once and fail immediately if it doesn't match. Non-retrying assertions, or Generic Assertions, perform a single check at the exact moment they are called. Unlike web-first assertions, they do not poll the page or wait for the UI to update; if the condition is not met immediately, the test fails instantly. These are typically used for static JavaScript values, API response bodies, or mathematical calculations that do not change over time.

Ex: const count = 5;
// Checks immediately; fails instantly if count is not 5
expect(count).toBe(5);

Ex: const text = await page.locator('.status').textContent();
expect(text).toBe('Completed');
Here’s what happens internally:

- Locator resolves once
- Value is extracted (textContent())
- Assertion is applied immediately
- If mismatch → test fails instantly

Internal Characteristics

- No polling loop
- No re-evaluation
- Works on captured values (snapshots)
- Faster but not resilient to delays

Problem with Dynamic UI: **If UI updates after a delay**
expect(await locator.textContent()).toBe('Success');

- Fails if text is still "Loading..." at that moment

- Even if it becomes "Success" milliseconds later

This leads to flaky tests

1) Page Assertions:

Page assertions validate the overall state of the web page. These assertions ensure that the user is on the correct page and that the page has loaded properly. They are mainly used to verify navigation, redirection, and page rendering correctness. Playwright provides built-in assertions with auto-waiting, meaning it waits until the expected condition is satisfied or timeout occurs.

Types of Page Assertions:

A) URL Validation: Validates whether the current page URL is correct.

- Verifies the current page URL
- Ensures correct navigation or redirection
- Supports full or partial URL matching
- Auto-retries until URL matches

Ex: await expect(page).toHaveURL("https://www.saucedemo.com/inventory.html");

B) Title Validation: Validates the title of the web page.

- Verifies the page title
- Confirms correct page is loaded
- Ensures correct page is loaded
- Useful for verifying page identity
- Supports exact and partial match
- Auto-retries until matched

Ex: await expect(page).toHaveTitle('Dashboard');

C) Page Load State Validation: Validates whether the page has fully loaded.

- Ensures all resources are loaded
- Prevents interaction with partially loaded pages
- Common states: load, domcontentloaded, networkidle

Ex: await page.waitForLoadState('load');

2) Checkbox & Radio Validation:

These validations are used to verify the selection state of input controls such as checkboxes and radio buttons. They ensure that the user's selection is correctly applied and reflected in the UI.

- Checkbox validation (Checked / Unchecked)
- Radio button validation (Selected option)
- Used to validate user input selection.

A) Checkbox Validation: Checkbox validation is used to verify the state of a checkbox element, ensuring whether it is selected (checked) or not selected (unchecked). In Playwright, checkbox validation is performed using element assertions, which come with auto-retrying capability. This means Playwright continuously checks the checkbox state

until it matches the expected condition or the timeout is reached. Checkboxes allow multiple selections, so validation ensures that the correct options are selected based on user interaction or application logic.

- Verifies whether a checkbox is checked or unchecked
- Ensures correct selection state
- Useful in forms and settings
- Supports multiple selections
- Uses auto-waiting and retry mechanism

Ex: await expect(page.locator('#subscribe')).toBeChecked();

B) Radio Button Validation: Radio button validation is used to verify which single option is selected among a group of radio buttons. Unlike checkboxes, radio buttons follow the rule of mutual exclusivity, meaning only one option can be selected at a time within the same group. In Playwright, radio validation is also performed using auto-retrying element assertions, ensuring the correct option is selected before proceeding.

- Verifies which radio button is selected
- Ensures only one option is selected at a time
- Used for mutually exclusive choices
- Allows only one selection at a time
- Ensures correct option is selected
- Uses auto-retry mechanism
- Validates user choice accuracy

Ex: await expect(page.locator('#male')).toBeChecked();

3) Element Assertions: Element assertions are used to validate the state and behavior of web elements. These assertions ensure that elements are ready for interaction before performing actions like click, type, etc. Playwright provides auto-retrying assertions, meaning it waits until the element reaches the expected state or timeout occurs.

- Verifies whether element is visible on UI
- Checks if element is enabled or disabled
- Confirms element is attached to DOM
- Uses auto-waiting + retry mechanism
- Prevents failures due to timing issues
- Ensures elements are ready for interaction.

Examples:

```
await expect(page.locator('#login')).toBeVisible();
await expect(page.locator('#submit')).toBeEnabled();
await expect(page.locator('#username')).toBeAttached();
```

4) Text Validation: Text validation is used to verify that an element contains the expected text content. It ensures that the UI displays correct information to the user. Playwright supports exact and partial text matching with auto-retry.

- Verifies text content inside elements
- Supports full text and partial text validation
- Uses auto-retrying assertions
- Helps validate UI messages and data display
- Used for messages, labels, notifications.

Examples:

```
await expect(page.locator('#msg')).toHaveText('Login Successful');
await expect(page.locator('#msg')).toContainText('Success');
```

5) Attribute Validation: Attribute validation is used to verify the HTML attributes of elements. It ensures that elements have correct properties and configurations.

Examples of Attributes:

• id • class • value • placeholder • Custom attributes

- Validates element attributes and their values
- Ensures correct UI behavior and data binding
- Useful for checking dynamic values
- Uses Playwright's auto-retrying assertions
- Ensures correct configuration of UI elements.

Examples:

```
await expect(page.locator('#email')).toHaveAttribute('placeholder', 'Enter email');
await expect(page.locator('#input')).toHaveAttribute('value', 'testuser');
```

****

Playwright Test Framework

Playwright Test Runner:

The Playwright Test Runner is a built-in framework used to write, organize, and execute automated test cases. It provides features like test execution, assertions, reporting, and parallel execution in a single tool.

- It provides a complete testing solution without requiring external tools or frameworks.
- It supports writing tests in a simple and structured format using JavaScript or TypeScript.
- The test runner handles test execution, validation, and reporting in a single environment.
- It includes built-in assertions using expect for validating test results.
- It automatically manages browser instances and test isolation.
- It supports parallel execution using workers to improve performance.
- It provides detailed reports such as HTML and JSON for test analysis.
- It includes lifecycle hooks for setup and cleanup operations.

Overall, it simplifies automation by making tests faster, reliable, and easy to maintain.

Example Test:

```
import { test, expect } from '@playwright/test';
test('Verify Google Title', async ({ page }) => {
  await page.goto('https://www.google.com');
  await expect(page).toHaveTitle(/Google/);
});
```

File Name: Save the file as: example.spec.ts

Playwright automatically detects .spec.ts files

How to Run Using Test Runner

▶ Run all tests: npx playwright test

▶ Run specific file: npx playwright test example.spec.ts

▶ Run in headed mode (visible browser): npx playwright test --headed

What Happens Internally

- Playwright Test Runner starts execution
- Browser launches automatically
- Test runs and validates using expect

- Result is shown in terminal
- Report is generated

Sample Output (Terminal)

Running 1 test using 1 worker

✓ Verify Google Title (2s)

1 passed (2s)

CLI Execution (npx playwright test)

The Playwright Test Runner provides a powerful Command Line Interface (CLI) to execute test cases efficiently. It allows users to run all tests, specific files, or selected test cases using simple commands. The CLI eliminates the need for complex build tools or external runners. It supports multiple options such as running tests in headed mode, debugging mode, and filtering tests by tags. The CLI also integrates seamlessly with configuration files like playwright.config.ts. It provides detailed execution output directly in the terminal, including pass/fail status. It also supports running tests in different browsers using project configurations. CLI execution is fast, flexible, and suitable for both local and CI/CD environments.

Example Test: [example.spec.ts]

```
import { test, expect } from '@playwright/test';
test('Open Example Page', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

File Name: Save the file as: example.spec.ts

Playwright automatically detects .spec.ts files

How to Run Using Test Runner

▶ Run all tests: npx playwright test

▶ Run specific file: npx playwright test example.spec.ts

▶ Run in headed mode (visible browser): npx playwright test –headed

▶ Run in specific browser: npx playwright test --project=chromium

How It Works:

- CLI command triggers Playwright Test Runner
- It discovers test files automatically
- Executes tests based on configuration
- Displays results in terminal

Sample Output:

Running 1 test using 1 worker

✓ Open Example Page (2s)

1 passed (2s)

Test Discovery (*.spec.ts):

The Playwright Test Runner automatically discovers test files based on predefined naming conventions such as .spec.ts and .test.ts. This eliminates the need for manual test suite configuration or XML files. The test runner scans the project directory and identifies all matching files during execution. It ensures that all valid test cases are included without explicitly listing them. Test discovery works seamlessly with parallel execution and tagging features. Overall, it improves efficiency, scalability, and maintainability of the test framework.

Example Test: [login.spec.ts]

```
import { test, expect } from '@playwright/test';
test('Login Test', async ({ page }) => {
  await page.goto('https://example.com/login');
  await expect(page).toHaveURL(/login/);
});
```

Project Structure Example

```
tests/
  ├── login.spec.ts
  ├── dashboard.spec.ts
  └── checkout.spec.ts
```

All .spec.ts files will be automatically picked

How It Works:

Playwright scans folders (default: tests/)

Identifies files with .spec.ts / .test.ts

Executes all detected test cases

How to Run:

➔ npx playwright test

No need to mention file names manually

Parallel Execution:

The Playwright Test Runner supports parallel execution using workers, which are separate processes that run tests simultaneously. This helps reduce overall test execution time and improves performance. By default, Playwright runs tests in parallel across multiple workers based on the system's CPU cores. Each worker runs tests independently with its own browser instance, ensuring proper test isolation. This prevents conflicts between tests and improves reliability. Parallel execution is especially useful for large test suites. Playwright allows controlling the number of workers through configuration or CLI options. This feature is essential for scaling automation in CI/CD pipelines.

Example Test:

```
import { test, expect } from '@playwright/test';
test('Test 1', async ({ page }) => {
await page.goto('https://example.com');
});
```

```
test('Test 2', async ({ page }) => {
await page.goto('https://example.com/about');
});
```

Configuration Example

```
// playwright.config.ts
export default {
  workers: 3
};
```

CLI Example

```
npx playwright test --workers=3
```

How It Works:

- Multiple workers are created
- Each worker runs tests independently
- Tests execute simultaneously
- Results are combined at the end

HTML / JSON Reports:

HTML Reports: The Playwright Test Runner provides built-in support for generating HTML reports after test execution. The HTML report gives a clear UI view of test results, including passed and failed tests, execution time, and error details. It is mainly used for manual analysis and debugging. The report is generated automatically when tests are executed and can be opened in the browser.

Configuration Example: [playwright.config.ts]

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  reporter: [['html']]
});
```

How to Run Using Test Runner

▶ Run tests: npx playwright test

How to Open Report

▶ npx playwright show-report

Sample Output:

Running 1 test using 1 worker

✓ Test Passed (2s)

1 passed (2s)

Report File Generated:

HTML Report → playwright-report/index.html

How It Works:

• CLI command triggers Playwright Test Runner

• Test execution starts

• Results are collected
• HTML report is generated (playwright-report/)

JSON Reports: The Playwright Test Runner also supports JSON reports for storing execution results in structured format. This report is mainly used for integrations with CI/CD pipelines or external tools. It contains details like test status, duration, and execution data. The JSON file is generated automatically based on configuration.

Configuration Example: [playwright.config.ts]

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  reporter: [['json', { outputFile: 'test-results.json' }]]
});
```

How to Run Using Test Runner

▶ Run tests: npx playwright test

How to Open Report

▶ Open in VS Code

test-results.json → Right click → Open

Sample JSON Report:

```
{
  "status": "passed",
  "tests": [
    {
      "title": "Test Passed",
      "duration": 2000
    }   ]    }
```

Report File Generated:

• JSON Report → test-results.json

How It Works:

• CLI command triggers Playwright Test Runner
• Test execution starts
• Results are collected
• JSON report file is generated (test-results.json)

- page.locator('#id') → identify an element using its ID property
- page.locator('tagname') → identify an element using its Tagname
- page.locator('.classname') → identify an element using its classname
- page.locator('[attribute="value"]') → identify using its attribute
- page.locator('#id.classnametagname') → combining multiple locators
- page.locator('xpath="value") → identify using its xpath
- page.locator(':text("desiredtext")') → identify using some portion of text
- page.locator(':has-text("desired text")') → identify using some portion of text
- page.locator(':text-is("exact text")') → identify using exact text